



**Universidad Privada del Valle
Facultad de Informática y Electrónica
Carrera de Licenciatura en Ingeniería de Sistemas**

**MÓDULO DE INTELIGENCIA PREDICTIVA DEL ERP ISI MUSTANG
BASADO EN EL APROVECHAMIENTO DE DATOS HISTÓRICOS
OPERATIVOS PARA ANTICIPAR DESVIACIONES Y RIESGOS EN
LA GESTIÓN DE PROYECTOS DE INGENIERÍA EPC**

**PROYECTO DE GRADO PARA OPTAR
AL TÍTULO DE
LICENCIATURA EN INGENIERÍA DE SISTEMAS**

**POSTULANTE: FRANCO JAVIER TORREZ ALVARADO
TUTOR: ROLANDO LARA SANCHEZ**

Santa Cruz – Bolivia

2026

AGRADECIMIENTOS

Pendiente de redacción.

RESUMEN

Pendiente de redacción.

Palabras clave:

ABSTRACT

Pending.

Keywords:

ÍNDICE DE CONTENIDO

INTRODUCCIÓN	1
CAPÍTULO I: MARCO GENERAL	2
1.1. ANTECEDENTES	2
1.2. PLANTEAMIENTO DEL PROBLEMA.	4
1.2.1. FORMULACIÓN DEL PROBLEMA	8
1.3. OBJETIVOS	8
1.3.1. OBJETIVO GENERAL.	9
1.3.2. OBJETIVOS ESPECÍFICOS.	9
1.4. JUSTIFICACIÓN	9
1.4.1. JUSTIFICACIÓN TÉCNICA	10
1.4.2. JUSTIFICACIÓN ECONÓMICA.	11
1.4.3. JUSTIFICACIÓN SOCIAL	11
1.5. METODOLOGÍA.	12
1.5.1. TIPO DE INVESTIGACIÓN	12
1.5.2. MÉTODO DE INVESTIGACIÓN	12
1.5.2.1 Análisis documental	12
1.5.2.2 Histórico-lógico	13
1.5.2.3 Modelación.	13
1.5.2.4 Enfoque de sistemas	13
1.5.3. TÉCNICAS E INSTRUMENTOS DE INVESTIGACIÓN	14
1.5.3.1 Análisis de datos	14
1.5.3.2 Entrevista	14
1.5.3.3 Pruebas de los modelos predictivos.	14
1.5.4. POBLACIÓN Y MUESTRA.	15
1.5.4.1 Análisis de datos	15
1.5.4.2 Entrevista	15
1.6. DELIMITACIÓN DE LA INVESTIGACIÓN	15
1.6.1. DELIMITACIÓN TEMÁTICA	16
1.6.2. DELIMITACIÓN ESPACIAL	16
1.6.3. DELIMITACIÓN TEMPORAL	17
CAPÍTULO II: MARCO TEÓRICO Y TECNOLÓGICO	18
2.1. SISTEMAS ERP.	18
2.1.1. DEFINICIÓN Y CARACTERÍSTICAS	18
2.1.2. ERP EN EMPRESAS DE INGENIERÍA.	19
2.1.3. ERP EN EL SECTOR DE PETRÓLEO, GAS Y MINERÍA.	21

2.2.	APRENDIZAJE AUTOMÁTICO	22
2.2.1.	TIPOS DE APRENDIZAJE	22
2.2.2.	APRENDIZAJE SUPERVISADO	23
2.2.3.	ALGORITMOS: GRADIENT BOOSTING Y XGBOOST	26
2.2.4.	FEATURE ENGINEERING	28
2.3.	INGENIERÍA DE DATOS	29
2.3.1.	PROCESOS ETL Y ELT	29
2.3.2.	ARQUITECTURA MEDALLION (BRONZE, SILVER, GOLD).	31
2.3.3.	CALIDAD DE DATOS	33
2.4.	ANALÍTICA PREDICTIVA.	35
2.4.1.	DEFINICIÓN Y APLICACIONES	35
2.4.2.	ANALÍTICA PREDICTIVA EN GESTIÓN DE PROYECTOS	36
2.4.3.	PREDICCIÓN DE DESVIACIONES Y COSTOS.	38
2.5.	GESTIÓN DE PROYECTOS EPC	40
2.5.1.	DEFINICIÓN Y CARACTERÍSTICAS	40
2.5.2.	CERTIFICACIONES Y CONTROL DE AVANCE	41
2.5.3.	DESVIACIONES PRESUPUESTARIAS Y DE CRONOGRAMA.	43
2.6.	INGENIERÍA DE SOFTWARE	45
2.6.1.	PROCESO DE DESARROLLO DE SOFTWARE	45
2.6.2.	METODOLOGÍA DE DESARROLLO ITERATIVA E INCREMENTAL	47
2.6.3.	ARQUITECTURA DE SOFTWARE POR CAPAS	48
2.7.	TECNOLOGÍAS DEL STACK DE DESARROLLO.	50
2.7.1.	PYTHON Y SU ECOSISTEMA ML.	50
2.7.2.	DUCKDB Y POLARS	51
2.7.3.	FASTAPI COMO MICROSERVICIO	52
2.7.4.	NESTJS Y ANGULAR.	53
2.8.	GESTIÓN DE IDENTIDAD Y ACCESO.	56
2.8.1.	OAuth 2.0 Y OPENID CONNECT	56
2.8.2.	KEYCLOAK.	59
2.9.	MODEL CONTEXT PROTOCOL (MCP)	61
2.9.1.	DEFINICIÓN Y ARQUITECTURA	61
2.9.2.	INTEGRACIÓN DE SISTEMAS EMPRESARIALES CON MODELOS DE LENGUAJE	63
2.10.	INTELIGENCIA DE NEGOCIOS	65
2.10.1.	DEFINICIÓN Y REPORTE DESCRIPTIVOS	65
2.10.2.	POWER BI	67
	CAPÍTULO III: ANÁLISIS Y DISEÑO DEL SISTEMA PROPUESTO	69
3.1.	ANÁLISIS DE DATOS HISTÓRICOS	69
3.1.1.	EXPLORACIÓN Y CARACTERIZACIÓN DE LAS FUENTES DE DATOS	69
3.1.2.	IDENTIFICACIÓN DE VARIABLES PREDICTORAS	72

3.1.3.	DEFINICIÓN DE CASOS DE USO PREDICTIVOS	74
3.1.3.1	Desviación en certificaciones	75
3.1.3.2	Sobrecosto	75
3.1.3.3	Retraso en la fecha de cierre	76
3.2.	ANÁLISIS DE PROCESOS DEL PORTAL ANTERIOR	77
3.2.1.	RELEVAMIENTO DE PROCESOS DEL PORTAL ANTERIOR	77
3.2.2.	PROCESOS A REIMPLEMENTAR EN EL ERP	80
3.2.3.	DEFECTOS Y BRECHAS QUE AFECTAN LA CALIDAD DE LOS DATOS	80
3.3.	REQUERIMIENTOS DEL SISTEMA	82
3.3.1.	REQUERIMIENTOS FUNCIONALES	83
3.3.2.	REQUERIMIENTOS NO FUNCIONALES	88
3.4.	DISEÑO DE LA ARQUITECTURA	90
3.4.1.	ARQUITECTURA POR CAPAS	90
3.4.2.	DISEÑO DEL PIPELINE DE DATOS	93
3.4.2.1	Bronze	93
3.4.2.2	Silver	94
3.4.2.3	Gold	95
3.4.3.	DISEÑO DE MODELOS ML	96
3.4.4.	DISEÑO DE LA API PREDICTIVA	98
3.4.5.	DISEÑO DEL DASHBOARD	99
3.4.6.	DISEÑO DE LA AUTENTICACIÓN CENTRALIZADA	100
3.4.7.	DISEÑO DE LAS INTEGRACIONES CON MCP Y POWER BI	102
3.4.7.1	Power BI	102
3.4.7.2	Servidor MCP	103
	CAPÍTULO IV: CONSTRUCCIÓN E IMPLEMENTACIÓN DEL SISTEMA	105
4.1.	INCREMENTO 1: PROCESOS DEL PORTAL ANTERIOR MIGRADOS AL ERP	105
4.2.	INCREMENTO 2: AUTENTICACIÓN CENTRALIZADA CON KEYCLOAK	105
4.3.	INCREMENTO 3: PIPELINE DE DATOS Y ENTRENAMIENTO DE MODELOS	105
4.3.1.	INTEGRACIÓN DUCKDB CON MYSQL Y POSTGRESQL	105
4.3.2.	TRANSFORMACIÓN DE DATOS CON POLARS Y FEATURE ENGINEERING .	105
4.3.3.	MODELO DE DESVIACIÓN EN CERTIFICACIONES	105
4.3.4.	MODELO DE SOBRECOSTO DE PROYECTOS	105
4.3.5.	MODELO DE RETRASO EN LA FECHA DE CIERRE	105
4.4.	INCREMENTO 4: MICROSERVICIO FASTAPI E INTEGRACIÓN CON EL ERP	105
4.4.1.	DESARROLLO DEL MICROSERVICIO FASTAPI	105
4.4.2.	ENDPOINTS DE PREDICCIÓN EN NESTJS	105
4.4.3.	DASHBOARD DE INTELIGENCIA PREDICTIVA EN ANGULAR	106
4.5.	INCREMENTO 5: INTEGRACIÓN CON POWER BI Y MCP	106
4.5.1.	API DE DATOS PARA POWER BI	106
4.5.2.	SERVIDOR MCP	106

4.6. EVALUACIÓN Y VALIDACIÓN	106
4.6.1. MÉTRICAS DE RENDIMIENTO DE MODELOS	106
4.6.2. PRUEBAS CON USUARIOS CLAVE	106
CONCLUSIONES	107
RECOMENDACIONES	108
REFERENCIAS BIBLIOGRÁFICAS	109

ÍNDICE DE FIGURAS

Figura 1.1. Diagrama de Ishikawa: causas de la gestión reactiva de riesgos en los proyectos EPC de ISI Mustang	6
Figura 1.2. Árbol del problema: causas y efectos de la gestión reactiva de desviaciones y riesgos en los proyectos de ISI Mustang	8
Figura 3.1. Centros de costo cerrados por año de inicio.	71
Figura 3.2. Arquitectura por capas del sistema.	91
Figura 3.3. Diagrama de contexto (C4, nivel 1).	92
Figura 3.4. Diagrama de contenedores (C4, nivel 2). En naranja, los contenedores nuevos; FastAPI y el servidor MCP también validan tokens de Keycloak.	93

ÍNDICE DE TABLAS

Tabla 3.1. Tablas del portal anterior usadas en el análisis.	70
Tabla 3.2. Variables predictoras candidatas	73
Tabla 3.3. Correlación de rangos de tres señales con el desenlace, por banda de avance	74
Tabla 3.4. Casos de uso predictivos y tamaño de sus conjuntos de datos	76

ÍNDICE DE CUADROS

Cuadro 1.1. Análisis FODA de ISI Mustang Bolivia respecto a la explotación predictiva de los datos de su ERP	7
Cuadro 2.1. Métricas de evaluación de modelos supervisados y su uso en el proyecto . .	25
Cuadro 2.2. Capas de la arquitectura <i>medallion</i> y su correspondencia en el proyecto . . .	32
Cuadro 2.3. Dimensiones de calidad de datos adoptadas en el proyecto	34
Cuadro 2.4. Estudios revisados sobre predicción de desviaciones, costos y plazos en proyectos	39
Cuadro 2.5. Variaciones, índices y pronóstico de la gestión del valor ganado.	42
Cuadro 2.6. Modelos de proceso de software comparados.	46
Cuadro 2.7. Tecnologías del sistema por capa de la arquitectura.	55
Cuadro 2.8. Roles y flujos de OAuth 2.0 / OpenID Connect y su uso en el proyecto	59
Cuadro 2.9. Primitivas del servidor MCP y su uso previsto en el proyecto	63
Cuadro 2.10. Herramientas de BI consideradas y su forma de acceso a los datos del ERP	68
Cuadro 3.1. Calidad del histórico del portal anterior por dimensión	72
Cuadro 3.2. Procesos operativos del portal anterior y su estado en el ERP nuevo	78
Cuadro 3.3. Procesos de gestión de proyectos y estructura del portal anterior y su estado en el ERP nuevo.	79
Cuadro 3.4. Procesos a reimplementar en el ERP nuevo.	80
Cuadro 3.5. Defectos que afectan los datos del módulo predictivo.	82
Cuadro 3.6. Requerimientos funcionales: procesos y calidad de datos del ERP	84
Cuadro 3.7. Requerimientos funcionales: autenticación centralizada	85
Cuadro 3.8. Requerimientos funcionales: pipeline de datos y modelos	86
Cuadro 3.9. Requerimientos funcionales: servicio de predicción y tablero	87
Cuadro 3.10. Requerimientos funcionales: integraciones con Power BI y MCP	88
Cuadro 3.11. Requerimientos no funcionales: seguridad y datos	89
Cuadro 3.12. Requerimientos no funcionales: operación del sistema	90
Cuadro 3.13. Correspondencia de entidades en la capa Silver	95
Cuadro 3.14. Modelos del módulo predictivo	97
Cuadro 3.15. Endpoints del servicio de predicción	98
Cuadro 3.16. Vistas del tablero de predicciones.	99
Cuadro 3.17. Clientes registrados en Keycloak	101
Cuadro 3.18. Endpoints de la API de datos descriptivos	103
Cuadro 3.19. Herramientas del servidor MCP	104

LISTA DE SIGLAS Y ABREVIATURAS

AC: Costo real (*Actual Cost*)
API: Interfaz de programación de aplicaciones (*Application Programming Interface*)
ASGI: Interfaz asíncrona de pasarela de servidor (*Asynchronous Server Gateway Interface*)
BAC: Presupuesto a la conclusión (*Budget at Completion*)
BI: Inteligencia de negocios (*Business Intelligence*)
CPI: Índice de desempeño del costo (*Cost Performance Index*)
CV: Variación del costo (*Cost Variance*)
DAX: Expresiones de análisis de datos (*Data Analysis Expressions*)
EAC: Estimación a la conclusión (*Estimate at Completion*)
ELT: Extracción, carga y transformación (*Extract, Load, Transform*)
EPC: Ingeniería, procura y construcción (*Engineering, Procurement and Construction*)
ERP: Planificación de recursos empresariales (*Enterprise Resource Planning*)
ETL: Extracción, transformación y carga (*Extract, Transform, Load*)
EV: Valor ganado (*Earned Value*)
EVM: Gestión del valor ganado (*Earned Value Management*)
FIDIC: Federación Internacional de Ingenieros Consultores
HTTP: Protocolo de transferencia de hipertexto (*Hypertext Transfer Protocol*)
HTTPS: HTTP seguro (*Hypertext Transfer Protocol Secure*)
IA: Inteligencia artificial
JSON: Notación de objetos de JavaScript (*JavaScript Object Notation*)
JWT: Token web JSON (*JSON Web Token*)
KPI: Indicador clave de desempeño (*Key Performance Indicator*)
LDAP: Protocolo ligero de acceso a directorios (*Lightweight Directory Access Protocol*)
LLM: Modelo de lenguaje de gran tamaño (*Large Language Model*)
LSTM: Memoria de corto y largo plazo (*Long Short-Term Memory*)
MAE: Error absoluto medio (*Mean Absolute Error*)
MAPE: Error porcentual absoluto medio (*Mean Absolute Percentage Error*)
MCP: Protocolo de contexto de modelo (*Model Context Protocol*)
MSE: Error cuadrático medio (*Mean Squared Error*)
OE: Objetivo específico
OIDC: OpenID Connect
OLAP: Procesamiento analítico en línea (*Online Analytical Processing*)
OLTP: Procesamiento de transacciones en línea (*Online Transaction Processing*)
OP: Proveedor de OpenID (*OpenID Provider*)
ORM: Mapeo objeto-relacional (*Object-Relational Mapping*)
PKCE: Clave de prueba para el intercambio de código (*Proof Key for Code Exchange*)
PMBOK: Guía de los fundamentos para la dirección de proyectos (*Project Management Body of Knowledge*)
PV: Valor planificado (*Planned Value*)
RAG: Generación aumentada por recuperación (*Retrieval-Augmented Generation*)
REQ: Relevamiento de requerimientos del ERP de ISI Mustang
REST: Transferencia de estado representacional (*Representational State Transfer*)
RF: Requerimiento funcional
RMSE: Raíz del error cuadrático medio (*Root Mean Squared Error*)

RNF: Requerimiento no funcional

ROPC: Credenciales de contraseña del propietario del recurso (*Resource Owner Password Credentials*)

RRHH: Recursos humanos

SAML: Lenguaje de marcado para confirmaciones de seguridad (*Security Assertion Markup Language*)

SHAP: Explicaciones aditivas de Shapley (*SHapley Additive exPlanations*)

SPI: Índice de desempeño del cronograma (*Schedule Performance Index*)

SQL: Lenguaje de consulta estructurado (*Structured Query Language*)

SSO: Inicio de sesión único (*Single Sign-On*)

SSR: Renderizado del lado del servidor (*Server-Side Rendering*)

SV: Variación del cronograma (*Schedule Variance*)

TLS: Seguridad de la capa de transporte (*Transport Layer Security*)

URL: Localizador uniforme de recursos (*Uniform Resource Locator*)

WSGI: Interfaz de pasarela de servidor web (*Web Server Gateway Interface*)

INTRODUCCIÓN

CAPÍTULO I

MARCO GENERAL

1.1. ANTECEDENTES

Los sistemas de planificación de recursos empresariales (ERP) constituyen el núcleo operativo de las empresas de ingeniería: registran de forma continua los datos de proyectos, certificaciones, recursos humanos, horas trabajadas, requisiciones y compras que sostienen la operación diaria. Sin embargo, como señalan Jawad y Villányi (2024), la mayoría de las organizaciones utilizan estos sistemas principalmente para la gestión operativa y la generación de reportes descriptivos, sin explotar el potencial predictivo de los datos históricos que acumulan. En el ámbito de la gestión de proyectos de ingeniería, el aprendizaje automático ha demostrado superar en precisión a los métodos tradicionales de planificación y control: Shamim et al. (2025) concluyen, a partir de una revisión sistemática de 39 estudios publicados entre 2016 y 2024, que los modelos de inteligencia artificial mejoran significativamente la estimación de costos frente a los métodos convencionales, mientras que Ottaviani et al. (2025) proponen pipelines automatizados capaces de predecir costos y duración de proyectos con mayor exactitud que las herramientas basadas en la gestión del valor ganado (*Earned Value Management*).

En cuanto a trabajos que abordan problemas análogos al planteado, Nguyen et al. (2026) desarrollaron un modelo de XGBoost explicable para predecir sobrecostos en proyectos de construcción a partir de variables estructuradas de proyecto; no obstante, su estudio se sustenta en un conjunto de mil observaciones simuladas y no contempla la integración del modelo en el sistema de información operativo de una empresa. İnan et al. (2022) reportan que los modelos de aprendizaje automático entrenados sobre datos históricos de proyectos producen estimaciones de costo más precisas que los índices tradicionales del valor ganado. Por su parte, Ottaviani et al. (2025) evidencian que la integración de múltiples fuentes de datos mejora la precisión predictiva; aun así, su pipeline opera sobre datos consolidados de diversas organizaciones y no sobre el historial operativo de una empresa en particular. En el terreno de los sistemas ERP del sector, Al Jafari y Nair (2018) analizan, mediante un estudio de caso con enfoque mixto en el Sultanato de Omán, cómo la implementación del ERP influye en la eficiencia y la productividad de las empresas de petróleo y gas, y concluyen recomendando su adopción; su trabajo se ocupa del ERP como registro de la operación y no de la explotación predictiva de los datos que acumula.

Los antecedentes revisados confirman la viabilidad técnica de aplicar aprendizaje automático para anticipar desviaciones y riesgos en proyectos de ingeniería, pero comparten una limitación común: se desarrollan sobre conjuntos de datos públicos o multiorganizacionales y no se integran en el sistema de gestión operativo de la empresa que origina los datos. Tampoco se identificaron antecedentes de aplicación de inteligencia predictiva sobre los datos del ERP de ISI Mustang ni trabajos similares en el contexto de empresas de ingeniería bolivianas. Esta doble brecha, la ausencia de soluciones integradas al ERP y la falta de precedentes en el contexto local, constituye el punto de partida del presente proyecto de grado.

ISI Mustang Bolivia es una empresa con más de 25 años de operación en ingeniería y servicios tecnológicos para los sectores de gas y petróleo, energía y minería, con sede en Santa Cruz de la Sierra. Ejecuta proyectos integrales de tipo EPC (*Engineering, Procurement and Construction*), modalidad en la que asume de forma completa el diseño de ingeniería, la procura de equipos y materiales, y la construcción de las instalaciones. Para planificar y controlar estos proyectos, la empresa se ha apoyado durante más de una década en un sistema de información propio, el Portal ISIMustang Bolivia, desarrollado en PHP sobre una base de datos MySQL 5.7 cuyo esquema comprende 175 tablas. En ese sistema se encuentra el histórico de más de diez años de operación: proyectos y centros de costo, actividades, facturación, horas trabajadas, requisiciones y compras. De forma complementaria, parte del control de proyectos se lleva en planillas Excel fuera del sistema.

En la actualidad la empresa reconstruye ese portal como un ERP nuevo, de desarrollo propio y en curso, con un backend en NestJS y TypeScript sobre PostgreSQL y un frontend en Angular. El ERP nuevo reimplementa progresivamente los módulos del portal anterior y está próximo a entrar en producción. El paso de un sistema a otro se hará mediante un corte, sin migración de la base de datos: lo vigente, como los contratos y los proyectos activos, se registrará de nuevo en el ERP nuevo, que desde entonces será el sistema de los proyectos activos, mientras que los centros de costo cerrados y el resto del histórico permanecerán solo en el portal anterior. Esta coexistencia de dos sistemas, uno con el pasado y otro con el presente de la operación, define el escenario técnico del presente trabajo: los datos para aprender están en un esquema y los proyectos sobre los que se quiere anticipar riesgos en otro.

Se parte de la hipótesis de que ese histórico contiene un activo de especial valor para el aprendizaje automático: el registro del avance facturado de cada proyecto frente a lo planificado, junto con las horas, compras y presupuestos asociados, que permitiría reconstruir las desviaciones ocurridas y, en la medida en que las causas hayan sido registradas de forma consistente, conformar un conjunto de datos etiquetado para aprendizaje supervisado. La confirmación de esa hipótesis corresponde al análisis de datos del Capítulo III. Nguyen et al. (2026) muestran, sobre datos simulados, el uso de XGBoost para predecir sobrecostos a partir de variables estructuradas de proyecto, lo que sugiere la viabilidad de aprovechar el historial de ISI Mustang para anticipar desviaciones y riesgos operativos. El presente trabajo propone, en consecuencia, el desarrollo de un módulo de inteligencia predictiva integrado al ERP de ISI Mustang que, mediante técnicas de aprendizaje automático aplicadas sobre ese histórico, permita anticipar desviaciones y riesgos en la gestión de sus proyectos EPC, en el marco de una modernización del ERP que incorpora autenticación centralizada, integración con herramientas de inteligencia de negocios y exposición de sus datos mediante el protocolo MCP.

1.2. PLANTEAMIENTO DEL PROBLEMA

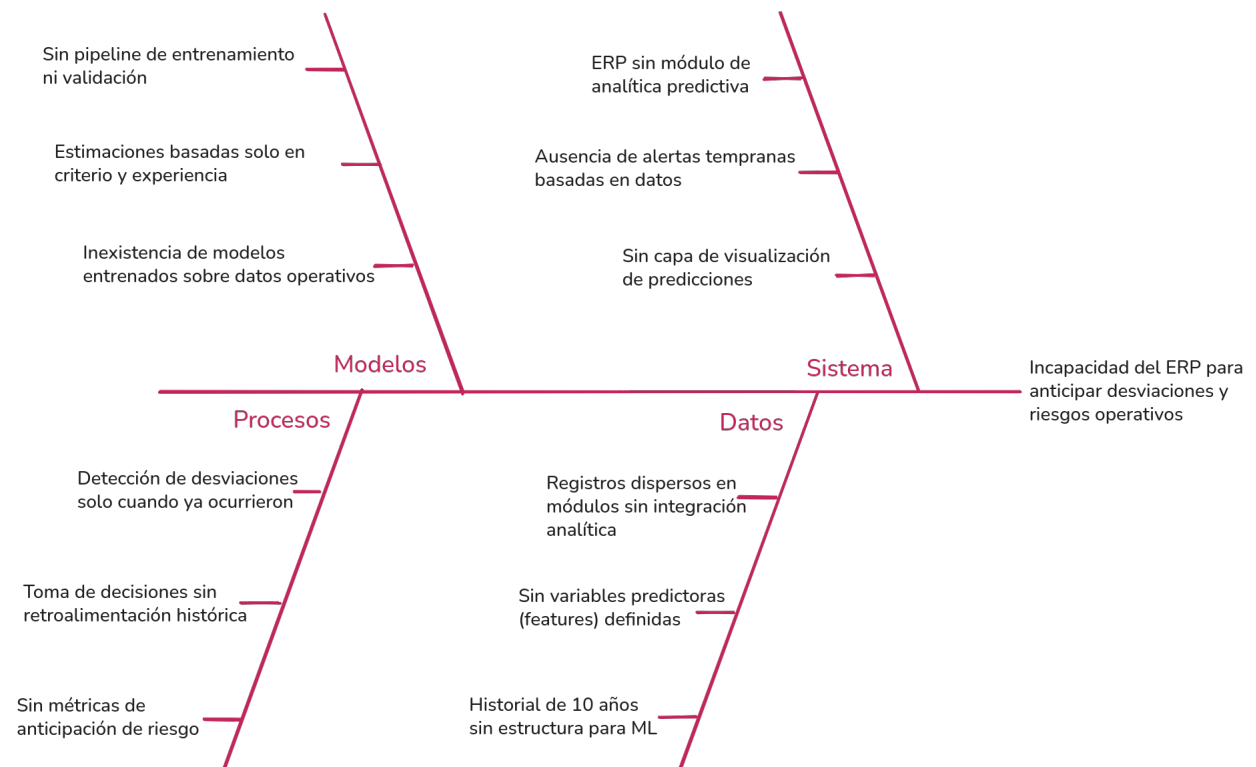
Los proyectos EPC que ejecuta ISI Mustang para los sectores de gas, petróleo, minería y energía involucran alta complejidad técnica, presupuestos considerables y plazos estrictos. Para planificarlos y controlarlos, avances, certificaciones, recursos, compras y horas trabajadas, la empresa se apoya en su sistema de información, que acumula más de diez años de datos históricos operativos. Sin embargo, dicha información se utiliza únicamente con fines descriptivos y de control inmediato: sirve para saber qué ocurrió y cuánto se ejecutó, pero no existe un mecanismo que permita identificar patrones en el historial ni anticipar comportamientos futuros. Como consecuencia, la detección de desviaciones presupuestarias, retrasos y cuellos de botella ocurre de forma reactiva, una vez que la desviación ya se materializó y su corrección resulta más costosa, lo que limita la capacidad de gestión proactiva de riesgos en proyectos de alta envergadura. La literatura del sector confirma que se trata de un problema recurrente: Habibi et al. (2018), a partir de una revisión de más de doscientos artículos sobre proyectos EPC, señalan que más de la mitad de los proyectos enfrenta retrasos significativos y escaladas de costo importantes,

identifican el cambio de diseño como la causa más común de retraso y sobrecosto, seguida de la escasez de recursos y la fluctuación de precios, y observan que todo retraso termina traducéndose en sobrecosto.

A la gestión reactiva se suma la situación del propio sistema de información. El histórico reside en el portal anterior, cuyos registros están dispersos en módulos sin integración analítica y cuya calidad depende de la disciplina de carga de cada usuario; el ERP nuevo, en construcción, todavía no cubre todos los procesos del portal anterior y presenta defectos que ensucian los datos que registra; y ninguno de los dos cuenta con un módulo de analítica predictiva, una capa de visualización de predicciones ni un mecanismo de alertas tempranas basado en datos. Anticipar riesgos exige, por tanto, no solo construir modelos, sino también preparar el ERP para que sus datos y sus integraciones lo permitan.

El diagrama de Ishikawa de la Figura 1.1 sintetiza las causas de esta problemática agrupadas en cuatro categorías. En la categoría de *procesos*, la detección de desviaciones solo cuando ya ocurrieron, la toma de decisiones sin retroalimentación histórica y la ausencia de métricas de anticipación de riesgo. En la de *datos*, los registros dispersos en módulos sin integración analítica, la inexistencia de variables predictoras definidas y un historial de diez años sin estructura para el aprendizaje automático. En la de *modelos*, la ausencia de un pipeline de entrenamiento y validación, las estimaciones basadas únicamente en criterio y experiencia, y la inexistencia de modelos entrenados sobre datos operativos. Finalmente, en la de *sistema*, un ERP sin módulo de analítica predictiva, sin alertas tempranas basadas en datos y sin una capa de visualización de predicciones. La figura recoge las causas relevadas en la etapa de perfil; el análisis posterior de los dos sistemas de información añadió, en la categoría de datos, la coexistencia del histórico y de los proyectos activos en esquemas distintos y los defectos de registro del ERP nuevo, causa que se incorpora al árbol del problema de la Figura 1.2.

Figura 1.1. Diagrama de Ishikawa: causas de la gestión reactiva de riesgos en los proyectos EPC de ISI Mustang



Fuente: Elaboración propia, 2026.

El análisis FODA del Cuadro 1.1 resume la situación de la empresa frente a la explotación predictiva de sus datos. Entre las fortalezas destacan el volumen del histórico acumulado y el control total sobre el sistema de información, al tratarse de un desarrollo propio; entre las debilidades, las posibles inconsistencias del registro histórico y la dependencia de la disciplina de carga. Las oportunidades provienen de la demanda de gestión proactiva en el sector energético y de la madurez de las técnicas de análisis predictivo, mientras que las amenazas se concentran en la adopción por parte de los usuarios y en la vigencia de los modelos ante cambios en los procesos.

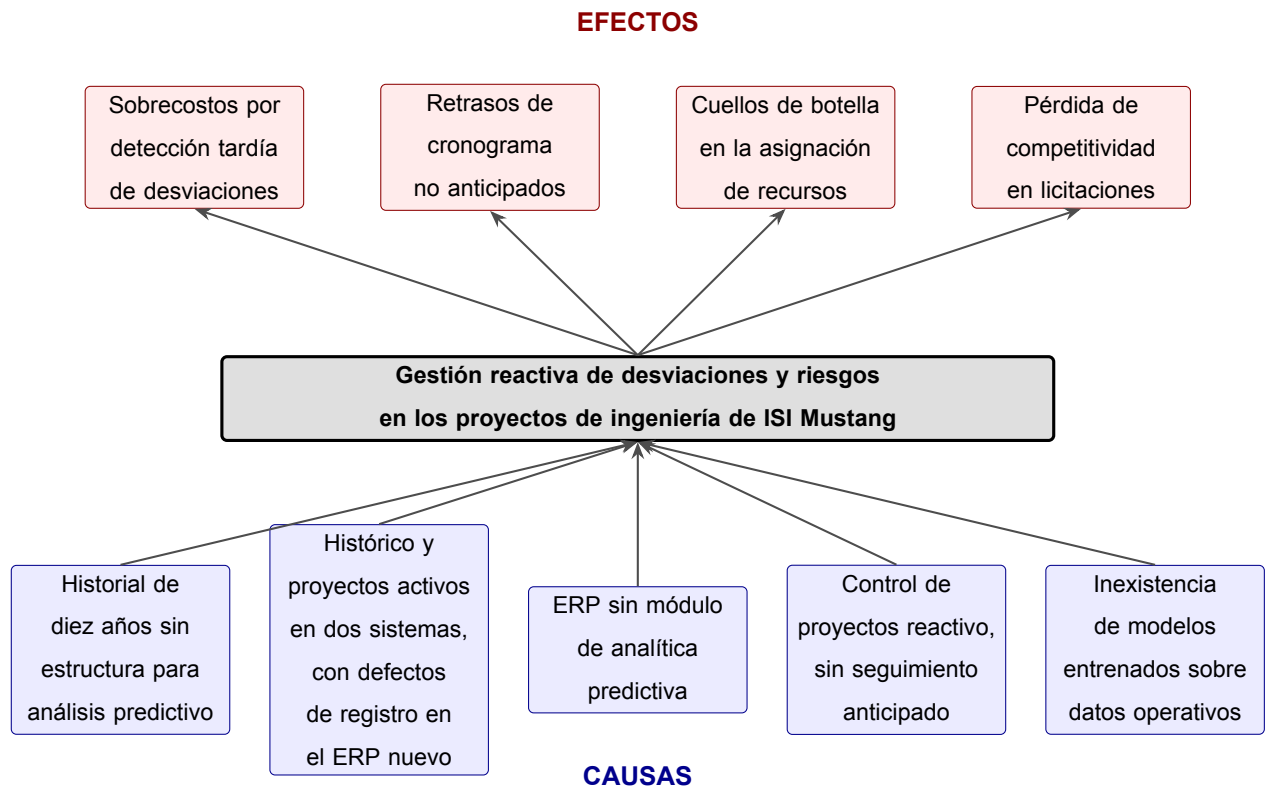
Cuadro 1.1. Análisis FODA de ISI Mustang Bolivia respecto a la explotación predictiva de los datos de su ERP

Fortalezas	Oportunidades
• Más de diez años de datos históricos operativos acumulados en el portal anterior. • Registro del avance facturado frente a lo planificado que, de confirmarse en el análisis de datos, permitiría reconstruir las desviaciones ocurridas. • Sistema de información propio, con control total sobre el código y los datos operativos.	• Alta demanda de gestión proactiva en proyectos de ingeniería del sector energético. • Madurez y disponibilidad de técnicas de aprendizaje automático para datos tabulares. • Mejora del posicionamiento competitivo en la estimación y adjudicación de contratos.
Debilidades	Amenazas
• Posibles inconsistencias en el registro histórico de información. • Histórico y proyectos activos en dos sistemas con esquemas distintos. • Ausencia de una cultura de análisis de datos en la organización. • Dependencia de la disciplina en el registro operativo para la calidad de los resultados.	• Resistencia al cambio por parte de los usuarios ante nuevas herramientas de gestión. • Cambios en los procesos operativos que reduzcan la vigencia de los modelos desarrollados. • Baja adopción si los resultados no se comunican de forma clara a los usuarios clave.

Fuente: Elaboración propia, 2026.

El árbol del problema de la Figura 1.2 ordena esta situación en términos de causas y efectos. El problema central es la gestión reactiva de desviaciones y riesgos en los proyectos de ingeniería de ISI Mustang. Sus causas son un historial de diez años sin estructura para el análisis predictivo, la coexistencia del histórico y de los proyectos activos en dos sistemas con esquemas distintos y con defectos de registro en el ERP nuevo, un ERP sin módulo de analítica predictiva, un control de proyectos reactivo y sin seguimiento anticipado, y la inexistencia de modelos entrenados sobre los datos operativos de la empresa. Sus efectos son los sobrecostos por detección tardía de desviaciones, los retrasos de cronograma no anticipados, los cuellos de botella en la asignación de recursos y la pérdida de competitividad en licitaciones, dado que la estimación de nuevas propuestas no se beneficia de lo aprendido en los proyectos anteriores.

Figura 1.2. Árbol del problema: causas y efectos de la gestión reactiva de desviaciones y riesgos en los proyectos de ISI Mustang



Fuente: Elaboración propia, 2026.

El objeto de estudio del presente trabajo es, en consecuencia, el proceso de gestión y control de desviaciones y riesgos en los proyectos de ingeniería de ISI Mustang, a partir de los datos históricos registrados en su sistema de información.

1.2.1. FORMULACIÓN DEL PROBLEMA

¿Cómo aprovechar los datos históricos del ERP ISI Mustang para anticipar desviaciones y riesgos en los proyectos de ingeniería EPC de la empresa?

1.3. OBJETIVOS

1.3.1. OBJETIVO GENERAL

Desarrollar un módulo de inteligencia predictiva integrado al ERP ISI Mustang, mediante técnicas de aprendizaje automático aplicadas sobre los datos históricos operativos de la empresa, para anticipar desviaciones y riesgos en la gestión de proyectos de ingeniería EPC, en el marco de una modernización del ERP que incorpora autenticación centralizada, integración con herramientas de inteligencia de negocios y exposición de sus datos mediante el protocolo MCP.

1.3.2. OBJETIVOS ESPECÍFICOS

1. Identificar los procesos del portal anterior (Portal ISIMustang Bolivia) pendientes de reimplementar en el ERP y los defectos que afectan la calidad de sus datos operativos.
2. Analizar los datos históricos de proyectos registrados en el sistema legado y en las planillas de control complementarias, para la definición de los casos de uso del módulo predictivo.
3. Diseñar la arquitectura del módulo predictivo y de sus integraciones con el ERP, incluyendo la selección de los modelos de aprendizaje automático adecuados para cada caso de uso.
4. Implementar en el ERP los procesos identificados y la autenticación centralizada con Keycloak (OAuth 2.0 / OpenID Connect), como base para las integraciones del sistema.
5. Construir el pipeline de datos y los modelos predictivos para los casos de uso definidos: desviación en certificaciones, sobrecosto de proyecto y retraso en la fecha de cierre.
6. Integrar al ERP el servicio de predicción con su dashboard de visualización, los reportes descriptivos en Power BI y el servidor MCP.
7. Evaluar la precisión de los modelos y la funcionalidad del sistema mediante métricas de rendimiento y pruebas con usuarios clave de ISI Mustang.

1.4. JUSTIFICACIÓN

1.4.1. JUSTIFICACIÓN TÉCNICA

El sistema de información de ISI Mustang acumula más de diez años de datos operativos distribuidos en múltiples módulos (proyectos y centros de costo, actividades, facturación, horas trabajadas, requisiciones y compras), complementados por las planillas de control que la empresa lleva fuera del sistema. Ese volumen es suficiente para el entrenamiento de modelos de aprendizaje automático sobre datos tabulares. En particular, el portal anterior calculaba indicadores de desvío de costo, de ingresos y de avance por proyecto a partir del avance facturado frente a lo planificado; se parte de la hipótesis de que los datos con los que se calculaban esos indicadores permiten reconstruir las desviaciones ocurridas en cada proyecto y, junto con las causas que los usuarios hayan documentado, conformar un conjunto de datos etiquetado para aprendizaje supervisado. La confirmación de esa hipótesis, tanto de la consistencia del registro de avance facturado frente a lo planificado como de la completitud con que se registraron las causas, corresponde a la exploración de datos que se desarrolla en el Capítulo III; si las causas no resultan aprovechables, la etiqueta se derivaría únicamente de la comparación entre lo planificado y lo ejecutado.

Las técnicas de aprendizaje automático supervisado, en particular los modelos de *gradient boosting* como XGBoost, son suficientemente maduras para aplicarse sobre este tipo de datos, como muestran los antecedentes revisados, y cuentan con implementaciones abiertas y ampliamente utilizadas en el ecosistema de Python. La ingeniería de datos dispone además de herramientas, como DuckDB y Polars, capaces de leer directamente bases MySQL y PostgreSQL y archivos de planilla, y de unificarlos en un pipeline analítico sin infraestructura especializada, lo que resuelve la principal dificultad técnica del caso: que el histórico y los proyectos activos residan en dos sistemas con esquemas distintos.

Por último, la integración del módulo predictivo al ERP es técnicamente viable porque el ERP es un desarrollo propio de la empresa. Ello permite exponer los datos y las predicciones mediante servicios con contratos bien definidos, incorporar la autenticación centralizada con Keycloak como base común de acceso para el ERP, el servicio de predicción, el servidor MCP y Power BI, y corregir en el propio ERP los defectos que degradan la calidad de los datos, algo que no sería posible sobre un paquete comercial cerrado.

1.4.2. JUSTIFICACIÓN ECONÓMICA

Los proyectos de tipo EPC que ejecuta ISI Mustang en los sectores de gas y petróleo, minería y energía involucran presupuestos de alta envergadura, en los que una desviación pequeña en términos porcentuales representa montos significativos. La detección temprana de desviaciones permite actuar cuando las medidas correctivas todavía son baratas, reasignar recursos, adelantar una compra o ajustar el cronograma, y reduce el costo de corrección frente a la detección tardía. La anticipación de cuellos de botella en la asignación de recursos evita sobre costos por sobreasignación o por tiempos improductivos. Asimismo, la capacidad de contrastar nuevas propuestas con el comportamiento de proyectos anteriores mejora la competitividad de la empresa en la estimación y adjudicación de contratos.

La solución se construye, además, sobre activos que la empresa ya posee: sus datos históricos, su ERP y su servidor propio. Las tecnologías del módulo predictivo (Python, DuckDB, Polars, XGBoost, FastAPI) y de la autenticación centralizada (Keycloak) son de código abierto; la única herramienta con licencia comercial es Power BI, cuya adopción para los reportes descriptivos ya está prevista por la empresa con independencia de este trabajo. La inversión se concentra, por tanto, en el desarrollo.

1.4.3. JUSTIFICACIÓN SOCIAL

Una gestión de proyectos más proactiva y basada en datos contribuye a entregas más confiables en sectores críticos para el desarrollo de Bolivia, como los hidrocarburos, la energía y la minería, cuyas instalaciones dependen de que los proyectos de ingeniería se completen en plazo y con la calidad comprometida. La reducción de la gestión reactiva disminuye la carga operativa y las urgencias de último momento de los equipos de proyecto, lo que se traduce en mejores condiciones de trabajo. A nivel más amplio, el proyecto muestra que una empresa boliviana puede capitalizar sus propios datos históricos mediante inteligencia artificial, con herramientas abiertas y sobre infraestructura existente, y deja un antecedente replicable para otras empresas de ingeniería del país.

1.5. METODOLOGÍA

1.5.1. TIPO DE INVESTIGACIÓN

El presente trabajo corresponde a una investigación aplicada, dado que utiliza conocimiento científico existente en el área del aprendizaje automático y de la ingeniería de software para dar solución a un problema concreto en el contexto operativo de ISI Mustang Bolivia. De acuerdo con sus fases, combina tres niveles de alcance: exploratorio, en la etapa inicial de comprensión y caracterización de los datos históricos del portal anterior y de las planillas de control; descriptivo, para identificar y caracterizar las variables operativas relevantes y los patrones de desviación; y correlacional, al modelar las relaciones entre dichas variables, como horas trabajadas, líneas de presupuesto, compras y avance certificado, con el fin de anticipar desviaciones y riesgos en los proyectos.

1.5.2. MÉTODO DE INVESTIGACIÓN

El trabajo adopta un enfoque mixto, dado que combina la recolección y el análisis de datos cuantitativos, los registros históricos del sistema de información, con datos cualitativos provenientes de las entrevistas a usuarios clave. Para operacionalizar este enfoque se emplean los siguientes métodos.

1.5.2.1 *Análisis documental*

Se aplica a la revisión de la literatura científica sobre aprendizaje automático, analítica predictiva, ingeniería de datos, sistemas ERP, gestión de identidad y protocolos de integración, con el fin de establecer los fundamentos teóricos del trabajo, y a la documentación del portal anterior y del ERP nuevo, para identificar los procesos pendientes de reimplementar y los defectos que afectan a los datos.

1.5.2.2 Histórico-lógico

Se aplica sobre los datos acumulados durante más de diez años de operación, con el fin de identificar patrones y tendencias en el comportamiento de los proyectos a lo largo del tiempo y de reconstruir, a partir de ellos, las desviaciones ocurridas.

1.5.2.3 Modelación

Se emplea para la construcción de los modelos de aprendizaje automático que representan el comportamiento de las variables operativas y generan las predicciones, y para el diseño de la arquitectura del sistema.

1.5.2.4 Enfoque de sistemas

Se utiliza para analizar el ERP de ISI Mustang como un sistema con entradas, procesos y salidas, identificando las interrelaciones entre sus módulos, los flujos de datos relevantes para el módulo predictivo y los puntos de integración con los componentes nuevos.

En cuanto al desarrollo del sistema, se adopta una metodología iterativa e incremental, en la que el producto se construye en ciclos sucesivos que agregan funcionalidad de manera progresiva y permiten validar tempranamente cada entrega con los usuarios de la empresa. El trabajo se organiza en cinco incrementos: la reimplementación en el ERP de los procesos del portal anterior; la autenticación centralizada con Keycloak; el pipeline de datos y el entrenamiento de los modelos; el microservicio de predicción y su integración con el ERP mediante el dashboard; y la integración con Power BI y el servidor MCP. Cada incremento comprende la definición de su alcance, el análisis, el diseño, el desarrollo y la validación, y se documenta en el Capítulo IV. Los fundamentos de esta metodología se desarrollan en el Capítulo II.

1.5.3. TÉCNICAS E INSTRUMENTOS DE INVESTIGACIÓN

Se emplean tres técnicas, cada una con sus instrumentos.

1.5.3.1 *Análisis de datos*

Se aplica sobre los registros históricos del portal anterior y las planillas de control complementarias, con el propósito de caracterizar las fuentes, identificar variables predictoras, detectar patrones de comportamiento y construir los conjuntos de datos necesarios para el entrenamiento de los modelos. Sus instrumentos son los scripts de extracción y transformación de datos del pipeline y los cuadernos de análisis exploratorio (*notebooks*).

1.5.3.2 *Entrevista*

Se aplica a usuarios clave del ERP en ISI Mustang Bolivia en la etapa de evaluación y validación del sistema, con el fin de validar las predicciones generadas y recoger criterios de utilidad desde la perspectiva operativa; sus resultados se presentan en el Capítulo IV. Los requisitos funcionales del módulo, en cambio, no dependen de estas entrevistas: provienen del relevamiento de negocio ya documentado en la especificación del ERP, en particular de las preguntas del requerimiento registradas para cada proceso, y de la entrevista preliminar realizada en la etapa de perfil, y se presentan en el Capítulo III. El instrumento de las entrevistas de validación es una guía semiestructurada organizada en bloques: contexto del rol y gestión de proyectos; desviaciones y riesgos frecuentes; uso del sistema y datos disponibles; calidad y confiabilidad del registro; y utilidad y expectativas respecto al módulo predictivo.

1.5.3.3 *Pruebas de los modelos predictivos*

Se aplican sobre un conjunto de datos de prueba independiente del de entrenamiento para evaluar el desempeño de los modelos. Para los casos de clasificación se emplean la exactitud, la

precisión, la sensibilidad, el F1 y la matriz de confusión; para los casos de regresión, la raíz del error cuadrático medio (RMSE), el error absoluto medio (MAE) y el coeficiente de determinación (R^2). Adicionalmente se aplica validación cruzada para verificar la estabilidad de los resultados. Sus instrumentos son el cuadro de métricas de evaluación y las rutinas de validación cruzada.

1.5.4. POBLACIÓN Y MUESTRA

1.5.4.1 *Análisis de datos*

La población corresponde a la totalidad de los registros históricos generados en el Portal ISIMustang Bolivia durante su período operativo y a las planillas de control de proyectos complementarias. Se trabaja con el conjunto completo de datos disponibles en los módulos relevantes para los casos de predicción definidos, sin muestreo, dado que el volumen es tratable en su totalidad por el pipeline analítico. Los proyectos activos registrados en el ERP nuevo constituyen la población sobre la que se realiza la inferencia.

1.5.4.2 *Entrevista*

La población está conformada por aproximadamente 20 usuarios del ERP en ISI Mustang Bolivia. La muestra es de 5 a 8 usuarios clave seleccionados mediante muestreo no probabilístico intencional, priorizando los perfiles con roles de jefe de proyecto, certificador y administrador del sistema, por ser quienes registran, controlan y consumen la información sobre la que opera el módulo predictivo.

1.6. DELIMITACIÓN DE LA INVESTIGACIÓN

1.6.1. DELIMITACIÓN TEMÁTICA

El presente trabajo se delimita a las áreas de aprendizaje automático supervisado, ingeniería de datos y analítica predictiva, aplicadas sobre los datos históricos operativos de ISI Mustang, y a la ingeniería de software aplicada a la modernización de su ERP: autenticación centralizada con Keycloak (OAuth 2.0 / OpenID Connect), integración con Power BI, exposición de datos mediante un servidor MCP y reimplementación en el ERP de los procesos del portal anterior. Los datos de entrenamiento provienen del sistema legado (Portal ISIMustang Bolivia, MySQL) y de las planillas de control de proyectos que se llevan fuera del sistema; la inferencia se realiza sobre los proyectos activos del ERP nuevo (PostgreSQL). El pipeline de datos es exclusivamente analítico: no migra datos al ERP. Los reportes en Power BI son descriptivos y no incorporan predicciones, que se presentan únicamente en el dashboard del ERP. En cuanto a la gestión de proyectos, el estudio se circunscribe a los proyectos de tipo EPC ejecutados por ISI Mustang.

Quedan fuera del alcance las fuentes de datos externas, como sensores IoT, datos de mercado o información climática; los módulos del sistema que no se encuentren activos o poblados con datos históricos suficientes; la migración del histórico del sistema legado al esquema del ERP nuevo; y el desarrollo de un asistente conversacional o agente de inteligencia artificial sobre el servidor MCP, que se plantea como trabajo futuro.

1.6.2. DELIMITACIÓN ESPACIAL

El estudio se desarrolla en las instalaciones de ISI Mustang Bolivia, con sede en Santa Cruz de la Sierra, Bolivia. Los datos históricos utilizados corresponden a proyectos ejecutados por la empresa principalmente en territorio boliviano. El módulo de inteligencia predictiva se integra y valida en el ERP nuevo de ISI Mustang en su sede central, desplegado en un servidor propio de la empresa; la migración a la nube se plantea como trabajo futuro.

1.6.3. DELIMITACIÓN TEMPORAL

Los datos históricos utilizados para el entrenamiento de los modelos abarcan un período de más de diez años de operación de ISI Mustang, registrados en el sistema legado y en las planillas de control complementarias. El desarrollo del proyecto de grado se inicia en abril de 2026 y comprende las etapas de análisis, diseño, construcción por incrementos y evaluación del sistema, con un cierre sujeto al calendario académico de la carrera.

En cuanto a la validez temporal de la investigación, dado que los modelos predictivos se entrenan sobre patrones históricos, se estima que la solución mantiene su vigencia mientras las condiciones operativas de la empresa permanezcan estables. Se recomienda un reentrenamiento periódico de los modelos, de forma anual o ante cambios sustanciales en los procesos de la empresa, con el fin de mitigar la pérdida de precisión por deriva de datos (*model drift*).

CAPÍTULO II

MARCO TEÓRICO Y TECNOLÓGICO

2.1. SISTEMAS ERP

2.1.1. DEFINICIÓN Y CARACTERÍSTICAS

Los sistemas de planificación de recursos empresariales (*enterprise resource planning*, ERP) no cuentan con una definición única. Klaus et al. (2000) encontraron visiones divergentes sobre el fenómeno y concluyeron que era poco probable alcanzar una definición ampliamente consensuada. Existe, sin embargo, un núcleo común. Davenport (1998) los describe como paquetes de software comerciales que prometen la integración fluida de toda la información que circula por una empresa: la financiera y contable, la de recursos humanos, la de la cadena de suministro y la de los clientes. Shi y Halpin (2003) lo formulan en términos operativos: un entorno integrado que permite articular las principales funciones de gestión sobre una única base de datos común, de modo que la información se comparta entre ellas.

Ese núcleo se explica por su origen. El ERP nació en la industria manufacturera (Shi & Halpin, 2003); Jacobs y Weston (2007) reconstruyen esa historia y el papel de los principales proveedores. Klaus et al. (2000) sitúan el origen del término en la planificación de requerimientos de materiales (MRP) y la planificación de recursos de manufactura (MRP II): cuando las empresas advirtieron que la rentabilidad y la satisfacción del cliente eran objetivos de toda la organización, la integración se extendió a finanzas, ventas, distribución y recursos humanos, y a esa solución integral se la llamó ERP.

De esa caracterización se desprenden tres rasgos. El primero es la integración: una sola base de datos elimina la duplicación y la inconsistencia entre sistemas heredados dispersos, el atractivo que Davenport (1998) identifica frente a sistemas incompatibles. El segundo es la organización en módulos funcionales que comparten esa base y pueden adoptarse de forma progresiva. El tercero es la estandarización de procesos: el paquete trae incorporada una forma de operar y, como advierte el mismo autor, impone su propia lógica sobre la estrategia, la organización y la cultura de la empresa, por lo que su adopción debe ser responsabilidad de la gerencia general y no solo de los tecnólogos.

Los beneficios y los riesgos están documentados. Shang y Seddon (2002), a partir de 233 casos y entrevistas con gerentes de 34 organizaciones, consolidan los beneficios en cinco dimensiones (operacional, gerencial, estratégica, de infraestructura de TI y organizacional) y los evalúan en perspectiva longitudinal, años después de la implementación. Del lado de los riesgos, Umble et al. (2003) describen al ERP como un sistema de información altamente complejo, cuya implementación es difícil y costosa, y señalan que muchas implementaciones han sido calificadas como fracasos por no alcanzar los objetivos corporativos predeterminados.

Frente al paquete comercial existe la alternativa del desarrollo propio. Olsen y Sætre (2007) sostienen que para muchas pequeñas y medianas empresas el software desarrollado en casa es una opción óptima: ofrece control total, flexibilidad y adaptación a requisitos específicos, con costos a menudo comparables a los de un ERP estándar. Cuando el software soporta el núcleo del negocio, el desarrollo propio otorga ventaja competitiva, mientras que las estructuras rígidas de los paquetes pueden amenazar la dinámica organizacional. Es la situación de ISI Mustang: sus procesos centrales (centros de costo, certificaciones por avance, requisiciones, viáticos) se apoyan en un ERP desarrollado a medida, que hoy se reconstruye sobre NestJS y PostgreSQL a partir de un portal anterior en PHP y MySQL, y que este trabajo toma a la vez como objeto de mejora y como fuente de datos.

2.1.2. ERP EN EMPRESAS DE INGENIERÍA

La aplicación del ERP a las empresas de ingeniería y construcción exige revisar el supuesto manufacturero. Shi y Halpin (2003) advierten que la naturaleza particular de la industria de la construcción impide adoptar directamente los sistemas orientados a manufactura y proponen las características fundamentales de un ERP para construcción: funciones de gestión de proyectos, técnicas de planificación avanzada y estandarización de procesos. En estas empresas la unidad de gestión deja de ser el producto repetitivo y pasa a ser el proyecto, con alcance, presupuesto, plazo y cliente propios, y el sistema debe seguir cada uno por separado sin perder la visión consolidada de la empresa.

Aun con esas adaptaciones, la adopción en el sector no es sencilla. Chung et al. (2008)

reconocen que los ERP ofrecen muchos beneficios a la industria de ingeniería y construcción, pero que muchas firmas dudan en adoptarlos por su alto costo, sus incertidumbres y sus riesgos; en respuesta identifican factores críticos de éxito y fracaso e indicadores para evaluar el éxito del ERP, y proponen un modelo que los relaciona. El trabajo traslada al sector la discusión genérica de Umble et al. (2003), y su enfoque asume que el resultado de una implementación depende de factores organizacionales y no solo técnicos.

Una segunda línea conecta el ERP con el control de desempeño de los proyectos. Skibniewski y Ghosh (2009) se preguntan qué áreas de proceso de la construcción no pueden ser cubiertas por el ERP a la hora de recolectar indicadores clave de desempeño (KPI), y lo responden con una encuesta a los responsables de ERP de grandes firmas de ingeniería y construcción de Estados Unidos. El planteamiento del trabajo supone que una parte de los procesos de proyecto queda fuera del alcance de un ERP estándar, lo que refuerza el argumento de Olsen y Sætre (2007) a favor del desarrollo propio cuando el núcleo del negocio es el proyecto.

Lo anterior se corresponde con la forma en que ISI Mustang ha organizado su propio sistema. En el ERP de la empresa la unidad que articula toda la operación es el centro de costo, que puede representar un proyecto, un servicio o un área de estructura y tiene código, responsable, moneda y vigencia propios. Sobre esa unidad se registran las horas del personal, las requisiciones y las órdenes de compra emitidas a los proveedores, los viáticos y sus rendiciones, y el presupuesto, con una línea base separada del seguimiento. El ingreso ejecutado de cada proyecto se registra mediante certificaciones, cuyo flujo distingue la solicitud por parte del gerente del centro de costo, el registro y el cobro.

El portal anterior, cuyos procesos se analizan en el Capítulo III, calculaba además indicadores de seguimiento y control por proyecto, entre ellos el desvío de costo, el desvío de ingresos, el avance del proyecto y el avance de trabajo, a partir del monto de la orden de compra emitida por el cliente, lo ejecutado y el flujo de caja planificado. Esos indicadores son el antecedente directo de las desviaciones que el módulo predictivo busca anticipar, y los datos con los que se calculaban son los que se exploran en el Capítulo III.

2.1.3. ERP EN EL SECTOR DE PETRÓLEO, GAS Y MINERÍA

La literatura sobre ERP en petróleo, gas y minería es más escasa que la de manufactura o construcción y está compuesta sobre todo por estudios de caso, cercanos a los sectores en los que ISI Mustang ejecuta sus proyectos. En petróleo y gas, Al Jafari y Nair (2018) analizan, con un enfoque mixto, cómo la implementación del ERP influye en la eficiencia y la productividad de las empresas del sector en el Sultanato de Omán. Parten de que el papel del ERP en esa industria no puede negarse y concluyen recomendando su implementación como palanca de crecimiento.

Los retos organizacionales aparecen con más nitidez en Ali et al. (2023), que mediante investigación-acción estudian las barreras culturales a la implementación de ERP en el sector de petróleo y gas de Medio Oriente, desde la planificación hasta la posimplementación. Identifican como causas del retraso el conflicto entre equipos, la autoridad gerencial, la falta de una cultura de TI y de compromiso con la capacitación, y la tecnofobia. Gool y Seymour (2018), por su parte, plantean el dilema de cuándo personalizar el ERP y cuándo cambiar los procesos de negocio para ajustarlos a la funcionalidad estándar. En una petrolera multinacional africana encuentran que la personalización continúa después de la implementación y describen prácticas para gestionarla: capacitación, eliminación sistemática de modificaciones y procesos de aprobación.

Para la minería, Larasati et al. (2023) encuestan a la gerencia y el personal de una empresa minera indonesia y ordenan los factores críticos de fracaso del ERP con el método TOPSIS (*Technique for Order of Preference by Similarity to Ideal Solution*); los dos más importantes resultan ser la mala comprensión de los procesos de negocio de la organización y una reingeniería de procesos deficiente. El hallazgo coincide con lo observado en petróleo y gas: en las industrias extractivas el ERP fracasa o se retrasa menos por la tecnología que por el conocimiento de los procesos, la cultura de uso y la gestión continua de la adaptación. Cabe suponer que lo mismo vale para las empresas EPC que trabajan para ellas, aunque ninguno de los casos revisados las estudia directamente.

Tres lecciones de esa literatura orientan este proyecto: documentar los flujos de negocio antes de sistematizarlos, criterio que se sigue en el análisis de procesos del portal anterior; asumir la personalización como un proceso continuo, lo que es coherente con la metodología iterativa e

incremental de la construcción; y anticipar que la resistencia al uso señalada por Ali et al. (2023) puede afectar también a un módulo predictivo, de ahí la validación con usuarios clave prevista en el Capítulo IV.

Queda, por último, el papel del ERP que más interesa a este trabajo: el de fuente de datos. Un sistema que durante más de diez años ha registrado horas, compras, presupuestos e ingresos facturados por proyecto, complementado con las planillas de control que la empresa lleva fuera del sistema, acumula un histórico del que pueden aprenderse patrones. Jawad y Villányi (2024) revisan la integración del aprendizaje automático con los sistemas ERP: sus algoritmos extraen patrones complejos de grandes conjuntos de datos y permiten pronósticos más precisos, con un énfasis creciente en la interpretabilidad de los resultados. Ese es el enlace con las secciones que siguen: el aprendizaje automático aporta los métodos, la analítica predictiva el marco de aplicación, y el ERP de ISI Mustang, tanto el histórico del portal anterior en MySQL como los proyectos activos del ERP nuevo en PostgreSQL, los datos.

2.2. APRENDIZAJE AUTOMÁTICO

2.2.1. TIPOS DE APRENDIZAJE

El aprendizaje automático (*machine learning*) es la rama de la inteligencia artificial que estudia los algoritmos capaces de mejorar su desempeño de manera automática a partir de la experiencia. Mitchell (1997) lo formaliza con tres elementos: se dice que un programa aprende de una experiencia E respecto de una clase de tareas T y una medida de desempeño P si su desempeño en T , medido por P , mejora con E . Trasladada al presente proyecto, esa formalización ubica la experiencia en el histórico operativo de más de diez años de ISI Mustang y la tarea en anticipar desviaciones en sus proyectos, mientras que la medida de desempeño está dada por las métricas que se describen en la subsección siguiente.

La forma en que esa experiencia se presenta al algoritmo define los paradigmas clásicos del campo. Géron (2022) organiza los sistemas de aprendizaje automático según la supervisión que

reciben durante el entrenamiento en tres familias: supervisado, no supervisado y por refuerzo. En el aprendizaje supervisado cada observación de entrenamiento viene acompañada de la respuesta correcta, la etiqueta o valor objetivo, y el algoritmo aprende una función que asocia las entradas con esa salida para predecirla sobre observaciones nuevas (Hastie et al., 2009). En el aprendizaje no supervisado no existe tal respuesta: el algoritmo recibe únicamente las entradas y busca estructura en ellas, como grupos de observaciones similares o direcciones de mayor variabilidad (James et al., 2021). El aprendizaje por refuerzo, por último, no parte de un conjunto de datos fijo sino de un agente que interactúa con un entorno y ajusta su comportamiento según las recompensas que recibe (Mitchell, 1997).

Dentro del aprendizaje supervisado, la naturaleza de la variable de salida distingue dos tipos de problema. Cuando la salida es cuantitativa (un monto, una duración, un porcentaje) se habla de regresión; cuando es cualitativa o categórica (una clase entre un conjunto finito) se habla de clasificación (Hastie et al., 2009; James et al., 2021). La distinción no es solo nominal: determina qué algoritmos son aplicables, qué función de pérdida se optimiza y con qué métricas se evalúa el resultado.

Los tres casos de uso del módulo predictivo se plantean en el paradigma supervisado: la predicción de desviación en certificaciones como un problema de clasificación, y la estimación del sobrecosto y del retraso en la fecha de cierre como problemas de regresión. El planteo supone que el histórico de la empresa contiene, para cada proyecto cerrado, el resultado que se quiere predecir: cuánto costó respecto de lo presupuestado, cuándo se cerró respecto de la fecha comprometida y cómo evolucionaron sus certificaciones frente a lo planificado. La existencia, completitud y recuperabilidad de esas respuestas para una proporción suficiente de proyectos se examina en el análisis de datos del Capítulo III, del que dependen también los detalles de las variables y del protocolo de evaluación que esta sección presenta en términos generales. El aprendizaje no supervisado y el aprendizaje por refuerzo quedan fuera del alcance del proyecto.

2.2.2. APRENDIZAJE SUPERVISADO

Formalmente, el aprendizaje supervisado parte de un conjunto de entrenamiento compuesto

por pares (x_i, y_i) , donde x_i es un vector de variables predictoras y y_i la respuesta observada, y busca una función f tal que $f(x)$ aproxime a y con el menor error posible sobre observaciones que no participaron del entrenamiento (Hastie et al., 2009). Esta última condición es la que separa aprender de memorizar. James et al. (2021) distinguen el error de entrenamiento, que mide el ajuste sobre los datos vistos, del error de prueba, que mide la capacidad de generalizar; un modelo puede reducir el primero indefinidamente aumentando su complejidad, pero a partir de cierto punto el segundo empieza a crecer. Ese fenómeno es el sobreajuste, y se explica por el compromiso entre sesgo y varianza: los modelos simples tienen sesgo alto porque no capturan la estructura de los datos, mientras que los modelos muy flexibles tienen varianza alta porque se ajustan al ruido de la muestra particular con la que fueron entrenados (Hastie et al., 2009; James et al., 2021).

Como el error de prueba no puede calcularse sobre los datos de entrenamiento, se requiere un procedimiento para estimarlo. La práctica básica consiste en separar una parte de los datos como conjunto de prueba, que se reserva y solo se usa al final para evaluar el modelo elegido (Géron, 2022). Para seleccionar entre modelos o ajustar hiperparámetros sin contaminar ese conjunto se emplea la validación cruzada de k particiones (*k-fold*): los datos de entrenamiento se dividen en k bloques, el modelo se entrena k veces dejando fuera un bloque distinto en cada corrida y el error se promedia sobre las k evaluaciones (James et al., 2021). En un estudio empírico de referencia sobre selección de clasificadores, Kohavi (1995) concluye que la validación cruzada estratificada de diez particiones es el mejor método para selección de modelos, incluso cuando la capacidad de cómputo permite usar más particiones, y advierte que la variante *leave-one-out*, aunque casi insesgada, tiene alta varianza y produce estimaciones poco confiables. La estratificación conserva en cada bloque la proporción de clases del conjunto completo; es propia de los problemas de clasificación e importa cuando una de las clases es minoritaria.

La evaluación del modelo depende del tipo de problema. En clasificación, las medidas usuales derivan de la matriz de confusión, que cruza la clase real con la clase predicha (Géron, 2022; Sokolova & Lapalme, 2009); de ella se obtienen la exactitud, la precisión, la sensibilidad o *recall* y la medida F1, definidas en el cuadro 2.1. Sokolova y Lapalme (2009) analizan qué cambios en la matriz de confusión dejan invariante cada medida y muestran que estas cuatro reaccionan de forma distinta ante cambios en la distribución de clases, por lo que conviene reportarlas juntas en lugar de limitarse a la exactitud.

En regresión, el punto de partida es el error cuadrático medio (MSE), que al elevar al cuadrado las diferencias entre valor observado y predicho da más peso a los errores grandes (James et al., 2021); su raíz, el RMSE, se expresa en las unidades de la variable, mientras que el error absoluto medio (MAE), al promediar diferencias en valor absoluto, penaliza menos esos errores. Chai y Draxler (2014) sostienen que el RMSE es más apropiado cuando se espera que los errores sigan una distribución gaussiana y que evaluar un modelo requiere una combinación de métricas y no una sola. El coeficiente de determinación R^2 mide la proporción de la variabilidad de la respuesta que el modelo explica; toma la forma de una proporción y no depende de la escala de la variable (James et al., 2021), aunque qué valor se considera bueno depende de la aplicación. El cuadro 2.1 resume las métricas y su uso en el proyecto.

Cuadro 2.1. Métricas de evaluación de modelos supervisados y su uso en el proyecto

Métrica	Qué mide	Uso en el proyecto
<i>Clasificación</i>		
Matriz de confusión	Cruce de clase real y clase predicha (VP, VN, FP, FN)	Base de las demás métricas; muestra qué tipo de error comete el clasificador de desviación en certificaciones
Exactitud	$(VP + VN)/total$	Referencia general; insuficiente si las clases están desbalanceadas
Precisión	$VP/(VP + FP)$	Proporción de alertas de desviación que son correctas
Sensibilidad	$VP/(VP + FN)$	Proporción de desviaciones reales que el modelo detecta
F1	Media armónica de precisión y sensibilidad	Resumen único cuando ambas importan
<i>Regresión</i>		
RMSE	Raíz del promedio de errores al cuadrado	Error en unidades de la variable; penaliza sobrecostos o retrasos grandes mal estimados
MAE	Promedio de errores en valor absoluto	Error típico esperado, menos sensible a valores extremos
R^2	Proporción de la variabilidad explicada por el modelo	Ajuste de cada regresor sobre su propia respuesta, independiente de la escala; complementa al RMSE y al MAE

Fuente: Elaboración propia, 2026, en base a Sokolova & Lapalme (2009), James et al. (2021) y Chai & Draxler (2014).

Con estas definiciones, el protocolo de evaluación del módulo predictivo es el siguiente: el clasificador de desviación en certificaciones se evalúa con exactitud, precisión, sensibilidad, F1 y matriz de confusión; los regresores de sobrecosto y de retraso en la fecha de cierre, con RMSE, MAE y R^2 . En los tres casos la selección de hiperparámetros se realiza con validación cruzada de k particiones sobre el conjunto de entrenamiento (estratificada solo en el clasificador) y se reserva un conjunto de prueba independiente para la evaluación final.

2.2.3. ALGORITMOS DE APRENDIZAJE SUPERVISADO: GRADIENT BOOSTING Y XGBOOST

El modelo base de los algoritmos empleados en este proyecto es el árbol de decisión. Un árbol particiona recursivamente el espacio de las variables predictoras mediante reglas binarias del tipo «presupuesto mayor que cierto umbral» y asigna a cada región resultante una predicción constante: la clase mayoritaria en clasificación o el promedio de las respuestas en regresión (Hastie et al., 2009; James et al., 2021). El procedimiento CART de Breiman et al. (1984) es la formulación de referencia de este tipo de árboles binarios. Los árboles son fáciles de interpretar y, en su formulación teórica, pueden dividir directamente sobre variables categóricas; las implementaciones actuales de *gradient boosting* admiten esa capacidad de forma nativa como opción, XGBoost desde su versión 1.5 y Scikit-learn en sus estimadores basados en histogramas. En la configuración adoptada en este proyecto, sin embargo, los predictores categóricos se codifican de forma explícita antes del entrenamiento, como se describe en la subsección siguiente, lo que deja la representación de las variables bajo el control del pipeline de datos y no de la biblioteca. Por otra parte, un árbol aislado tiene alta varianza (pequeños cambios en los datos producen árboles muy distintos) y su capacidad predictiva suele ser inferior a la de otros métodos (James et al., 2021).

Los métodos de ensamble corrigen esa debilidad combinando muchos árboles, con dos estrategias distintas. En el *bagging*, los árboles se entrenan en paralelo sobre remuestreos de los datos y sus predicciones se promedian, lo que reduce la varianza; los bosques aleatorios de Breiman (2001) parten de esa idea y añaden una selección aleatoria de variables en cada división. En el *boosting*, en cambio, los árboles se construyen de forma secuencial: cada nuevo árbol se

ajusta a lo que los anteriores todavía no explican, de modo que el ensamble mejora por etapas (Hastie et al., 2009; James et al., 2021).

Friedman (2001) da a esta idea su formulación general al plantear la estimación de la función como un problema de optimización numérica en el espacio de funciones: en cada etapa se ajusta un árbol de regresión al gradiente negativo de la función de pérdida evaluada sobre el modelo acumulado, lo que equivale a un descenso por gradiente en el que cada paso es un árbol. El marco admite distintas funciones de pérdida (mínimos cuadrados, desviación absoluta o Huber para regresión, y verosimilitud logística para clasificación), de modo que el mismo algoritmo sirve para los problemas de regresión y de clasificación del proyecto. Sus hiperparámetros principales son el número de árboles, la tasa de aprendizaje o *shrinkage*, que reduce la contribución de cada árbol, y la profundidad de los árboles, que controla el orden de las interacciones entre variables (James et al., 2021); el submuestreo de observaciones en cada etapa agrega una regularización adicional (Hastie et al., 2009).

XGBoost es la implementación de *gradient boosting* de árboles que se prevé utilizar. T. Chen y Guestrin (2016) la presentan como un sistema escalable, ampliamente usado para obtener resultados de vanguardia en competencias de aprendizaje automático, y describen sus aportes: un algoritmo *sparsity-aware* para datos dispersos, un esquema de cuantiles ponderados para construir árboles de forma aproximada, y optimizaciones de caché, compresión y particionado que le permiten escalar a miles de millones de ejemplos con menos recursos que otros sistemas.

Dos características de XGBoost interesan al diseño del módulo predictivo (T. Chen & Guestrin, 2016). La primera es el tratamiento de valores ausentes: el algoritmo de búsqueda de divisiones *sparsity-aware* incorpora en cada nodo una dirección por defecto, aprendida de los datos, hacia la que se envían las observaciones que carecen del valor de la variable de división, lo que permite entrenar sin imputar previamente los valores ausentes. La segunda es la función objetivo regularizada, que suma a la pérdida un término de complejidad que penaliza el número de hojas y la magnitud de los pesos de cada árbol; sus coeficientes son hiperparámetros adicionales a los del *boosting* ya mencionados y se ajustan mediante validación cruzada.

La elección de *gradient boosting* frente a alternativas más recientes se apoya en evidencia

sobre datos tabulares. Grinsztajn et al. (2022) comparan modelos de árboles y redes neuronales profundas sobre 45 conjuntos de datos tabulares y concluyen que los primeros siguen siendo el estado del arte en conjuntos de tamaño medio, del orden de diez mil observaciones, aun sin considerar su ventaja en velocidad; atribuyen el resultado a sesgos inductivos de los árboles, como la robustez frente a variables no informativas y la capacidad de aprender funciones irregulares, que las redes no comparten. Shwartz-Ziv y Armon (2022) llegan a la misma conclusión al enfrentar XGBoost con modelos profundos diseñados específicamente para datos tabulares: XGBoost los supera incluso en los conjuntos de datos usados por los artículos que los proponen y requiere mucho menos ajuste. Ambos resultados se obtienen sobre datos tabulares, la forma que toma el histórico de la empresa, y respaldan la elección de XGBoost para los tres casos de uso. En la implementación se emplea su interfaz compatible con Scikit-learn, la biblioteca de aprendizaje automático en Python presentada por Pedregosa et al. (2011). Cabe una salvedad: los conjuntos evaluados en esos estudios son del orden de miles de observaciones, mientras que el histórico de la empresa, con un vector de variables por proyecto, probablemente reúne bastantes menos, lo que condiciona hasta qué punto esa evidencia es trasladable.

2.2.4. *FEATURE ENGINEERING*

La ingeniería de características (*feature engineering*) es el conjunto de operaciones que transforman los datos crudos en las variables predictoras que recibe el modelo. Kuhn y Johnson (2019) la sitúan dentro del proceso de modelado predictivo, junto con la división de datos, el remuestreo, el ajuste de hiperparámetros y la evaluación, y sostienen que la representación de los predictores condiciona el desempeño tanto como la elección del algoritmo. Géron (2022) muestra el mismo proceso desde la práctica con Scikit-learn: limpieza, codificación de variables categóricas, escalado y encadenamiento de esas transformaciones en *pipelines* que se ajustan únicamente con datos de entrenamiento, para evitar que información del conjunto de prueba se filtre al modelo.

Tres grupos de técnicas son relevantes para este proyecto. El primero es la codificación de predictores categóricos, como el cliente, el tipo de proyecto o el rubro, que en este proyecto se realiza de forma explícita en el pipeline según la decisión indicada en la subsección anterior: Kuhn y Johnson (2019) describen la codificación *one-hot* o por variables indicadoras, el agrupamiento

de categorías poco frecuentes y las codificaciones supervisadas, que reemplazan cada categoría por un estadístico de la respuesta, útiles cuando la variable tiene muchos niveles. El segundo es la transformación de predictores numéricos, que incluye el centrado y escalado, las transformaciones para corregir asimetría y la discretización en intervalos. El tercero, y el más sensible en un histórico operativo, es el manejo de datos faltantes: los mismos autores recomiendan visualizar primero el patrón de ausencia y luego decidir entre eliminar observaciones o variables, imputar los valores o emplear modelos tolerantes a faltantes. Esta última opción se corresponde con el tratamiento de valores ausentes de XGBoost descrito en la subsección anterior.

La selección de variables completa el proceso. Guyon y Elisseeff (2003) le atribuyen tres objetivos: mejorar el desempeño del predictor, obtener modelos más rápidos y económicos, y comprender mejor el proceso que generó los datos. Kuhn y Johnson (2019) clasifican los métodos en filtros, que evalúan cada variable de forma independiente del modelo, y envoltentes (*wrappers*), que evalúan el modelo sobre subconjuntos candidatos de variables, ya sea mediante búsquedas voraces, como la eliminación recursiva, o mediante búsquedas globales; y advierten que, si la selección se realiza fuera del esquema de remuestreo, la estimación del error queda sesgada de forma optimista.

En el proyecto estas operaciones constituyen una etapa propia del pipeline de datos, que construye para cada proyecto un vector de variables a partir de sus atributos maestros y de agregados de sus registros transaccionales a una fecha de corte. Un requisito específico es que las mismas variables puedan calcularse tanto desde el histórico del portal anterior, que alimenta el entrenamiento, como desde el ERP nuevo, sobre el que se realiza la inferencia; el diseño de esa etapa se desarrolla en el Capítulo III.

2.3. INGENIERÍA DE DATOS

2.3.1. PROCESOS ETL Y ELT

Los procesos de extracción, transformación y carga (ETL, por *extract, transform, load*) son

el mecanismo clásico para alimentar un almacén de datos desde los sistemas operacionales. Vassiliadis (2009) los define como los procesos de software que facilitan la carga original y el refresco periódico del contenido del almacén, y revisa los problemas propios de cada una de las tres etapas y los que afectan al proceso completo. El destino es el almacén de datos que Inmon (2005) caracteriza como una colección de datos orientada a temas, integrada, variable en el tiempo y no volátil, en apoyo de la toma de decisiones; la integración y la orientación a temas son, precisamente, el trabajo que ETL realiza sobre datos que en origen están organizados por aplicación y por transacción.

Desde la práctica del modelado dimensional, Kimball y Ross (2013) advierten que el sistema ETL consume una parte desproporcionada del tiempo y del esfuerzo necesarios para construir un entorno de almacén de datos e inteligencia de negocios, y lo descomponen en 34 subsistemas agrupados en cuatro áreas: extraer los datos de las fuentes, limpiarlos y conformarlos, entregarlos preparados para la presentación y gestionar el entorno ETL. Dos de sus aportes interesan a este proyecto. El primero es el conformado (*conforming*), orientado a que atributos equivalentes de fuentes distintas se representen de manera uniforme. El segundo es la distinción entre la carga histórica única, que puebla el almacén con todo el pasado disponible, y el procesamiento incremental, que lo actualiza periódicamente con los registros nuevos.

La variante ELT (*extract, load, transform*) invierte las dos últimas etapas: los datos se cargan crudos en el destino y la transformación se ejecuta allí, con el motor de consulta del propio almacén o lago. Reis y Housley (2022) ubican ambos patrones en la etapa de transformación de un ciclo de vida de la ingeniería de datos que va de la generación a la ingesta, el almacenamiento, la transformación y el servicio; explican que ETL surgió cuando el cómputo era un recurso escaso y convenía transformar antes de cargar, que ELT es el patrón propio de los lagos de datos y *lakehouses* modernos, y recomiendan elegir según el caso en lugar de adherir a uno por principio. Armbrust et al. (2021) muestran el costo de encadenar ambos patrones: en la arquitectura de dos niveles habitual en las empresas los datos primero se someten a ETL hacia el lago y luego a ELT hacia el almacén, y cada paso adicional agrega demoras, complejidad y nuevas oportunidades de introducir errores que reducen la calidad de los datos.

El pipeline de datos de este proyecto es un caso de ELT con fines exclusivamente analíticos.

Las fuentes son la base MySQL del portal anterior, la base PostgreSQL del ERP nuevo y las planillas Excel de control de proyectos que la empresa lleva fuera del sistema; sus datos se copian sin modificaciones a una capa cruda y se transforman después, en las capas que describe la subsección siguiente. La distinción de Kimball y Ross se traduce en dos modos de operación: una carga histórica del portal anterior y de las planillas, que produce el conjunto de entrenamiento, y una carga incremental desde el ERP nuevo, que alimenta la inferencia sobre los proyectos activos. En ningún caso el pipeline escribe de vuelta en el ERP ni migra el histórico al esquema nuevo; esa migración queda fuera del alcance del trabajo.

2.3.2. ARQUITECTURA MEDALLION (BRONZE, SILVER, GOLD)

El principio de organizar los datos en capas de calidad creciente tiene su fundamento académico en la propuesta de *lakehouse* de Armbrust et al. (2021). Los autores parten de un diagnóstico: los lagos de datos de segunda generación, que aplican el esquema en el momento de la lectura, trasladaron aguas abajo el problema de la calidad y la gobernanza, y combinarlos con un almacén separado produce datos desactualizados, poca fiabilidad y un alto costo total. En respuesta proponen una plataforma única que almacena los datos en formatos abiertos de acceso directo, como Parquet, ofrece soporte de primera clase al aprendizaje automático y a la ciencia de datos, y alcanza un rendimiento comparable al de un almacén. Dos rasgos de esa propuesta importan aquí: un *lakehouse* debe poder guardar los datos crudos, como un lago, y al mismo tiempo soportar procesos ETL/ELT que los curen para mejorar su calidad con vistas al análisis; y su capa de metadatos es el lugar natural para aplicar controles de calidad, como la imposición de esquema y las restricciones que rechazan o ponen en cuarentena los registros inválidos.

La nomenclatura con la que hoy se designa ese curado por etapas no proviene de ese artículo, sino de la documentación de Databricks, es decir, de literatura de proveedor. Databricks (2025) define la arquitectura *medallion*, también llamada de múltiples saltos (*multi-hop*), como un patrón de diseño para organizar lógicamente los datos en una serie de capas que denotan la calidad de lo almacenado. La capa Bronze recibe los datos crudos de las fuentes sin validación y conserva su estado original; la capa Silver contiene al menos una representación validada y no agregada de cada registro, y es donde se realizan la limpieza, la deduplicación y la normalización; la capa Gold

consiste en datos agregados y ajustados a los requisitos de análisis y reporte. Al hacer progresar los datos por las capas, su calidad y fiabilidad mejoran de manera incremental hasta hacerlos aptos para inteligencia de negocios y aprendizaje automático.

El proyecto lo adopta sin la plataforma del proveedor, en línea con el argumento de Armbrust et al. (2021) a favor de los formatos abiertos: se prevé materializar las capas como archivos en el servidor de la empresa y ejecutar las transformaciones con dos herramientas ligeras que se describen en la sección 2.7, DuckDB para Silver y Polars para Gold. El cuadro 2.2 resume la correspondencia. Bronze reúne copias crudas de las tres fuentes: las tablas de MySQL del portal anterior, las tablas de PostgreSQL del ERP nuevo y las planillas Excel. Silver es la capa en la que el conformado de Kimball y Ross (2013) se convierte en un requisito del modelo predictivo: los dos esquemas relacionales describen los mismos objetos de negocio (centros de costo, certificaciones, órdenes de compra, presupuestos) con nombres y estructuras distintos, y DuckDB los mapea a un conjunto único de variables con las mismas unidades y dominios, de modo que un modelo entrenado sobre el histórico pueda predecir sobre los proyectos activos. Gold construye con Polars las tablas de características por proyecto a una fecha de corte, único insumo de los modelos. El diseño detallado de cada capa se desarrolla en el Capítulo III.

Cuadro 2.2. Capas de la arquitectura *medallion* y su correspondencia en el proyecto

Capa	Contenido según el patrón	En el proyecto
Bronze	Datos crudos de las fuentes, sin validación, en su estado original	Copias de las tablas MySQL del portal anterior, de las tablas PostgreSQL del ERP nuevo y de las planillas Excel de control de proyectos
Silver	Al menos una representación validada y no agregada de cada registro; limpieza, deduplicación y normalización	Esquema unificado en DuckDB: los dos esquemas relacionales y las planillas mapeados a las mismas variables, con tipos y dominios conformados y registros deduplicados
Gold	Datos agregados y ajustados a los requisitos de análisis y reporte	Tablas de características por proyecto construidas con Polars; entrada del entrenamiento (histórico) y de la inferencia (proyectos activos)

Fuente: Elaboración propia, 2026, en base a Databricks (2025) y Armbrust et al. (2021).

2.3.3. CALIDAD DE DATOS

La calidad de los datos no se reduce a su exactitud. Wang y Strong (1996) observan que los esfuerzos de mejora tienden a concentrarse en la exactitud, mientras que los consumidores de datos manejan una conceptualización mucho más amplia, y construyen un marco jerárquico que organiza las dimensiones de calidad en cuatro categorías: intrínseca, las propiedades que los datos tienen por sí mismos; contextual, que exige considerarlos en el contexto de la tarea en la que se usan; representacional, referida a la forma en que se presentan; y de accesibilidad. Datos de alta calidad son, en esa formulación, intrínsecamente buenos, apropiados para la tarea, claramente representados y accesibles. De la categoría contextual se sigue que la calidad de un conjunto de datos no se juzga en abstracto, sino respecto de la tarea a la que se destina.

Las dimensiones concretas más usadas son la exactitud, la completitud, la consistencia y la oportunidad, a las que Batini y Scannapieco (2016) dedican un capítulo de su tratado junto con otras dimensiones temporales; el cuerpo de conocimiento de DAMA International (2017) agrega, entre otras, la unicidad y la validez. Batini et al. (2009) muestran, además, que la literatura ofrece una amplia gama de metodologías de evaluación y mejora, que difieren en fases, estrategias y tipos de datos y de sistemas. El cuadro 2.3 fija las definiciones operativas que este trabajo adopta para seis dimensiones aplicables a un histórico relacional, y la verificación con que cada una se mide.

Cuadro 2.3. Dimensiones de calidad de datos adoptadas en el proyecto

Dimensión	Definición operativa	Verificación prevista
Exactitud	El valor registrado coincide con el valor real del objeto que describe	Contraste de montos y fechas del sistema contra las planillas de control y documentos de respaldo, sobre una muestra
Compleitud	Los valores requeridos para la tarea están presentes	Proporción de proyectos con presupuesto, fechas y certificaciones registrados; patrón de valores nulos por variable
Consistencia	Los valores no se contradicen entre registros ni entre tablas	Sumas de detalle frente a totales de cabecera; estados de flujo compatibles con las fechas
Oportunidad	El dato está disponible cuando la tarea lo necesita	Rezago entre la fecha del evento y la fecha de registro en el sistema
Unicidad	Cada objeto de negocio aparece una sola vez	Duplicados de proyectos, clientes y proveedores por clave natural
Validez	Los valores pertenecen al dominio definido	Valores fuera de rango o de catálogo; tipos incorrectos en campos de texto libre

Fuente: Elaboración propia, 2026, en base a Wang & Strong (1996), Batini & Scannapieco (2016) y DAMA International (2017).

La relación entre esas dimensiones y el desempeño de un modelo de aprendizaje automático cuenta con evidencia empírica reciente. Mohammed et al. (2025) sostienen que datos de entrenamiento incompletos, erróneos o inapropiados conducen a modelos poco confiables, y estudian de forma sistemática el efecto de seis dimensiones de calidad sobre el desempeño de 19 algoritmos de clasificación, regresión y agrupamiento sobre datos tabulares, contaminando los datos de entrenamiento, los de prueba o ambos; un diseño pertinente para un proyecto cuyos datos son tabulares y cuyos modelos se entrenan sobre un histórico y se aplican sobre datos nuevos. Sambasivan et al. (2021) aportan la dimensión organizacional: a partir de entrevistas con 53 practicantes de inteligencia artificial en contextos de alto riesgo, describen las cascadas de datos, efectos negativos que se propagan aguas abajo a partir de problemas en los datos, y las encuentran omnipresentes, invisibles, de efecto retardado y con frecuencia evitables, fruto de prácticas que subvaloran los datos frente al modelado. Gudivada et al. (2017) agregan que, para el aprendizaje automático, la calidad va más allá de la limpieza y las transformaciones y requiere

gobernar el ciclo de vida completo de los datos.

De esa literatura el proyecto toma dos decisiones. La primera es que la exploración del histórico del Capítulo III usa las dimensiones del cuadro 2.3 como instrumento y las juzga respecto de la tarea predictiva concreta; qué grado de calidad tiene ese histórico en cada dimensión es un resultado de esa exploración y no se anticipa aquí. La segunda es que la mejora se aborda por dos vías: la limpieza en la capa Silver, que corrige los datos, y la corrección de los defectos del ERP nuevo, que corrige el proceso del que saldrán los datos de inferencia, de modo que los registros sobre los que se realiza la inferencia nazcan con mejor calidad que los históricos con los que se entrena.

2.4. ANALÍTICA PREDICTIVA

2.4.1. DEFINICIÓN Y APLICACIONES

Davenport y Harris (2007) sostienen que las empresas que compiten con éxito no se limitan a acumular datos, sino que construyen sus estrategias alrededor del conocimiento que extraen de ellos, combinando análisis cuantitativo, modelado predictivo, liderazgo orientado a los datos e infraestructura tecnológica. Interesa que el modelado predictivo figure allí como un componente de la analítica, y no como una disciplina aparte.

Dentro de ese campo, la clasificación más difundida distingue tres niveles. Delen y Demirkan (2013) separan la analítica descriptiva, que resume lo que ocurrió, de la predictiva, que estima qué podría ocurrir a continuación, y de la prescriptiva, que busca los mejores pasos a seguir. Lepenioti et al. (2020) confirman en su revisión que el grueso del trabajo académico e industrial se concentra todavía en los dos primeros niveles y presentan a la prescriptiva como el siguiente escalón de madurez analítica, orientado a anticipar decisiones optimizadas. La tríada sirve para ubicar con precisión lo que este proyecto construye: los reportes de Power BI cubren el nivel descriptivo sobre los datos del ERP, los modelos de aprendizaje automático cubren el predictivo, y el nivel prescriptivo, que supondría recomendar acciones correctivas, queda fuera del alcance

como trabajo futuro.

Que el objetivo sea predecir tiene consecuencias metodológicas que no siempre se hacen explícitas. Shmueli (2010) observa que en muchas disciplinas domina el modelado explicativo, orientado a probar relaciones causales, y que con frecuencia se asume que un modelo con alto poder explicativo tiene también alto poder predictivo; la autora muestra que esa confusión es común y que la distinción afecta cada paso del proceso de modelado, desde el diseño y la recolección de datos hasta la selección de variables, la elección del método y la evaluación. Para este trabajo la implicancia es directa: los modelos se juzgan por su error sobre observaciones no vistas, con las métricas y la validación cruzada descritas en la sección 2.2, y no por la significancia o la interpretabilidad de sus coeficientes. Esa prioridad es la que justifica emplear *gradient boosting* aunque sea menos transparente que una regresión lineal.

La analítica predictiva aplicada a operaciones exige, además del método, conocimiento del negocio. Waller y Fawcett (2013), al presentarla como una transformación del diseño y la gestión de cadenas de suministro, insisten en que la ciencia de datos requiere tanto dominio del área como un conjunto amplio de habilidades cuantitativas. En el contexto de los sistemas empresariales, Jawad y Villányi (2024) describen la integración de algoritmos de aprendizaje automático en el ERP como el medio para producir predicciones más precisas y decisiones basadas en datos, con el requisito de que los modelos resulten comprensibles para los interesados. El presente proyecto sigue esa línea: el conocimiento del dominio EPC (certificaciones, avance, presupuesto) guía la construcción de variables a partir del histórico del portal anterior y de las planillas de control, y las predicciones se integran al ERP mediante su propio tablero, de modo que llegan al jefe de proyecto en la misma herramienta en que gestiona su trabajo.

2.4.2. ANALÍTICA PREDICTIVA EN GESTIÓN DE PROYECTOS

La técnica de pronóstico clásica en control de proyectos es la gestión del valor ganado (*earned value management*, EVM), cuyas magnitudes, variaciones e índices se presentan en la sección 2.5.2. En lo que aquí interesa, el EVM proyecta el resultado final extrapolando el desempeño observado hasta la fecha.

Esa extrapolación tiene límites conocidos. Vandevorde y Vanhoucke (2006) recuerdan que el EVM se desarrolló originalmente para gestionar costos y que su empleo para pronosticar la duración ha sido escaso; comparan los indicadores clásicos de cronograma con los del cronograma ganado (*earned schedule*, ES) y contrastan tres métodos de pronóstico de duración sobre un ejemplo y sobre datos reales. Lipke et al. (2009) señalan igualmente que el método ofrece desde su origen procedimientos para pronosticar el costo final pero no la duración, e integran el valor ganado, el cronograma ganado y límites de confianza estadísticos para derivar el rango de resultados probables del costo y la duración finales. Para uno de los tres casos de este trabajo, el retraso en la fecha de cierre, ese antecedente es relevante: pronosticar la duración con EVM no es un problema nuevo.

La literatura reciente propone reemplazar la extrapolación de índices por modelos aprendidos del histórico. Wauters y Vanhoucke (2016) comparan cinco métodos de inteligencia artificial para predecir la duración final de un proyecto y encuentran que superan a los mejores métodos EVM/ES, sobre todo en las etapas temprana y media, a condición de que los conjuntos de entrenamiento y de prueba sean al menos similares; al aumentar la discrepancia entre ambos, los métodos de IA pierden ventaja. En el frente del costo, İnan et al. (2022) construyen un modelo de red neuronal recurrente (LSTM) alimentado con siete características que incluyen factores de cronograma y desempeño y sus medias móviles, y reportan, sobre 300 experimentos, estimaciones más precisas que el modelo tradicional basado en índices del EVM. Ottaviani et al. (2025) generalizan el enfoque con un pipeline automatizado de balanceo, mejora de características, entrenamiento y evaluación, probado con 30 algoritmos sobre 50 proyectos reales de construcción; sus modelos superan al EVM y a la gestión de cronograma convencionales en exactitud, precisión y oportunidad, y advierten que la literatura previa descuidó el sobreajuste.

Tres consecuencias de esa evidencia orientan el diseño del módulo predictivo. Primera, la condición de similitud señalada por Wauters y Vanhoucke (2016) es el argumento técnico para que la capa Silver del pipeline de datos mapee el esquema histórico del portal anterior (MySQL) y el del ERP nuevo (PostgreSQL) a las mismas variables: un modelo entrenado sobre proyectos cerrados solo puede predecir sobre proyectos activos si ambos se describen del mismo modo. Segunda, los indicadores de desempeño y sus medias móviles, empleados como características por İnan et al. (2022), son candidatos naturales a variables predictoras en la medida en que el histórico

permita reconstruirlos, lo que se determina en la exploración de datos del Capítulo III. Tercera, la advertencia sobre el sobreajuste de Ottaviani et al. (2025) refuerza el protocolo de evaluación con conjunto de prueba independiente y validación cruzada ya adoptado en la sección 2.2.

2.4.3. PREDICCIÓN DE DESVIACIONES Y COSTOS

Aprender el sobrecosto del histórico supone que tiene un patrón. Flyvbjerg et al. (2002), sobre una muestra de 258 proyectos de infraestructura de transporte por 90 mil millones de dólares, concluyen que las estimaciones usadas para decidir su ejecución son sistemáticamente engañosas y que la subestimación no se explica por error, sino por tergiversación estratégica. Aunque se trata de obra pública y no de proyectos EPC, el hallazgo sostiene la premisa de este trabajo: si el sobrecosto tiene un patrón, puede aprenderse del histórico. En el sector EPC, Habibi et al. (2018) revisan los indicadores líderes de desempeño de costo y cronograma, con poca consistencia entre ellos y una concentración de los estudios en la fase de construcción. La noción de indicador líder es la que el módulo predictivo operacionaliza como variable predictora.

Sobre qué técnica emplear, las revisiones del área convergen en los métodos basados en árboles. Elmousalami (2020) revisa las técnicas de inteligencia artificial para la estimación paramétrica de costos en construcción, desde la lógica difusa y las redes neuronales hasta los árboles de decisión y el *gradient boosting*, y en un caso de estudio con veinte técnicas obtiene la mejor exactitud con XGBoost (MAPE de 9,09 % y R^2 ajustado de 0,929), resultado que fue objeto de una discusión posterior en la misma revista. Discute, además, el tratamiento de valores faltantes y atípicos, cuestión pertinente para un histórico cuya calidad de registro está por relevar. Shamim et al. (2025), en una revisión sistemática de 39 artículos (2016–2024), encuentran que las redes neuronales son la técnica más frecuente (26,3 % de los estudios) y que los árboles de decisión y el *gradient boosting* ofrecen mejoras sustanciales, mientras que la regresión conviene cuando las relaciones son lineales; los rangos de exactitud que reportan agregan métricas heterogéneas, por lo que se toman como tendencia y no como referencia comparable.

Dos trabajos recientes ilustran la arquitectura de solución que este proyecto adopta. Nguyen et al. (2026) combinan XGBoost con valores SHAP para predecir sobrecostos a partir de variables

como tamaño del proyecto, costo estimado, presión de cronograma y cambios de diseño, con validación cruzada de cinco particiones y evaluación con MAE y R^2 ; su conjunto de datos, sin embargo, consta de mil observaciones simuladas, por lo que se cita como ejemplo de método y no como evidencia de desempeño. Shen et al. (2025) parten de que los modelos basados solo en datos no incorporan el conocimiento experto del dominio y proponen, para predecir retrasos, un marco sobre XGBoost con características derivadas de reglas, penalizaciones en la función de pérdida y reglas de postprocesamiento, que sobre datos reales mejora la exactitud y reduce los falsos negativos. El cuadro 2.4 resume los estudios revisados.

Cuadro 2.4. Estudios revisados sobre predicción de desviaciones, costos y plazos en proyectos

Estudio	Técnica	Variable	Dato relevante
Vandevoorde & Vanhoucke (2006)	Índices EVM y ES	Duración	El EVM se creó para costos y se usó poco para pronosticar duración; compara tres métodos sobre datos reales
Wauters & Vanhoucke (2016)	Cinco métodos de IA	Duración	Superan a EVM/ES si entrenamiento y prueba son similares; pierden ventaja al crecer la discrepancia
İnan et al. (2022)	LSTM	Costo	Siete características con medias móviles de indicadores; 300 experimentos; más preciso que el EVM por índices
Ottaviani et al. (2025)	Pipeline con 30 algoritmos	EAC	50 proyectos reales de construcción; supera a EVM y gestión de cronograma; alerta sobre sobreajuste
Habibi et al. (2018)	Revisión de indicadores líderes	Costo y plazo (EPC)	Más de la mitad de los proyectos con retrasos y escaladas de costo importantes
Elmousalami (2020)	Veinte técnicas de IA	Costo	XGBoost con la mejor exactitud: MAPE 9,09 %, R^2 ajustado 0,929
Shamim et al. (2025)	Revisión sistemática (39 artículos)	Costo	ANN más frecuente (26,3 %); árboles y <i>gradient boosting</i> con mejoras sustanciales
Nguyen et al. (2026)	XGBoost + SHAP	Sobrecosto	CV de 5 particiones, MAE y R^2 ; 1 000 observaciones simuladas
Shen et al. (2025)	XGBoost con reglas de dominio	Retraso	Mejora la exactitud y reduce falsos negativos sobre datos reales

Fuente: Elaboración propia, 2026, en base a las fuentes citadas.

De esta revisión se desprende el planteo de los tres casos del proyecto: la desviación en certificaciones como problema de clasificación, y el sobrecosto y el retraso en la fecha de cierre como problemas de regresión, los tres con *gradient boosting* sobre el histórico del portal anterior y las planillas de control. Ninguna de las fuentes revisadas aporta cifras de sobrecosto para el sector de petróleo, gas y minería en Bolivia, por lo que la magnitud de las desviaciones en ISI Mustang solo puede establecerse con sus propios datos. Se prevé incorporar reglas de negocio como variables y validaciones de consistencia, como en Shen et al. (2025). Las predicciones se muestran únicamente en el tablero del ERP; los reportes de Power BI se mantienen en el nivel descriptivo.

2.5. GESTIÓN DE PROYECTOS EPC

2.5.1. DEFINICIÓN Y CARACTERÍSTICAS

Un proyecto EPC (*engineering, procurement and construction*) es aquel en el que un mismo contratista asume, bajo un único contrato, la ingeniería de diseño, la procura de equipos y materiales y la construcción de la instalación. La forma contractual de referencia es el Libro Plata de FIDIC (2017), las condiciones de contrato para proyectos EPC «llave en mano», en las que el contratista responde de manera integral por el diseño y la ejecución y entrega al cliente una instalación lista para operar. Galloway (2009) describe la posición del contratista desde el punto de vista del propietario: una ventanilla única y a la vez la última parada para todos los costos del proyecto, desde su inicio hasta el cierre.

El rasgo que distingue esta modalidad es la distribución del riesgo. La propia FIDIC declara que el Libro Plata no es adecuado cuando el comitente pretende supervisar de cerca la obra ni cuando existe obra subterránea sustancial que los oferentes no pueden inspeccionar (FIDIC, 2017); en otras palabras, es la forma contractual en la que más riesgo se traslada al contratista, y a cambio el cliente obtiene un precio y un plazo comprometidos con menor intervención de su parte. Galloway (2009) matiza el atractivo de ese arreglo: aunque el esquema puede ahorrar al propietario sumas importantes en el precio inicial y en órdenes de cambio, no es la solución definitiva que se percibe,

porque el tratamiento del cambio sigue siendo el problema peor definido en ese entorno. Para el contratista, cada cambio de alcance que no logra trasladar al precio es una desviación que absorbe.

Las tres fases que dan nombre a la modalidad forman una cadena. Yeo y Ning (2002) proponen para los proyectos EPC un marco de procura que integra la gestión de la cadena de suministro con la cadena crítica derivada de la teoría de restricciones, lo que pone de relieve el lugar de la procura, con sus plazos de entrega y sus dependencias con proveedores y subcontratistas, entre la ingeniería y la construcción. En los sectores donde esta modalidad predomina los resultados son con frecuencia adversos: Merrow (2011), a partir de plataformas costa afuera, plantas químicas y proyectos de petróleo, gas y minerales, sostiene que más de la mitad de los grandes proyectos de ingeniería y construcción tienen resultados pobres, con sobrecostos y retrasos, y lo atribuye a una gestión inadecuada, a la disfunción de los equipos, a una débil rendición de cuentas y a una inversión técnica insuficiente en las etapas tempranas.

ISI Mustang ejecuta proyectos de este tipo en gas, petróleo, minería y energía, y ocupa en ellos la posición del contratista que describe la literatura: asume la ingeniería, la procura y la construcción, y con ellas el riesgo de que el costo o el plazo se aparten de lo comprometido. En el ERP nuevo cada proyecto se modela como un centro de costo con presupuesto, moneda, responsable y vigencia propios, sobre el que se imputan horas, compras y viáticos; el portal anterior organiza también su histórico por centros de costo. Esa correspondencia entre la unidad contractual y la unidad de registro es lo que hace posible observar, proyecto por proyecto, el fenómeno que las dos subsecciones siguientes describen: cómo se mide el avance y cómo se manifiestan las desviaciones.

2.5.2. CERTIFICACIONES Y CONTROL DE AVANCE

El marco de referencia para el control de avance es la gestión del valor ganado (*earned value management*, EVM). Anbari (2003) la define como un método que integra el alcance, el costo y el tiempo, y que requiere el monitoreo periódico de los gastos reales y de los logros físicos del alcance; a partir de ellos calcula variaciones de costo y de cronograma junto con índices

de desempeño, pronostica el costo a la terminación y señala la posible necesidad de acciones correctivas. El Project Management Institute (2019) lo formula en los mismos términos en su norma vigente: una metodología para integrar alcance, cronograma y recursos, medir de forma objetiva el desempeño y el avance, y pronosticar el resultado del proyecto. Fleming y Koppelman (2010) describen el ciclo de aplicación: definir el alcance mediante una estructura de desglose del trabajo, fijar una línea base de medición del desempeño, medir periódicamente el valor ganado, calcular variaciones e índices y pronosticar el costo final.

El método se apoya en tres magnitudes acumuladas a una fecha de corte: el valor planificado (PV), presupuesto del trabajo que debía haberse hecho; el valor ganado (EV), presupuesto del trabajo efectivamente realizado; y el costo real (AC), lo gastado en ese trabajo. De su comparación derivan las variaciones e índices del cuadro 2.5, que la séptima edición de la guía PMBOK incluye entre las métricas del dominio de desempeño de medición (Project Management Institute, 2021). La dificultad práctica está en el valor ganado, porque exige medir el avance físico; la norma admite para ello varios métodos, entre ellos la fórmula fija, los hitos ponderados, el porcentaje completado y la medición física (Project Management Institute, 2019).

Cuadro 2.5. Variaciones, índices y pronóstico de la gestión del valor ganado

Indicador	Cálculo	Lectura
Variación de costo (CV)	$EV - AC$	Negativa: el trabajo realizado costó más de lo presupuestado
Variación de cronograma (SV)	$EV - PV$	Negativa: se hizo menos trabajo del planificado a la fecha
Índice de desempeño de costo (CPI)	EV/AC	Menor que 1: sobre costo en lo ejecutado
Índice de desempeño de cronograma (SPI)	EV/PV	Menor que 1: atraso respecto del plan
Estimación a la terminación (EAC)	BAC/CPI (forma básica)	Costo final proyectado si se mantiene el desempeño actual; BAC es el presupuesto total

Fuente: Elaboración propia, 2026, en base a Anbari (2003), Fleming & Koppelman (2010) y Project Management Institute (2019).

En un contrato EPC el avance es la base del pago. La cláusula de precio y pago del Libro Plata establece que el contratista presenta estados de cuenta periódicos con el monto que considera

devengado y que el propio comitente los evalúa y paga, sin el ingeniero independiente que en otras formas de la suite emite el certificado de pago provisional (FIDIC, 2017). En la interpretación de este trabajo, la certificación de avance es el acto contractual por el cual el cliente reconoce como ejecutada una porción del alcance y habilita su facturación. En el ERP nuevo se modela como certificación, el ingreso ejecutado de un centro de costo, que el gerente del centro solicita y el área de compras registra y cobra, con estados de borrador, enviada y registrada. Con la misma lectura, esa entidad cumple la función del ciclo de estado de cuenta, certificación y pago de FIDIC: cada certificación es a la vez una medición de lo ejecutado y un ingreso reconocido.

Los índices del valor ganado se usan además como predictores, según se vio en la sección 2.4.2 con Vandevorde y Vanhoucke (2006) y Lipke et al. (2009). Ese uso es el antecedente directo del enfoque de este proyecto, con una precisión de alcance: el ERP nuevo no gestiona una estructura de desglose del trabajo, por lo que no aplica EVM en sentido estricto; lo que ofrece, por centro de costo, es un presupuesto con línea base y seguimiento, los costos imputados y la serie de certificaciones, con los que se prevé aproximar las variaciones e índices del cuadro anterior como variables del modelo, y, como los modelos se entrenan con el histórico del portal anterior, esas variables deben poder reconstruirse también en su esquema, lo que se verifica en el análisis de datos del Capítulo III.

2.5.3. *DESVIACIONES PRESUPUESTARIAS Y DE CRONOGRAMA*

Los sobrecostos y los retrasos en grandes proyectos son sistemáticos y no excepcionales. Flyvbjerg (2014) lo resume en la «ley de hierro» de los megaproyectos: por encima del presupuesto, por encima del plazo, una y otra vez; estima el gasto mundial en ellos entre seis y nueve billones de dólares anuales, cerca del ocho por ciento del producto bruto mundial, y explica la persistencia del patrón por un modelo de romper y arreglar, en el que los proyectos se corrigen sobre la marcha. En la literatura sobre proyectos EPC la evidencia coincide: Habibi et al. (2018) señalan que más de la mitad de los proyectos enfrenta retrasos significativos y escaladas de costo importantes, y en hidrocarburos Olaniran et al. (2015) recogen un reporte según el cual el 64 % de los megaproyectos en ejecución en el mundo presenta sobrecostos, y advierten que la investigación sobre por qué y cómo ocurren es limitada.

Las causas identificadas se repiten entre estudios. Assaf y Al-Hejji (2006), en una encuesta en Arabia Saudita, identifican 73 causas de retraso, encuentran que el 70 % de los proyectos considerados sufrió sobreplazo y que la causa más frecuente para las tres partes es la orden de cambio. Olawale y Sun (2010) distinguen las causas de las desviaciones de los factores que impiden controlarlas: en 250 organizaciones del Reino Unido, los cinco inhibidores principales del control de costo y plazo son los cambios de diseño, los riesgos e incertidumbres, la evaluación inexacta de la duración, la complejidad y el incumplimiento de los subcontratistas; sus medidas de mitigación se clasifican en preventivas, predictivas, correctivas y organizacionales. Para EPC, Habibi et al. (2018) revisan más de doscientos artículos en busca de indicadores adelantados de costo y plazo por fase de ingeniería, procura y construcción; confirman el cambio de diseño como causa principal, agregan la escasez de recursos y la fluctuación de precios, y observan que todo retraso termina generando sobre costo.

En el sector de ISI Mustang los estudios son menos numerosos pero apuntan en la misma dirección. Olaniran et al. (2015) sostienen que los sobre costos en petróleo y gas surgen de interacciones complejas entre las características del proyecto, las personas, la tecnología, la estructura y la cultura, y proponen la teoría del caos como marco para explicarlas. Un estudio preliminar en fabricación EPC para petróleo y gas en Malasia, basado en entrevistas a diez especialistas, vincula los sobre costos con la asignación de recursos y ubica en el presupuesto insuficiente el problema principal (Mohd Roshdi et al., 2022); su alcance es limitado. El rasgo común de estos trabajos, y de Merrow (2011), es que la desviación no obedece a una causa aislada sino a la combinación de muchas, lo que argumenta a favor de un modelo que integre múltiples variables en lugar de una regla de umbral sobre un solo indicador.

Para este proyecto, la literatura reseñada delimita qué se entiende por desviación. La desviación presupuestaria es la diferencia entre el costo real de un proyecto y su presupuesto, que en la nomenclatura del valor ganado corresponde a la variación y al índice de costo; la desviación de cronograma es la diferencia entre la fecha de cierre efectiva y la comprometida; y la desviación en certificaciones es la diferencia entre lo certificado a una fecha y lo planificado certificar, la forma en que el atraso o el sobre costo se hacen visibles en el ingreso; como problema de clasificación, la etiqueta se obtiene aplicando un umbral a esa diferencia, y la disponibilidad de lo planificado certificar en el histórico se confirma en el Capítulo III. Son las tres respuestas

que el módulo predictivo busca anticipar. Los indicadores adelantados de Habibi et al. (2018), las órdenes de cambio de Assaf y Al-Hejji (2006) y los inhibidores de Olawale y Sun (2010) orientan la búsqueda de variables predictoras en el histórico; la categoría de medidas predictivas de estos últimos autores es el lugar que ocupa un módulo de predicción integrado al ERP.

2.6. INGENIERÍA DE SOFTWARE

2.6.1. PROCESO DE DESARROLLO DE SOFTWARE

Un proceso de desarrollo de software es el conjunto de actividades que conduce a la producción de un producto de software. La norma ISO/IEC/IEEE (2017) establece un marco común de procesos del ciclo de vida, con terminología definida, que contiene procesos, actividades y tareas aplicables durante la adquisición, el suministro, el desarrollo, la operación, el mantenimiento y el retiro de sistemas, productos y servicios de software; los agrupa en procesos de acuerdo, de habilitación organizacional, de gestión técnica y técnicos. La norma aclara, sin embargo, que no prescribe ningún modelo de ciclo de vida ni metodología de desarrollo: quien la usa es responsable de elegir un modelo para su proyecto y de mapear sobre él los procesos y actividades. Esa distinción entre proceso (qué actividades se realizan) y modelo de ciclo de vida (cómo se ordenan en el tiempo) es la que organiza esta sección y la siguiente.

Sommerville (2016) presenta los modelos de proceso como representaciones abstractas y distingue tres: el modelo en cascada, dirigido por plan, con fases separadas de especificación y desarrollo; el desarrollo incremental, que intercala especificación, desarrollo y validación; y la integración y configuración, basada en reutilizar componentes existentes. Pressman y Maxim (2020) agrupan los modelos en cascada, incremental y evolutivos bajo la etiqueta de modelos prescriptivos, en contraste con los procesos ágiles. El modelo en cascada suele remontarse a Royce (1970), que propone una secuencia de requisitos de sistema, requisitos de software, análisis, diseño de programa, codificación, pruebas y operación, aunque el nombre es posterior y el propio autor advierte que esa implementación es riesgosa e invita al fracaso, porque la fase de pruebas al final es el primer momento en que se experimentan, y no solo se analizan, los tiempos

y el almacenamiento, y un fallo ahí obliga a rediseñar. Sus correcciones incluyen documentar el diseño, hacerlo dos veces (que la versión entregada sea en realidad la segunda) e involucrar al cliente.

Esa crítica se formaliza en el modelo en espiral. Boehm (1988) lo presenta como un enfoque evolutivo guiado por el riesgo que provee un marco para conducir el proceso de software; según Larman y Basili (2003), su aporte fue hacer prominente el concepto de iteraciones guiadas por riesgo, con un paso explícito de evaluación de riesgos en cada iteración. Los métodos ágiles llevan la iteración más lejos: el manifiesto de Beck et al. (2001) antepone los individuos y sus interacciones a los procesos y herramientas, el software funcionando a la documentación exhaustiva, la colaboración con el cliente a la negociación contractual y la respuesta al cambio al seguimiento de un plan. Larman y Basili (2003) señalan que los diecisiete firmantes representaban métodos como XP, Scrum y DSDM, todos ellos iterativos e incrementales. El cuadro 2.6 resume los modelos comparados.

Cuadro 2.6. Modelos de proceso de software comparados

Modelo	Rasgo central	Fortaleza	Limitación
Cascada	Fases secuenciales con especificación cerrada antes del desarrollo	Simple de explicar; percibido como ordenado y medible	Los problemas se descubren en las pruebas finales; el cambio es costoso
Espiral	Ciclos guiados por el riesgo, con evaluación explícita en cada uno	Hace explícita la evaluación del riesgo en cada iteración	Su autor reconoce dificultades de uso por su elaboración incompleta
Incremental	Especificación, desarrollo y validación intercalados en entregas parciales	Menor costo del cambio; retroalimentación temprana; entrega útil más rápida	Menor visibilidad para la gestión; la estructura se degrada sin refactorización
Ágil	Iteraciones cortas, colaboración continua con el cliente, respuesta al cambio	Adaptación rápida; software funcionando como medida de avance	Depende de la colaboración continua con el cliente

Fuente: Elaboración propia, 2026, en base a Royce (1970), Boehm (1988), Sommerville (2016), Larman & Basili (2003) y Beck et al. (2001).

Para este trabajo, las actividades del proceso quedan fijas: cada incremento comprende definición de alcance, análisis, diseño, desarrollo y validación, que corresponden a procesos técnicos y de gestión técnica de la norma. Lo que se elige es el modelo de ciclo de vida que las ordena, y se prevé que sea el iterativo e incremental, por las razones que se desarrollan en la subsección siguiente.

2.6.2. METODOLOGÍA DE DESARROLLO ITERATIVA E INCREMENTAL

Larman y Basili (2003) reconstruyen la historia del desarrollo iterativo e incremental y muestran que, aunque muchos lo consideran una práctica moderna, su aplicación se remonta a mediados de los años cincuenta; un ejemplo temprano es el Proyecto Mercury de la NASA, a principios de los sesenta, que trabajó con iteraciones de medio día. La definición clásica proviene de Basili y Turner (1975), citados por Larman y Basili (2003): comenzar con una implementación simple de un subconjunto de los requisitos y mejorar iterativamente la secuencia de versiones hasta implementar el sistema completo, incorporando en cada iteración modificaciones de diseño junto con nuevas capacidades funcionales. El rasgo común de todos los enfoques de esta familia es evitar el paso único, secuencial, dirigido por documentos y con compuertas entre fases; y la palabra «iterativo» implica no solo revisar el trabajo, sino avanzar evolutivamente.

La misma fuente documenta proyectos que fijaron de antemano el número de incrementos: el sistema de defensa de TRW para el ejército estadounidense se desarrolló en cinco iteraciones en 1972; y el proyecto Trident de IBM, en cuatro iteraciones de unos seis meses. En el plano normativo, la norma militar Mil-Std-498 de 1994 reemplazó la exigencia de cascada de una sola pasada por un desarrollo en una o más construcciones incrementales, cada una de las cuales implementa un subconjunto especificado de las capacidades planificadas. Sommerville (2016) resume los beneficios del desarrollo incremental frente a la cascada: menor costo de acomodar cambios en los requisitos, retroalimentación más fácil del cliente y entrega más rápida de software útil; y sus problemas: menor visibilidad para la gestión y degradación de la estructura si no se refactoriza.

Tres condiciones del proyecto justifican la elección del modelo iterativo e incremental sobre

la cascada. La primera es que el desarrollo lo realiza una sola persona: un modelo secuencial concentraría toda la validación al final, que es exactamente el riesgo que Royce (1970) describe. La segunda es que el alcance se fija por incrementos: cada uno implementa un subconjunto especificado de las capacidades planificadas, en la lógica de las construcciones incrementales de Mil-Std-498, y cierra con una validación propia, de modo que al terminar cada uno existe software funcionando. La tercera es la dependencia de los datos: las variables predictoras y los casos de uso solo se confirman al explorar el histórico del portal anterior, lo que impide cerrar la especificación completa de antemano; el modelo incremental es el que abarata ese cambio. Como la norma ISO/IEC/IEEE (2017) deja la elección del modelo de ciclo de vida al proyecto, esta decisión no contradice el marco de procesos.

Se prevé, en consecuencia, un desarrollo en cinco incrementos: los procesos del portal anterior migrados al ERP, la autenticación centralizada con Keycloak, el pipeline de datos y el entrenamiento de los modelos, el microservicio de predicción con su integración al ERP y al tablero, y la integración con Power BI y MCP. La documentación de cada incremento en el Capítulo IV mitiga el problema de visibilidad que señala Sommerville; la degradación de la estructura se atiende con la arquitectura por capas que se describe a continuación.

2.6.3. *ARQUITECTURA DE SOFTWARE POR CAPAS*

Bass et al. (2021) entienden la arquitectura de software como el conjunto de estructuras necesarias para razonar sobre el sistema, compuestas por elementos, relaciones entre ellos y propiedades de ambos, y distinguen las estructuras de módulos, entre ellas las capas, de las de componentes y conectores y de las de asignación al entorno de despliegue. Richards y Ford (2020) añaden que las decisiones de arquitectura definen las reglas de cómo debe construirse el sistema: por ejemplo, que solo las capas de negocio y de servicios puedan acceder a la base de datos, con lo que se impide que la capa de presentación haga llamadas directas.

El patrón en capas es una estructura de módulos. Sommerville (2016) lo describe como la organización del sistema en capas, cada una con funcionalidad relacionada, donde cada capa provee servicios a la capa superior y las inferiores representan los servicios centrales; su ventaja

principal es que permite reemplazar una capa entera mientras se mantenga su interfaz, y sus desventajas son la dificultad de lograr una separación limpia y el costo de rendimiento de atravesar varios niveles para atender una petición. Richards y Ford (2020) formulan el principio de capas de aislamiento, según el cual una capa depende solo de la inmediata inferior. Fowler (2002) fija el vocabulario para aplicaciones empresariales con tres capas principales, presentación, lógica de dominio y fuente de datos, y señala que esas capas lógicas pueden ejecutarse en distintos nodos físicos.

Cuando la separación se lleva más allá de un proceso, se llega a los microservicios. J. Lewis y Fowler (2014) los definen como un enfoque para desarrollar una única aplicación como un conjunto de servicios pequeños, cada uno ejecutándose en su propio proceso y comunicándose mediante mecanismos ligeros, a menudo una API HTTP de recursos, y entre sus características cuentan la organización alrededor de capacidades de negocio, la gestión descentralizada de datos y la gobernanza descentralizada, que admite distintas tecnologías por servicio. Newman (2021) trata la extracción de capacidades de un monolito hacia servicios, la comunicación síncrona por HTTP entre ellos y la autenticación entre servicios. Richards y Ford (2020) ponen capas y microservicios en el mismo marco comparativo: el estilo distribuido aporta contexto acotado, aislamiento de datos y una capa de API, a cambio de costos operativos.

En este trabajo se combinan ambos niveles. Dentro de cada servicio se aplica el patrón en capas: en el ERP, controladores, servicios de dominio y acceso a datos con Prisma sobre PostgreSQL, y Angular como presentación; en el módulo predictivo, rutas, lógica de predicción y acceso a modelos y datos. Entre servicios, el módulo predictivo se construye como un microservicio en FastAPI separado del ERP en NestJS: es una decisión de diseño justificada porque corre en su propio proceso y expone una API HTTP, es dueño de sus modelos y de sus datos analíticos, que obtiene como copias de solo lectura de las fuentes, usa un lenguaje distinto al del ERP en la lógica de la gobernanza descentralizada, y puede reentrenarse y desplegarse sin tocar el ERP, lo que responde a la modificabilidad y la integrabilidad que Bass et al. (2021) tratan como atributos de calidad. Todos los servicios validarán la identidad contra Keycloak y se desplegarán en contenedores Docker en un servidor propio de la empresa. Las seis capas del módulo predictivo planteadas en el perfil no son este patrón, sino etapas del flujo de datos repartidas entre esos servicios; su diseño detallado corresponde a la sección 3.4.

2.7. TECNOLOGÍAS DEL STACK DE DESARROLLO

Esta sección presenta las tecnologías con las que se construye el sistema y justifica su elección. Conviene distinguir dos grupos. El stack del ERP (NestJS, Prisma, PostgreSQL y Angular) está fijado por el sistema existente y este trabajo lo toma como dado. El stack del módulo predictivo (Python, DuckDB, Polars, Scikit-learn, XGBoost y FastAPI) es una decisión de diseño de este proyecto, todavía no implementada, y es la que requiere justificación. Keycloak y el servidor MCP abarcan ambos grupos, y Power BI solo el ERP; los tres se tratan en las secciones 2.8 a 2.10.

2.7.1. *PYTHON Y SU ECOSISTEMA ML*

Python es un lenguaje de programación interpretado, interactivo y orientado a objetos, que admite además los paradigmas procedural y funcional, es portable entre Linux, macOS y Windows, y es extensible en C o C++ (Python Software Foundation, s.f.). Esta última propiedad explica la forma que tomó su ecosistema científico: las bibliotecas numéricas exponen una interfaz en Python sobre un núcleo compilado. La base de ese ecosistema es NumPy, que Harris et al. (2020) presentan como la biblioteca principal de programación con arreglos del lenguaje y como el fundamento sobre el que se construye el resto de las bibliotecas científicas, con el papel adicional de capa de interoperabilidad entre ellas. La programación con arreglos ofrece una sintaxis compacta para operar sobre vectores y matrices completos en lugar de elemento por elemento; ese estilo vectorizado reaparece, con otra implementación, en los motores de DuckDB y Polars.

Sobre NumPy se apoya pandas, presentada por McKinney (2010) como una biblioteca de estructuras de datos para el cómputo estadístico, motivada por los problemas prácticos de trabajar con los conjuntos de datos propios de las finanzas y la estadística. Sus dos estructuras primarias son la *Series*, unidimensional y etiquetada, y el *DataFrame*, tabular y con columnas de tipos heterogéneos; la biblioteca resuelve de forma nativa el manejo de valores faltantes, la alineación por etiquetas, las operaciones de agrupamiento y combinación, y funciones específicas para series temporales como ventanas móviles, desplazamientos y rezagos (pandas development team, s.f.). En este proyecto pandas cumple dos funciones: la exploración del histórico en el Capítulo III y el puente hacia las bibliotecas de modelado, que aceptan *DataFrame* como entrada.

La capa de modelado la forman Scikit-learn y XGBoost. Pedregosa et al. (2011) presentan Scikit-learn como una biblioteca de aprendizaje automático que pone el énfasis en la facilidad de uso, el desempeño, la documentación y la consistencia de su interfaz, con licencia BSD y dependencias mínimas. XGBoost, cuyo algoritmo se describió en la sección 2.2, se emplea aquí como biblioteca: su paquete para Python incluye estimadores compatibles con la interfaz de Scikit-learn (`XGBClassifier` y `XGBRegressor`), lo que permite usarlos dentro de los *pipelines* y de las rutinas de validación cruzada de esa biblioteca (T. Chen & Guestrin, 2016; xgboost developers, s.f.). La elección de Python para el módulo predictivo responde a que toda la cadena (arreglos, tablas, estimadores y servicio de predicción) se resuelve en un mismo lenguaje, y a que los tres componentes restantes de esa cadena, DuckDB, Polars y FastAPI, se usan desde Python. La versión concreta del intérprete y de cada biblioteca se fija en el Capítulo IV al construir el pipeline.

2.7.2. DUCKDB Y POLARS

La decisión de no ejecutar el análisis sobre la base de datos de producción del ERP tiene un fundamento de arquitectura. Stonebraker y Çetintemel (2005) sostienen que la arquitectura tradicional de los sistemas de gestión de bases de datos, diseñada y optimizada para el procesamiento de datos de negocio, se ha usado para sostener aplicaciones con características y requisitos muy distintos, y argumentan que la idea de un motor único para todos los usos ya no es aplicable: el mercado se fragmenta en motores independientes, y el almacén de datos es uno de los dos ejemplos con que sustentan el argumento. Trasladado a este proyecto, el procesamiento transaccional en línea (*online transaction processing*, OLTP) del ERP y el procesamiento analítico en línea (*online analytical processing*, OLAP) del pipeline Medallion tienen requisitos distintos y se resuelven con motores distintos. El argumento es de arquitectura, no de rendimiento medido en este trabajo.

El motor analítico elegido es DuckDB. Raasveldt y Mühleisen (2019) lo presentan como un sistema de gestión de datos diseñado para ejecutar consultas SQL analíticas embebido en otro proceso, una necesidad que la popularidad de SQLite hacía evidente y que ningún sistema cubría para cargas analíticas; se distribuye como software libre con licencia permisiva. Según su documentación, no tiene dependencias externas, usa un motor de ejecución columnar y

vectorizado, que procesa un lote de valores por operación, y es extensible en tipos, funciones y formatos (DuckDB Foundation, s.f.-d). Dos extensiones justifican su papel en la capa Silver: la de MySQL permite consultar directamente una instancia MySQL en ejecución y cargar sus tablas en DuckDB, y la de PostgreSQL hace lo propio con esa base, ambas mediante la sentencia `ATTACH` (DuckDB Foundation, s.f.-b, s.f.-c). Un solo motor SQL, embebido en el proceso Python del ETL y sin servidor adicional que desplegar, lee así el histórico del portal anterior, los proyectos activos del ERP nuevo y los archivos de la capa Bronze. La documentación de la extensión MySQL no indica versiones mínimas soportadas, por lo que su compatibilidad con MySQL 5.7, la versión del portal anterior, se verifica empíricamente en el Capítulo IV.

Para la transformación de la capa Gold se prevé utilizar Polars, una biblioteca de `DataFrame` con núcleo escrito en Rust y disponible para Python (Polars developers, s.f.-b). Su documentación declara ejecución paralela entre los núcleos del procesador, un motor de consultas vectorizado, una interfaz de flujo (*streaming*) para procesar resultados sin tener todos los datos en memoria, y soporte de Apache Arrow. Su interfaz perezosa (*lazy*) difiere la ejecución hasta que se solicita el resultado, lo que le permite filtrar filas y seleccionar columnas lo antes posible durante la lectura (empuje de predicados y de proyecciones); la documentación recomienda preferirla salvo en trabajo exploratorio (Polars developers, s.f.-a). Esas propiedades, y no una comparación de velocidad con pandas que no se ha verificado con fuentes independientes, motivan la elección. El traspaso entre las dos capas es directo: DuckDB consulta de forma nativa los `DataFrame` de Polars presentes en el ámbito de ejecución y devuelve sus resultados como `DataFrame` de Polars mediante la integración con Arrow, sin serializar a disco (DuckDB Foundation, s.f.-a).

2.7.3. FASTAPI COMO MICROSERVICIO

El servicio de predicción se construye sobre el estándar ASGI (*Asynchronous Server Gateway Interface*), que su especificación define como sucesor de WSGI, el estándar de compatibilidad entre servidores web, marcos de trabajo y aplicaciones en Python. Una aplicación WSGI es un único invocable síncrono que recibe una petición y devuelve una respuesta, lo que no admite conexiones de larga duración; ASGI extiende ese contrato al Python asíncrono (Django Software Foundation, s.f.). Sobre el estándar se apoya Starlette, un marco ligero para construir servicios

web asíncronos, con pocas dependencias obligatorias y código completamente anotado con tipos (Christie et al., s.f.), y lo ejecuta un servidor ASGI como Uvicorn (Ramírez, s.f.-b).

La segunda pieza es Pydantic, biblioteca de validación de datos en la que el esquema de validación y serialización se controla mediante anotaciones de tipo; su lógica central está escrita en Rust, admite un modo estricto sin conversión de tipos y uno laxo con coerción, y emite JSON Schema para integrarse con otras herramientas (Colvin et al., s.f.). FastAPI combina ambas: según su documentación, se apoya en Starlette para la parte web y en Pydantic para la parte de datos, y está basado en los estándares abiertos OpenAPI y JSON Schema, lo que le permite generar documentación interactiva automática (Swagger UI y ReDoc) y habilitar la generación de clientes en otros lenguajes (Ramírez, s.f.-a). FastAPI es, en consecuencia, un marco ASGI (Ramírez, s.f.-b).

Tres razones justifican su elección para el módulo predictivo. Primero, los modelos entrenados con Scikit-learn y XGBoost viven en Python; servirlos desde Python evita reimplementarlos o trasladarlos a otro entorno de ejecución. Segundo, Pydantic define el contrato de entrada y salida del endpoint de predicción (tipos, rangos y valores faltantes admitidos) y FastAPI lo publica como especificación OpenAPI, que es el contrato que el backend NestJS consume para tipar su cliente. Tercero, el modelo asíncrono de ASGI permite que la obtención de las claves públicas de Keycloak para validar tokens (sección 2.8) y otras operaciones de entrada y salida no bloqueen al proceso mientras espera la respuesta. El servicio se despliega como un contenedor independiente del ERP, de modo que el modelo puede actualizarse sin redespargar el backend. Las cifras de rendimiento y productividad que la documentación de FastAPI publica sobre sí misma no se toman como evidencia, por tratarse de autoevaluaciones sin estudio independiente.

2.7.4. NESTJS Y ANGULAR

El backend del ERP usa NestJS, un marco para construir aplicaciones de servidor en Node.js que está escrito en TypeScript y lo soporta por completo, combina elementos de la programación orientada a objetos, funcional y reactiva funcional, y utiliza Express como servidor HTTP por defecto (Myśliwiec et al., s.f.-a); el ERP corre NestJS 11 sobre Express y Node 22. Su unidad

de organización es el módulo, una clase anotada con el decorador `@Module()`: toda aplicación tiene al menos un módulo raíz desde el que Nest construye el grafo de la aplicación, y cada módulo encapsula sus proveedores, de modo que solo pueden inyectarse los propios o los exportados explícitamente por módulos importados (Myśliwiec et al., s.f.-b). Esa encapsulación permite agregar al ERP, como módulos separados, la integración con el servicio de predicción y la API para Power BI (sección 2.10); el servidor MCP, en cambio, corre como proceso independiente (sección 2.9).

El acceso a datos lo resuelve Prisma, un ORM (*object-relational mapping*) para Node.js y TypeScript que ofrece acceso a la base con seguridad de tipos, migraciones y un editor visual de datos; su esquema es la única fuente de verdad de los modelos de base de datos y de aplicación, y las consultas se validan en tiempo de compilación (Prisma Data, Inc., s.f.). El ERP usa Prisma 7.8, versión en la que el adaptador de controlador es obligatorio para las conexiones directas y la configuración vive en `prisma.config.ts`. La base es PostgreSQL 17, un sistema de gestión de bases de datos objeto-relacional que soporta gran parte del estándar SQL y ofrece integridad transaccional y control de concurrencia multiversión (The PostgreSQL Global Development Group, s.f.); esas son las propiedades OLTP que, según el argumento de Stonebraker y Çetintemel (2005) recogido en la subsección anterior, conviene no mezclar con la carga analítica.

El frontend es Angular 21.2, un marco web mantenido por un equipo de Google que organiza el código en componentes encapsulados e incorpora un modelo de reactividad de grano fino basado en *signals*, inyección de dependencias, enrutamiento y formularios con validación (Google LLC, s.f.-b). El ERP lo usa con renderizado del lado del servidor (*server-side rendering*, SSR), que genera en el servidor una página HTML completa para cada petición y luego la reutiliza en el cliente mediante hidratación (Google LLC, s.f.-a). Para una aplicación interna con inicio de sesión, el beneficio relevante no es el posicionamiento en buscadores sino el primer render rápido del dashboard donde se muestran las predicciones. La interfaz se compone con PrimeNG 21.1 y Tailwind 4.1. Todos los componentes se despliegan con Docker, que empaqueta y ejecuta cada aplicación en un contenedor aislado que sirve como unidad de distribución (Docker, Inc., s.f.); el monorepo define los servicios en un archivo de Docker Compose, y la portabilidad de las imágenes permite prever la migración desde el servidor propio de la empresa a la nube como trabajo futuro. El cuadro 2.7 resume el stack completo.

Cuadro 2.7. Tecnologías del sistema por capa de la arquitectura

Tecnología	Capa	Rol	Versión
<i>ERP existente</i>			
NestJS (Express, Node)	Backend del ERP	API y módulos de negocio; integración con FastAPI y API para Power BI; consumido por el servidor MCP	11 (Node 22)
Prisma	Backend del ERP	ORM y migraciones	7.8
PostgreSQL	Base OLTP del ERP	Datos transaccionales; fuente de inferencia	17
Angular (SSR)	Frontend del ERP	Interfaz de usuario y dashboard de predicciones	21.2
PrimeNG / Tailwind	Frontend del ERP	Componentes de interfaz y estilos	21.1 / 4.1
PHP + MySQL	Portal anterior	Fuente del histórico (entrenamiento)	MySQL 5.7
<i>Módulo predictivo (por implementar)</i>			
Python (NumPy, pandas)	Pipeline y modelos	Lenguaje del ETL, exploración y modelado	Cap. IV
DuckDB	Silver	Motor OLAP embebido; lee MySQL, PostgreSQL y archivos	Cap. IV
Polars	Gold	Ingeniería de características	Cap. IV
Scikit-learn / XGBoost	Modelos	Pipelines, validación cruzada y <i>gradient boosting</i>	Cap. IV
FastAPI (Starlette, Pydantic, Uvicorn)	Servicio de predicción	Endpoint de inferencia con contrato OpenAPI	Cap. IV
<i>Transversales</i>			
Keycloak	Identidad	SSO con OAuth 2.0 / OpenID Connect (sección 2.8)	Cap. IV
Servidor MCP	Integración con IA	Tools sobre el ERP y las predicciones (sección 2.9)	Cap. IV
Power BI	Inteligencia de negocios	Reportes descriptivos vía API NestJS (sección 2.10)	—
Docker / Docker Compose	Despliegue	Contenedores en servidor propio; migración a nube como trabajo futuro	—

Fuente: Elaboración propia, 2026, en base a la documentación y los repositorios del ERP de ISI Mustang.

2.8. GESTIÓN DE IDENTIDAD Y ACCESO

2.8.1. OAUTH 2.0 Y OPENID CONNECT

La gestión de identidad y acceso reúne dos procesos distintos: la autenticación, que identifica y valida a un usuario, y la autorización, que le concede acceso (Keycloak Project, 2026d). El estándar de referencia para el segundo es OAuth 2.0 (*Open Authorization*), un marco que permite a una aplicación obtener acceso limitado a un servicio HTTP, en nombre del propietario de un recurso o por cuenta propia (Hardt, 2012). La especificación define cuatro roles: el propietario del recurso (*resource owner*), que cuando es una persona se llama usuario final; el servidor de recursos (*resource server*), que aloja los recursos protegidos y responde a peticiones que presentan tokens de acceso; el cliente, que hace esas peticiones en nombre del propietario; y el servidor de autorización (*authorization server*), que emite los tokens tras autenticar al propietario. Un único servidor de autorización puede emitir tokens aceptados por múltiples servidores de recursos (Hardt, 2012), propiedad en la que se apoya el inicio de sesión único (*single sign-on*, SSO) que adopta este proyecto.

OAuth 2.0 distingue clientes confidenciales, que mantienen en secreto sus credenciales porque se ejecutan en un servidor, y clientes públicos, como una aplicación nativa o una aplicación en navegador (Hardt, 2012). Sobre esa distinción se organizan sus cuatro concesiones (*grants*). En el código de autorización (*authorization code*) el cliente redirige al usuario al servidor de autorización, que lo autentica y devuelve un código que el cliente canjea por el token; las credenciales del usuario nunca se comparten con el cliente y el token no pasa por el navegador. En el flujo implícito el token se emite directamente al navegador, el servidor no autentica al cliente y el token puede quedar expuesto a otras aplicaciones. En la concesión por credenciales del propietario (*resource owner password credentials*, ROPC) el cliente recibe directamente usuario y contraseña, y la de credenciales de cliente (*client credentials*) se usa cuando el cliente actúa por cuenta propia y está reservada a clientes confidenciales (Hardt, 2012).

En un cliente público el código de autorización puede ser interceptado en un canal no protegido por TLS. La prueba de clave para el intercambio de código (*Proof Key for Code Exchange*, PKCE)

mitiga ese ataque: el cliente genera un verificador secreto, envía un derivado en la petición de autorización y presenta el verificador original al canjear el código, de modo que el servidor comprueba que quien canjea es quien inició el flujo (Sakimura et al., 2015). La práctica actual de seguridad para OAuth 2.0 (Lodderstedt et al., 2025) lo vuelve obligatorio para los clientes públicos, incluidas las aplicaciones web; desaconseja el flujo implícito; establece que ROPC no debe usarse, porque expone las credenciales del propietario al cliente; y exige que cada servidor de recursos verifique en cada petición que el token fue emitido para él. El borrador de OAuth 2.1 (Hardt et al., 2026), aún en elaboración en el IETF, consolida esa dirección al incorporar PKCE al código de autorización y omitir los flujos implícito y ROPC. Fett et al. (2016) aportan el respaldo académico: en el primer análisis formal extensivo de OAuth 2.0 descubren cuatro ataques hasta entonces desconocidos y demuestran que, con las correcciones propuestas, el protocolo cumple autorización, autenticación e integridad de sesión. La seguridad del marco depende, por tanto, de implementar el conjunto completo de mitigaciones, lo que favorece adoptar un proveedor que ya las incorpora.

El resultado de estos flujos es un token de acceso, una cadena que representa una autorización y que suele ser opaca para el cliente, acompañada opcionalmente de un token de refresco para renovarlo (Hardt, 2012). Un tipo de token es el portador (*bearer*): cualquier parte que lo posea puede usarlo sin demostrar posesión de material criptográfico, por lo que debe protegerse en almacenamiento y en tránsito y enviarse en la cabecera *Authorization* (Jones & Hardt, 2012). Cuando el token es autocontenido, el formato habitual es el token web JSON (*JSON Web Token*, JWT): una representación compacta de afirmaciones (*claims*) sobre un sujeto, firmada digitalmente, con nombres registrados como emisor (*iss*), sujeto (*sub*), audiencia (*aud*) y expiración (*exp*) (Jones et al., 2015). OAuth 2.0, sin embargo, no dice quién es el usuario: sin un perfilado adicional es incapaz de proporcionar información sobre la autenticación del usuario final (Sakimura et al., 2023). OpenID Connect (OIDC) es la capa de identidad construida sobre OAuth 2.0 que cubre ese vacío: el cliente, llamado parte confiante (*relying party*), solicita el ámbito *openid* a un proveedor de OpenID (OP) y recibe, además del token de acceso, un token de identidad (*ID token*) en formato JWT con afirmaciones sobre la autenticación realizada, que debe validar comprobando emisor y audiencia (Sakimura et al., 2023).

El cuadro 2.8 traslada estos roles y flujos al proyecto. El ERP de ISI Mustang autentica hoy con

un endpoint propio que recibe usuario y contraseña y emite un JWT generado por el mismo servidor. Ese esquema sirve para una sola aplicación, pero el proyecto agrega un servicio de predicción en FastAPI, un servidor MCP y una API para Power BI, y Góes de Almeida y Canedo (2022) muestran que en arquitecturas de varios servicios cada uno debe asegurarse individualmente, que el acceso no autorizado a uno puede comprometer el sistema completo, y que OAuth 2.0, OpenID Connect y JWT están entre los mecanismos documentados para ese escenario. Con un servidor de autorización externo ningún servicio recibe contraseñas, cada uno valida los tokens por firma y audiencia, y la identidad es la misma en todos. Se prevé que Angular opere como cliente público con código de autorización y PKCE, que NestJS invoque a FastAPI con credenciales de cliente, y que NestJS, FastAPI y el servidor MCP sean servidores de recursos con audiencias distintas. El acceso de Power BI a la API de NestJS y el del servidor MCP hacia el ERP y FastAPI se deciden en el Capítulo III (véanse 2.9 y 2.10).

Cuadro 2.8. Roles y flujos de OAuth 2.0 / OpenID Connect y su uso en el proyecto

Rol o flujo	Definición	En el proyecto
<i>Roles</i>		
Servidor de autorización / OP	Autentica al usuario y emite tokens de acceso e identidad	Keycloak (véase 2.8.2)
Propietario del recurso	Usuario final que concede el acceso	Personal de ISI Mustang que usará el ERP
Cliente público	Aplicación en navegador o nativa, sin secreto	Frontend Angular del ERP
Cliente confidencial	Aplicación en servidor que guarda sus credenciales	NestJS al invocar a FastAPI
Servidor de recursos	Acepta tokens y sirve recursos protegidos	API NestJS, servicio FastAPI y servidor MCP, cada uno con audiencia propia
<i>Flujos</i>		
Código de autorización + PKCE	Inicio de sesión interactivo de personas	Angular ante Keycloak
Credenciales de cliente	Acceso máquina a máquina, solo clientes confidenciales	NestJS hacia FastAPI
Implícito	Token emitido directamente al navegador; desaconsejado	No se usa
ROPC	El cliente recibe usuario y contraseña; no debe usarse	No se usa; el inicio de sesión propio del ERP, equivalente en sus propiedades, se sustituye por código + PKCE

Fuente: Elaboración propia, 2026, en base a Hardt (2012), Lodderstedt et al. (2025) y Sakimura et al. (2023).

2.8.2. KEYCLOAK

Keycloak es una solución de inicio de sesión único para aplicaciones web y servicios web RESTful cuyo objetivo es simplificar la seguridad: las funciones que los desarrolladores tendrían que escribir por sí mismos vienen incorporadas (Keycloak Project, 2026d). Es un servidor separado que se administra en la red propia; las aplicaciones se configuran para apuntar a él y ser aseguradas por él mediante protocolos abiertos como OpenID Connect o SAML 2.0. Thorgersen y Silva (2023) le dedican una monografía que complementa la documentación oficial. En los términos de la sección anterior, Keycloak es a la vez servidor de autorización de OAuth 2.0

y proveedor de OpenID Connect: expone los endpoints de autorización, token, información de usuario, certificados e introspección, que las aplicaciones descubren a partir de un documento de configuración publicado por cada dominio (Keycloak Project, 2026c).

Su modelo se organiza en pocos conceptos (Keycloak Project, 2026d). Un dominio (*realm*) gestiona un conjunto de usuarios, credenciales, roles y grupos; los usuarios inician sesión en un dominio y los dominios están aislados entre sí. Los clientes son las entidades que pueden pedir a Keycloak que autentique a un usuario: aplicaciones que quieren asegurarse y ofrecer inicio de sesión único, o servicios que solo necesitan un token de acceso para invocar a otros servicios protegidos. Los roles identifican una categoría de usuario (administrador, gerente, empleado) y las aplicaciones asignan permisos a roles antes que a usuarios individuales; la asignación de roles puede encapsularse en los tokens, de modo que cada aplicación decide los permisos sobre sus recursos. Keycloak es además un proveedor de identidad (*identity provider*, IdP) capaz de federar la autenticación hacia otros proveedores OIDC o SAML y hacia directorios LDAP, y soporta cualquier plataforma que cuente con una librería de parte confiante de OpenID Connect; Thorgersen y Silva (2023) documentan el reemplazo del adaptador propio para Spring por librerías estándar.

El aspecto más relevante para una arquitectura de varios servicios es cómo se validan los tokens. Los tokens de acceso de Keycloak son JWT firmados con JSON Web Signature, por lo que pueden validarse localmente con la clave pública del dominio emisor, obtenida del endpoint de certificados y almacenada en caché; consultar el endpoint de introspección en cada petición, en cambio, puede ser lento y sobrecargar el servidor (Keycloak Project, 2026c). Es el patrón que Thorgersen y Silva (2023) describen en su aplicación de ejemplo: el token de identidad identifica al usuario, el token de acceso transporta los permisos para invocar el backend, y este valida los tokens y controla el acceso por los roles que lee de ellos. Keycloak implementa el código de autorización, con el que la aplicación obtiene token de acceso, de refresco y de identidad, y las credenciales de cliente, que no forman parte de OpenID Connect sino de OAuth 2.0; lista los flujos implícito y ROPC como desaconsejados y recomienda registrar las URI de redirección lo más específicas posible, en especial para clientes públicos (Keycloak Project, 2026c).

Estas propiedades explican por qué el proyecto reemplaza la autenticación propia del ERP

por Keycloak. Con el esquema actual la identidad vive dentro del backend en NestJS: cualquier servicio nuevo que quisiera reconocer al mismo usuario tendría que compartir el secreto con el que se firman los tokens o delegar la validación en el ERP, con el acoplamiento y el riesgo de compromiso en cadena que Góes de Almeida y Canedo (2022) señalan. Con Keycloak, un único dominio contiene a los usuarios y roles de ISI Mustang; Angular autentica a las personas con código de autorización y PKCE; NestJS, FastAPI y el servidor MCP validan localmente sus JWT, emitidos por el mismo dominio, con la clave pública de este, comprueban la audiencia y autorizan por los roles que trae; y NestJS invoca a FastAPI con credenciales de cliente, mientras que el acceso de Power BI y del servidor MCP se resuelve en el Capítulo III. Se prevé desplegar Keycloak en contenedor junto al resto de la plataforma, en el servidor propio de la empresa; las librerías con las que cada servicio valida los tokens y la configuración del dominio son decisiones de implementación que se documentan en el Capítulo IV.

2.9. MODEL CONTEXT PROTOCOL (MCP)

2.9.1. DEFINICIÓN Y ARQUITECTURA

El *Model Context Protocol* (MCP) es un protocolo abierto que permite la integración entre aplicaciones basadas en modelos de lenguaje de gran tamaño (*large language models*, LLM) y fuentes de datos y herramientas externas, y provee una forma estandarizada de conectar esos modelos con el contexto que necesitan (Model Context Protocol, 2026). Anthropic (2024) lo liberó en noviembre de 2024 con una motivación concreta: aun los modelos más sofisticados quedan aislados de los datos, tras silos de información y sistemas heredados, y cada nueva fuente exigía una implementación a medida. Hou et al. (2026) lo caracterizan como un estándar unificado, bidireccional y con descubrimiento dinámico, distinto de las integraciones manuales de API y de los *plugins* unidireccionales por ser abierto e independiente de la plataforma. En diciembre de 2025 Anthropic donó el protocolo a la Agentic AI Foundation, un fondo dirigido bajo la Linux Foundation, para garantizar su neutralidad respecto de proveedores (Soria Parra, 2025).

La arquitectura distingue tres participantes: los *hosts*, aplicaciones LLM que inician las

conexiones; los clientes, conectores dentro del host; y los servidores, servicios que proveen contexto y capacidades (Model Context Protocol, 2026). El host crea y administra los clientes, controla sus permisos y aplica las políticas de seguridad y consentimiento; cada cliente mantiene una conexión aislada con un único servidor, y el servidor, que puede ser un proceso local o un servicio remoto, opera con responsabilidades acotadas y no debería poder leer la conversación completa ni observar a otros servidores (Model Context Protocol, 2025). Los mensajes se codifican en JSON-RPC 2.0, un protocolo de llamada a procedimiento remoto (*remote procedure call*, RPC) ligero, sin estado e independiente del transporte (JSON-RPC Working Group, 2013): una solicitud lleva método, parámetros e identificador, y la respuesta devuelve un resultado o un error con el mismo identificador. La especificación define dos transportes: *stdio*, en el que el cliente lanza el servidor como subproceso, y *Streamable HTTP*, en el que el servidor atiende a múltiples clientes por un único *endpoint* HTTP (Model Context Protocol, 2025). La revisión vigente hizo el protocolo sin estado: eliminó las sesiones y cada solicitud lleva su versión y sus capacidades (Model Context Protocol, 2026).

Un servidor ofrece tres primitivas: los recursos (*resources*), contexto y datos para el usuario o el modelo; los *prompts*, mensajes con plantilla y flujos de trabajo; y las herramientas (*tools*), funciones que el modelo ejecuta (Model Context Protocol, 2026). Ehtesham et al. (2025) las clasifican según quién las controla: las herramientas por el modelo, los recursos por la aplicación y los *prompts* por el usuario. La herramienta es la primitiva central de este trabajo. Cada una declara nombre, descripción y un esquema de entrada en JSON Schema; el cliente las descubre con `tools/list`, las invoca con `tools/call` y recibe contenido no estructurado o estructurado, validable contra un esquema de salida (Model Context Protocol, 2025). La especificación separa los errores de protocolo de los errores de ejecución, como una regla de negocio violada, y recomienda que siempre exista una persona con capacidad de denegar una invocación. El cuadro 2.9 resume las primitivas y el uso que se prevé darles.

Cuadro 2.9. Primitivas del servidor MCP y su uso previsto en el proyecto

Primitiva	Control	Definición	Uso previsto
Herramientas (<i>tools</i>)	Modelo	Funciones que el modelo descubre e invoca, con esquema de entrada y salida	Consultas de lectura sobre el ERP (centros de costo, certificaciones, presupuesto) y sobre las predicciones del servicio FastAPI
Recursos (<i>resources</i>)	Aplicación	Contexto y datos que el host adjunta al modelo	No se contempla en el alcance; posible exposición futura de documentación del ERP
<i>Prompts</i>	Usuario	Mensajes con plantilla y flujos de trabajo	No se contempla; corresponden al asistente que queda como trabajo futuro

Fuente: Elaboración propia, 2026, en base a Model Context Protocol (2026) y Ehtesham et al. (2025).

El protocolo se apoya en la investigación sobre uso de herramientas por modelos de lenguaje. Schick et al. (2023) parten de que los modelos fallan en tareas básicas, como la aritmética o la consulta de hechos, y entrenan uno capaz de decidir qué API llamar, cuándo, con qué argumentos y cómo incorporar el resultado. Qin et al. (2024) llevan el problema a más de 16 000 API REST reales, y Patil et al. (2024) identifican el modo de fallo característico: argumentos incorrectos y alucinación del uso de una API, que se mitiga cuando el modelo consulta documentación en tiempo de ejecución. Esos trabajos resuelven el lado del modelo; MCP estandariza el opuesto, cómo un servidor describe sus herramientas con esquemas explícitos y las ejecuta, de modo que cualquier host las descubra sin los adaptadores a medida que Ehtesham et al. (2025) señalan como obstáculo entre marcos de agentes. Los mismos autores sitúan a MCP como el primer escalón, el acceso a herramientas, frente a los protocolos entre agentes, que quedan fuera del alcance de este trabajo.

2.9.2. INTEGRACIÓN DE SISTEMAS EMPRESARIALES CON MODELOS DE LENGUAJE

Un ERP acumula datos transaccionales estructurados, actualizados a diario y sujetos a permisos por usuario, y eso condiciona cómo ponerlos al alcance de un modelo de lenguaje. P. Lewis et al. (2020) señalan que los modelos preentrenados almacenan conocimiento factual en sus parámetros, pero acceden a él y lo manipulan con precisión limitada, y que dar procedencia

a sus respuestas y actualizar su conocimiento siguen siendo problemas abiertos. Su propuesta, la generación aumentada por recuperación (*retrieval-augmented generation*, RAG), combina esa memoria paramétrica con un índice vectorial consultado por un recuperador neuronal y produce lenguaje más específico y factual. Ese patrón recupera pasajes por similitud. En un ERP, en cambio, la pregunta habitual (cuánto lleva certificado un centro de costo, qué probabilidad de sobre costo estima el modelo) se responde con una consulta estructurada sobre la fuente vigente y con permisos acordes a quien pregunta, según el mecanismo de acceso que se defina en el Capítulo III. Anthropic (2024) plantea que un desarrollador puede exponer sus datos mediante servidores MCP o construir las aplicaciones cliente que los consumen; este proyecto toma el primer rol, con lo que la lógica de negocio y el control de acceso permanecen en el ERP.

El servidor del proyecto corre en Docker como proceso independiente y lo consumen clientes remotos, por lo que el transporte que corresponde es Streamable HTTP. La especificación define su flujo de autorización para los transportes HTTP, mientras que para *stdio* recomienda tomar las credenciales del entorno (Model Context Protocol, 2026). En ese flujo el servidor MCP actúa como servidor de recursos OAuth 2.1, el cliente MCP como cliente OAuth 2.1 y el servidor de autorización emite los tokens. El servidor MCP debe implementar *OAuth 2.0 Protected Resource Metadata* (Jones et al., 2025): ante una solicitud sin credenciales responde 401 con una cabecera *WWW-Authenticate* que remite a su documento de metadatos, donde declara su servidor de autorización. El token viaja como *bearer* en cada solicitud, nunca en la URL, y el servidor debe validar que fue emitido específicamente para él como audiencia (véase 2.8.1), conforme a *Resource Indicators for OAuth 2.0* (Campbell et al., 2020); un token inválido recibe 401 y un alcance insuficiente, 403 (Model Context Protocol, 2026).

De ahí se desprende una consecuencia de diseño. Si el servidor MCP consulta API ascendentes, actúa ante ellas como cliente OAuth con un token separado y no debe reenviar el que recibió del cliente MCP, para evitar el problema del delegado confundido (Model Context Protocol, 2026). En el proyecto, el token que valida el servidor MCP lleva como audiencia su URL canónica y no puede viajar al ERP en NestJS ni al servicio en FastAPI; hace falta un segundo token hacia esos servicios, cuyo mecanismo se define en los Capítulos III y IV. Del lado del proveedor de identidad, la guía oficial de Keycloak (2026) documenta el uso de Keycloak como servidor de autorización para MCP y declara que no soporta *Resource Indicators*; la audiencia se fija con un

client scope y un *Audience mapper* que incluye la URL del servidor MCP en el token, mientras el cliente MCP sigue enviando el parámetro *resource* porque la especificación lo exige aunque el servidor de autorización no lo procese. La verificación del token en sí se trata en la sección 2.8.

La autorización no agota la seguridad. Hou et al. (2026) sistematizan las amenazas por fase del ciclo de vida del servidor y por tipo de atacante, entre ellas el robo de credenciales durante la invocación y los conflictos de nombres que llevan a ejecutar la herramienta equivocada o escalar privilegios. Radosevich y Halloran (2025) demuestran que modelos de primera línea pueden ser inducidos a usar herramientas MCP para ejecutar código malicioso, tomar control remoto o robar credenciales, y proponen un auditor que genera casos adversarios a partir de lo que un servidor expone. La especificación reconoce que las herramientas representan ejecución de código arbitrario y establece que el servidor debe validar las entradas, controlar el acceso, limitar la tasa de invocaciones y sanear las salidas (Model Context Protocol, 2025). En este proyecto eso se traduce en un servidor que expone únicamente consultas de lectura acotadas sobre el ERP y las predicciones, sin nómina ni datos personales, con esquemas estrictos y acceso delegado en los alcances de Keycloak. Con ello el servidor deja el ERP y las predicciones listos para un asistente conversacional futuro; construir ese asistente queda fuera del alcance.

2.10. INTELIGENCIA DE NEGOCIOS

2.10.1. DEFINICIÓN Y REPORTES DESCRIPTIVOS

Negash (2004) formula la inteligencia de negocios (*business intelligence*, BI) desde los sistemas de información: los sistemas de BI combinan datos operativos con herramientas analíticas para presentar información compleja a quienes planifican y deciden, con el fin de mejorar la oportunidad y la calidad de los insumos de la decisión. Watson y Wixom (2007) la describen como un proceso con dos actividades principales: introducir los datos (*getting data in*), que corresponde al almacenamiento de datos tradicional y consiste en mover datos desde los sistemas fuente hacia un almacén integrado, y extraerlos (*getting data out*), cuando usuarios y aplicaciones acceden a ese almacén para reportes empresariales, consultas, procesamiento analítico y analítica predictiva.

Los mismos autores advierten que la primera actividad aporta por sí sola un valor limitado: la organización solo obtiene el valor completo cuando usa los datos para decidir.

Esa arquitectura tiene componentes reconocibles. Chaudhuri y Dayal (1997) presentan el almacén de datos y el procesamiento analítico en línea (*online analytical processing*, OLAP) como elementos esenciales del apoyo a la decisión y recorren sus piezas: herramientas de extracción, limpieza y carga (los procesos ETL de la sección 2.3), modelos de datos multidimensionales organizados en medidas y dimensiones, herramientas cliente de consulta y análisis, y gestión de metadatos. Kimball y Ross (2013) sistematizan el diseño de esa capa mediante el modelado dimensional: tablas de hechos que registran las medidas del negocio y tablas de dimensión que las describen, dispuestas en esquema en estrella.

El producto más visible de la BI es el reporte descriptivo. Lepenioti et al. (2020) distinguen en la analítica de negocios la descriptiva, la predictiva y la prescriptiva, que busca el mejor curso de acción futuro y constituye, según los autores, el siguiente paso en la madurez analítica, mientras que el foco académico e industrial sigue puesto en las dos primeras. Los reportes descriptivos suelen entregarse como tableros (*dashboards*): Eckerson (2010) los caracteriza como una capa de presentación sobre la infraestructura de BI que integra el monitoreo, el análisis y la gestión de indicadores de desempeño, y Few (2013) aporta los criterios de diseño visual de tableros para el monitoreo de un vistazo.

Estas nociones fijan el lugar de la BI en el proyecto. El pipeline de datos de la sección 2.3 es la actividad de introducir los datos; los reportes de Power BI y el tablero del ERP en Angular son dos canales para extraerlos. H. Chen et al. (2012) proponen un marco evolutivo que distingue etapas de la BI y la analítica según los datos que procesan. La separación entre canales es una decisión de arquitectura y no una definición, puesto que Watson y Wixom (2007) incluyen la analítica predictiva dentro de la BI. Con la tríada de Lepenioti et al. (2020) como criterio, se prevé que Power BI cubra la analítica descriptiva del ERP (horas, compras, presupuesto ejecutado y certificaciones por centro de costo), que el módulo de aprendizaje automático cubra la predictiva dentro de la interfaz transaccional, y que la prescriptiva quede como trabajo futuro. Las razones técnicas de esa separación se desarrollan en la subsección siguiente.

2.10.2. POWER BI

Power BI es la plataforma de analítica de negocios de Microsoft para conectar, visualizar y compartir datos; sus componentes principales son Power BI Desktop, aplicación gratuita de modelado y creación de reportes, y el servicio Power BI, que permite compartirlos mediante espacios de trabajo y refresco programado (Microsoft, 2023, 2026e). Power Query prepara los datos: extrae, transforma y carga desde cientos de fuentes, registra cada transformación como un paso en el lenguaje M y no modifica la fuente (Microsoft, 2026f). El modelado se apoya en DAX (*Data Analysis Expressions*), lenguaje de fórmulas sobre un modelo tabular con el que se definen medidas, columnas calculadas e inteligencia de tiempo (Microsoft, 2023). Los reportes se construyen sobre el modelo semántico (antes *dataset*), y *dashboard* designa allí una pantalla de mosaicos (Microsoft, 2026c); en este documento “tablero” conserva el sentido de gestión de la subsección anterior.

El modo de acceso a la fuente condiciona el diseño. En modo de importación los datos se cargan en la caché del modelo y los visuales no reflejan cambios hasta el siguiente refresco; en DirectQuery cada visual consulta la fuente en el momento. Microsoft recomienda importar por defecto, y documenta DirectQuery para bases de datos, mientras que los sitios web figuran entre las fuentes de importación (Microsoft, 2025b). En capacidad compartida cada modelo admite ocho refrescos programados por día, y si el servicio no alcanza la fuente por red se requiere una puerta de enlace de datos local (*on-premises data gateway*) (Microsoft, 2025a, 2025c); como la fase 1 corre en un servidor propio, su necesidad depende de si la API del ERP es alcanzable desde Internet, lo que resuelve el Capítulo III. Estas propiedades fundamentan la decisión anterior: una copia importada que se actualiza a lo sumo ocho veces al día sirve para reportes descriptivos, no para predicciones calculadas bajo demanda y atadas a la sesión del usuario.

El segundo condicionante es la autenticación. Power BI llega a la API de NestJS por el conector Web: `Web.Contents` admite, entre otros, los tipos `Anonymous`, `Web API` y `Organizational Account`, permite definir encabezados HTTP y lee JSON con `Json.Document`, pero solo hace POST de forma anónima (Microsoft, 2026d). El OAuth integrado de `Organizational Account` solo se inicia ante un 401 que remite a Microsoft Entra ID; más control sobre el flujo exige un conector personalizado (Microsoft, 2026a), que puede declarar OAuth con `StartLogin`, `FinishLogin`, `Refresh` y `PKCE`, o

el tipo Key, que por defecto envía la clave como contraseña de autenticación básica, de modo que un token *bearer* requiere construir el encabezado *Authorization* manualmente (Microsoft, 2026b). Como el proveedor de identidad es Keycloak, quedan dos caminos: un conector personalizado con OAuth y PKCE contra Keycloak, o una credencial de servicio emitida por Keycloak cuyo token se pasa en el encabezado desde *Web.Contents*, camino al que afecta la restricción sobre POST si el intercambio de credenciales se hiciera desde M. La elección se justifica en el Capítulo III y se construye en el incremento 5.

Frente a otras herramientas, el criterio deriva del alcance: la BI debe leer el ERP por una API protegida, no directamente de PostgreSQL. Apache Superset se conecta a bases de datos SQL (Apache Software Foundation, s.f.); Metabase documenta controladores para bases de datos y no API REST (Metabase, s.f.); Tableau accede a fuentes HTTP mediante un *web data connector* en JavaScript, cuya versión 2.0 está obsoleta desde la 2023.1 (Tableau (Salesforce), s.f.). Power BI trae de fábrica el conector Web con JSON y encabezados, y su herramienta de autoría es gratuita. El cuadro 2.10 resume la comparación, que no afirma una superioridad general sino la adecuación a este caso. Se prevé publicar el modelo semántico en el servicio Power BI y compartir los reportes con jefes de proyecto y administración para su consulta directa.

Cuadro 2.10. Herramientas de BI consideradas y su forma de acceso a los datos del ERP

Herramienta	Fuentes documentadas	Acceso a la API del ERP
Power BI	Más de 100 fuentes: bases de datos, servicios en la nube, archivos y fuentes web	Conector Web de fábrica: JSON, encabezados HTTP, clave de API. OAuth integrado solo con Entra ID; para Keycloak, conector personalizado o token en el encabezado
Apache Superset	Bases de datos SQL	Sin conector REST documentado; requeriría acceso directo a la base, excluido por el alcance
Metabase	Controladores para bases de datos (PostgreSQL, MySQL, SQL Server, entre otras)	Sin conector REST documentado; misma restricción
Tableau	Conectores nativos y <i>web data connector</i> para fuentes HTTP	Requiere programar un conector en JavaScript (WDC 3.0; la 2.0 está obsoleta)

Fuente: Elaboración propia, 2026, en base a Microsoft (2026e), Microsoft (2026d), Microsoft (2026a), Apache Software Foundation (s.f.), Metabase (s.f.) y Tableau (Salesforce) (s.f.).

CAPÍTULO III

ANÁLISIS Y DISEÑO DEL SISTEMA PROPUESTO

3.1. ANÁLISIS DE DATOS HISTÓRICOS

Este análisis responde al segundo objetivo específico: determinar, con los datos y no con supuestos, si el histórico permite construir los tres casos de uso del módulo predictivo, con qué variables y sobre cuántos casos. Se realizó sobre una copia completa de la base de datos de producción del portal anterior, exportada en septiembre de 2026, sin conexión con el sistema en operación. Las cifras de esta sección provienen del pipeline de datos descrito en la sección 3.4, que procesa esa copia de punta a punta. Las planillas Excel de control de proyectos que la empresa lleva fuera del sistema no estaban disponibles al cierre de este análisis, por lo que las cifras corresponden solo a la base de datos del portal y constituyen un resultado parcial: se recalculan cuando las planillas se incorporen.

3.1.1. *EXPLORACIÓN Y CARACTERIZACIÓN DE LAS FUENTES DE DATOS*

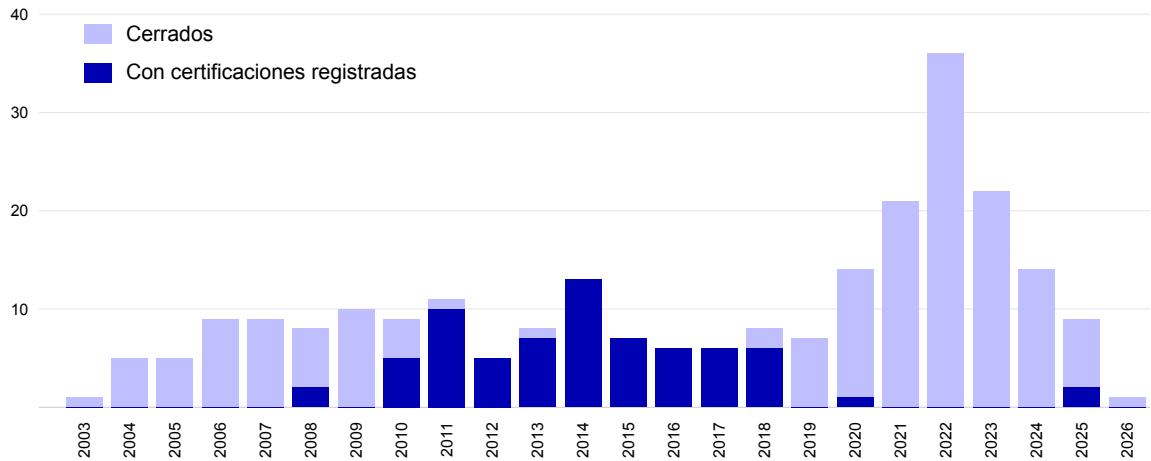
La base del portal contiene 175 tablas, en su mayoría catálogos, permisos y tablas de recursos humanos; estas últimas, con remuneraciones y datos personales, se excluyeron del análisis desde el inicio (RNF-03). La tabla 3.1 resume las tablas que describen la ejecución de los proyectos. No existe una tabla de certificaciones como tal: las certificaciones son ítems del flujo de caja de cada proyecto, identificados por su rubro, con fecha planificada y fecha real.

Tabla 3.1. Tablas del portal anterior usadas en el análisis

Tabla	Contenido	Filas	Período
act_proyecto	Maestro de centros de costo: fechas, monto de la orden de compra, cliente y estado	292	2003–2026
gbasebk	Copias de respaldo del presupuesto base, en 56 fechas	33 766	2011–2024
gpre	Flujo de caja vigente por ítem; las certificaciones son los rubros WI001 (orden de compra) y WA001 (adicionales)	10 872	2008–2027
gprepublicadomensual	Foto mensual del flujo de caja publicado	170 225	2013–2020
gpreeventos	Registro de cambios al flujo de caja	48 175	2013–2020
gpdir	Asientos contables por centro de costo	81 223	2008–2025
rubro	Catálogo de rubros: códigos de costo y de ingreso	58	—
horas, horasctroctos	Horas diarias por persona y su asignación a centros de costo	310 137 y 10 415	2005–2024
rq	Requisiciones de compra	6 242	2006–2026

Fuente: Elaboración propia, 2026, en base a la base de datos del portal anterior.

El universo de análisis son los centros de costo cerrados de tipo proyecto (prefijos 0 y C) y servicio (prefijo S): 244 centros de costo, 171 de proyecto u operación, 57 de servicio y 16 de consultoría, con fechas de inicio entre 2003 y 2026. Los centros de administración, áreas internas y operaciones (prefijos A, P, I y V) tienen presupuesto y gasto, pero no contrato ni cierre, y se excluyen. La figura 3.1 muestra la distribución por año de inicio y, para cada año, cuántos centros tienen certificaciones con fecha real: el flujo de caja del portal se usó para registrar certificaciones casi solo entre 2010 y 2018.

Figura 3.1. Centros de costo cerrados por año de inicio

Fuente: Elaboración propia, 2026, en base a la base de datos del portal anterior.

El hallazgo que condiciona todo el diseño es que, en el portal, el presupuesto vigente no es el plan: se replanificaba durante la ejecución hasta coincidir con lo ocurrido. En los 79 proyectos del panel de la sección 3.1.3, medido contra el flujo de caja vigente, solo 8 superan en más de un 10 % su presupuesto y el percentil 90 del sobrecosto es 1,10; medido contra la primera copia del presupuesto base, los que lo superan son 20 y el percentil 90 sube a 1,65. El presupuesto vigente absorbe las desviaciones grandes y no sirve para medirlas, por lo que el planificado de costo y de plazo se toma del presupuesto base guardado en `gbasebk`, que cubre 96 proyectos, y no de `gpre`. Esa línea base tampoco es el plan original: en la mediana, su primera copia es 25 meses posterior al inicio del proyecto, y solo 23 de los 96 proyectos la tienen dentro de los tres primeros meses. La tabla de cambios `gpreeventos`, que sí registra fechas desde el inicio, no puede reemplazarla, porque la creación del 88 % de las líneas no quedó registrada. Para la mayoría de los proyectos, el plan original no es recuperable.

La calidad de los datos se evaluó con las dimensiones de la sección 2.3, respecto de su uso en el módulo predictivo (cuadro 3.1). No se registraron causas de desviación: las columnas previstas para la causa y la mitigación están vacías, lo que descarta la hipótesis planteada en la sección 1.4 de que las causas documentadas pudieran usarse como parte del conjunto de datos.

Cuadro 3.1. Calidad del histórico del portal anterior por dimensión

Dimensión	Hallazgo	Efecto en el módulo
Exactitud	Presupuesto vigente replanificado hasta igualar lo ejecutado; línea base tomada con meses de retraso; fecha de cierre administrativa, muy posterior al fin operativo	Origen del planificado y de las etiquetas
Complejidad	Sin causas de desviación; certificaciones solo en 2010–2018; el 45 % de los asientos de proyecto no trae rubro; las horas no traen centro de costo y solo el 27 % puede asignarse a uno	Tamaño del conjunto y variables disponibles
Consistencia	Asientos de ingreso registrados como costo y gastos de la empresa (sueldos administrativos, intereses, impuestos a las transacciones, multas) cargados a proyectos	Clasificación del ejecutado
Unicidad	Códigos de centro de costo reutilizados por proyectos distintos	Depuración en Silver
Validez	Plazo de ejecución registrado con mediana de 90 días frente a 302 de duración real; tipo de cambio igual a uno, con moneda ambigua, en 2 962 asientos de 2008 a 2018; texto con codificación dañada	Variables excluidas; tipo de cambio pendiente de tratamiento
Oportunidad	No evaluable: el portal no registra cuándo se cargó cada dato respecto del evento	—

Fuente: Elaboración propia, 2026, en base a la base de datos del portal anterior y de la sección 2.3.

Los asientos sin rubro no eran datos faltantes: sus 20 descripciones y 29 cuentas contables permiten clasificarlos. Con una tabla de correspondencia construida sobre esas claves, el ejecutado queda clasificado en su totalidad, y la clasificación saca del costo de los proyectos los ingresos mal registrados y el gasto que no es de proyecto, y le agrega costo de mano de obra que faltaba, como aguinaldos, vacaciones y aportes.

3.1.2. IDENTIFICACIÓN DE VARIABLES PREDICTORAS

Una variable candidata debe cumplir dos condiciones: poder reconstruirse en el histórico usando solo registros con fecha anterior al momento de la predicción y tener equivalente en el ERP nuevo, donde se realiza la inferencia. La segunda condición recorta el costo a las cuatro categorías que maneja el ERP nuevo (servicios, materiales, logística y subcontratos), frente a las trece del portal; las demás no se usan, porque un modelo entrenado con ellas no podría evaluar los proyectos activos. La tabla 3.2 resume la evaluación.

Tabla 3.2. Variables predictoras candidatas

Variable	Equivalente en el ERP nuevo	Decisión
Costo base, duración planificada y composición por categoría	Líneas de presupuesto de línea base	Se usa
Consumo acumulado sobre el costo base, dividido por el avance temporal	Ejecutado del presupuesto	Se usa como regla de comparación
Categorías cuyo ejecutado superó su presupuesto base	Ejecutado por categoría	Se usa
Ritmo de carga de horas: horas cargadas sobre planificadas, dividido por el avance temporal	Horas aprobadas	Se usa
Republicaciones del flujo de caja hasta la fecha	Bitácora del presupuesto	Se usa
Certificaciones vencidas sin certificar	Certificaciones	Se usa
Requisiciones comprometidas sobre el presupuesto de compras	Órdenes de compra	Probada en una versión anterior del panel; sin relación con el desvío final
Viáticos y propuestas comerciales	Rendiciones y propuestas	Sin explorar
Proporción de asientos sin rubro	No existe	Se descarta: sin equivalente
Plazo de ejecución registrado	Fechas de vigencia	Se descarta: valores no comparables
Cliente	Cliente	Sin explorar
Sector industrial	Mercado de la propuesta	Se descarta: registrado solo desde 2020
Avance físico	No existe	Se descarta

Fuente: Elaboración propia, 2026, en base a la base de datos del portal anterior y del esquema del ERP nuevo.

Ninguna variable mide el avance físico, que el portal no registraba y el ERP nuevo tampoco define (sección 3.2); el avance se aproxima con la fracción transcurrida del plazo planificado. La regla con la que un jefe de proyecto anticipa el costo final, extrapolar el consumo acumulado según el avance, es la referencia que cualquier modelo debe superar. La tabla 3.3 compara esa regla con dos variables de ejecución, por banda de avance, mediante la correlación de rangos con el desenlace final del proyecto.

Tabla 3.3. Correlación de rangos de tres señales con el desenlace, por banda de avance

Desenlace	Señal	0–25 %	25–50 %	50–75 %	75 %+
Sobrecosto	Regla del consumo	–0,13	–0,11	0,00	0,28
	Ritmo de horas	–0,26	–0,18	–0,14	0,26
	Republicaciones	0,09	0,12	–0,09	–0,05
Retraso de cierre	Regla del consumo	–0,36	–0,51	–0,22	–0,03
	Ritmo de horas	–0,39	–0,30	–0,03	0,20
	Republicaciones	–0,26	–0,32	–0,40	–0,15

Fuente: Elaboración propia, 2026, en base al panel proyecto-mes de la sección 3.1.3 (1 287 filas de 79 proyectos; para el retraso, las filas de los 63 proyectos con esa etiqueta).

La regla del consumo no anticipa el sobrecosto antes del 75 % de avance: su correlación es nula o de signo contrario, y solo se vuelve positiva cuando la mayor parte del costo ya se ejecutó y la predicción no cambia ninguna decisión. Para el sobrecosto, el ritmo de carga de horas es la señal que aparece antes, con signo negativo: un proyecto que no carga horas al ritmo de su plan tiende a terminar con más sobrecosto. Para el retraso de cierre, en cambio, la regla del consumo y el ritmo de horas se asocian con fuerza parecida en la primera mitad, y las republicaciones del flujo de caja se asocian en todas las bandas con signo negativo: los proyectos cuyo plan se republicó más veces cerraron con menos retraso respecto de su línea base. Estas correlaciones son descriptivas: las filas de un mismo proyecto no son independientes, por lo que la capacidad predictiva de cada variable se evalúa con validación agrupada por proyecto (sección 3.4) y sus resultados se presentan en el Capítulo IV.

3.1.3. DEFINICIÓN DE CASOS DE USO PREDICTIVOS

La unidad de observación es el proyecto en un mes de su ejecución. Cada fila describe un proyecto al cierre de un mes, con variables calculadas solo con registros de fecha anterior o igual a ese cierre, y la etiqueta es el desenlace que se quiere anticipar. Así el mismo proyecto se observa en distintos momentos de su avance, que es como el módulo se usará: en la revisión mensual, para decidir sobre qué proyectos intervenir. La predicción solo es útil antes del 50 % de avance, cuando todavía se puede reasignar recursos, renegociar alcance o pedir un control de cambios.

El panel reúne 1 287 filas de 79 proyectos cerrados, iniciados entre 2008 y 2020, con una mediana de 12 meses observados por proyecto y una duración planificada mediana de 11 meses. Entran los proyectos con línea base y ejecutado clasificado; se excluyen los que reutilizan un código de centro de costo y aquellos cuyo presupuesto base, una vez retirada la contingencia, cae por debajo del mínimo para calcular un cociente. La contingencia se retira del presupuesto base porque el ERP nuevo no la maneja: incluirla entrenaría contra un objetivo que no puede reproducirse en la inferencia. Los tres casos de uso se definen con reglas de etiquetado explícitas, aplicables por igual al histórico y al ERP nuevo (tabla 3.4).

3.1.3.1 *Desviación en certificaciones*

Una fila es positiva si alguna certificación planificada para los tres meses siguientes se certifica con más de 15 días de atraso. De las 1 193 certificaciones con fecha real de los proyectos del panel, la mediana de atraso es cero y el 14 % supera los 15 días; con un umbral de cero días la etiqueta registra ruido de carga, y con uno de 30 quedan muy pocos casos para entrenar. Las fechas planificadas de las certificaciones se toman del flujo de caja vigente, porque la línea base no registra todas; en el análisis exploratorio, contrastadas con la primera foto mensual, las proporciones de atraso casi no cambiaban, lo que indica que en las certificaciones la replanificación pesa menos que en el costo. El horizonte de tres meses es un valor de diseño sin fuente en la empresa: cubre las certificaciones de más de un ciclo de revisión y se contrasta con los usuarios clave en la evaluación (sección 4.6). A diferencia de los otros dos casos, la etiqueta varía de un mes a otro dentro del mismo proyecto.

3.1.3.2 *Sobrecosto*

El sobrecosto es el cociente entre el costo ejecutado al cierre y el costo del presupuesto base, ambos en dólares y sobre las mismas cuatro categorías. Se modela como un valor continuo y la alerta se aplica después, sobre la predicción, con el umbral de 1,10, que coincide con el nivel rojo del semáforo de desviaciones del ERP (REQ 12.11). La mediana del cociente es 1,00, y 20 de los 79 proyectos (25 %) superan el umbral.

3.1.3.3 Retraso en la fecha de cierre

Como la fecha de cierre registrada es administrativa, el fin real se toma de la última certificación real, y el fin planificado, de la última certificación del presupuesto base. El retraso se expresa en días y la alerta se aplica sobre la predicción con un umbral de 30 días, que equivale al ciclo de la revisión mensual en la que se usa el módulo: un retraso menor a un mes no alcanza a cambiar ninguna decisión de esa revisión. De los 79 proyectos, 63 tienen certificaciones con fecha real para calcularlo: la mediana es de 92 días de anticipación y 14 proyectos (22 %) cierran con más de 30 días de retraso.

Tabla 3.4. Casos de uso predictivos y tamaño de sus conjuntos de datos

Caso de uso	Etiqueta	Casos	Casos desfavorables
Desviación en certificaciones	Alguna certificación de los próximos tres meses con más de 15 días de atraso	1 287 filas de 79 proyectos	126 filas de 28 proyectos
Sobrecosto	Costo ejecutado sobre presupuesto base, alerta sobre 1,10	79 proyectos	20 (25 %)
Retraso en la fecha de cierre	Días entre el fin real y el planificado, alerta sobre 30 días	63 proyectos	14 (22 %)

Fuente: Elaboración propia, 2026, en base a la base de datos del portal anterior.

Aunque el panel tiene más de mil filas, el número de desenlaces independientes es el de proyectos: 79 para el sobrecosto, con 20 positivos, y 63 para el retraso, con 14. Ese tamaño es la principal limitación del módulo predictivo y fija tres decisiones del diseño (sección 3.4): limitar el número de variables de cada modelo, preferir modelos simples y validar agrupando por proyecto, de modo que ninguna evaluación use un proyecto que el modelo ya vio. Queda abierta, además, la heterogeneidad de la línea base, tomada en momentos distintos de cada proyecto. Las planillas de control son la vía para ampliar el conjunto con proyectos posteriores a 2020 y con líneas base más tempranas; hasta que se incorporen, las conclusiones sobre el alcance del módulo son provisionales.

3.2. ANÁLISIS DE PROCESOS DEL PORTAL ANTERIOR

Este análisis responde al primer objetivo específico: identificar qué procesos del portal anterior faltan en el ERP nuevo y qué defectos afectan la calidad de los datos. Parte de dos fuentes. La primera es el manual de usuario del Portal ISIMustang Bolivia (ISI Mustang Bolivia, 2020), publicado por la empresa en 2020 como un blog en <https://manualportalbolivia.blogspot.com/>, con una entrada por módulo, que describe cada proceso tal como lo operan los usuarios. La segunda es el código fuente del ERP nuevo, revisado en septiembre de 2026, contra el que se verificó el estado de cada proceso. La documentación técnica del ERP no se usó para establecer ese estado porque, en varios puntos, describe brechas que el código ya había resuelto. Las decisiones de la empresa sobre cómo debe funcionar cada proceso se toman del relevamiento de requerimientos que la empresa consolidó en agosto de 2026 (ISI Mustang, 2026), citado en adelante como REQ seguido del apartado.

3.2.1. RELEVAMIENTO DE PROCESOS DEL PORTAL ANTERIOR

El manual organiza el portal en módulos de horas, requisiciones y compras, viáticos, proyectos, áreas de estructura y permisos. Los cuadros 3.2 y 3.3 listan sus procesos y su estado en el ERP nuevo, con cuatro valores: *implementado*, cuando el ERP nuevo cubre el proceso; *parcial*, cuando lo cubre con una diferencia que afecta sus datos; *reemplazado*, cuando la empresa decidió resolverlo de otro modo; y *fuera de alcance*.

Cuadro 3.2. Procesos operativos del portal anterior y su estado en el ERP nuevo

Proceso del portal	Manual	Estado	En el ERP nuevo
Carga de horas por centro de costo y tarea	01	Implementado	Borrador, envío y aprobación; horas de oficina y de campo, equivalentes a las HHC y PEMC del portal
Consulta de horas por persona o proyecto	04, 05.4, 06.3	Implementado	Listados filtrables por centro de costo y por persona
Pedido de requisición con aprobación	02	Implementado	Requisición contra centro de costo y bolsa, con motor de aprobaciones
Administración de requisiciones y emisión de la orden de compra	02.1	Parcial	Órdenes de compra con proveedor y recepción parcial; no registran la factura del proveedor, por lo que el ejecutado de compras se toma de la orden emitida
Pedido y administración de viáticos	03, 03.2	Implementado	Solicitud, aprobación del responsable y pago del anticipo por Compras
Política de viáticos: topes de gasto y plazo de rendición	03.1	Reemplazado	Sin topes ni plazos formales (REQ 11.3)
Rendición de viáticos	03.3	Implementado	Rendición en el sistema, con líneas en distintas monedas convertidas por separado; en el portal era una planilla externa

Fuente: Elaboración propia, 2026, en base a ISI Mustang Bolivia (2020) y al código del ERP nuevo.

Cuadro 3.3. Procesos de gestión de proyectos y estructura del portal anterior y su estado en el ERP nuevo

Proceso del portal	Manual	Estado	En el ERP nuevo
Asignación de personas y permisos en el proyecto	05.1	Implementado	Miembros del centro de costo con rol de responsable o de miembro
Seguimiento y control mensual: planificado frente a ejecutado	05, 05.2	Parcial	Flujo de caja con vendido, seguimiento y ejecutado por bolsa y mes; hereda la diferencia de compras señalada arriba
Indicadores de margen y desvíos de costo e ingreso	05, 05.2	Implementado	Margen bruto y desvíos por bolsa con semáforo de 5 y 10 %
Avance del proyecto y avance por actividad con escalas de valor ganado	05, 05.5	Reemplazado	Sin fórmula de avance por decisión de la empresa (REQ 6.6); las tareas registran solo horas planificadas
Publicación mensual del flujo de caja	05.2	Reemplazado	Bitácora de cambios de cada línea (REQ 12.1); falta consultar el presupuesto tal como estaba a una fecha (REQ 12.7)
Seguimiento de requisiciones y viáticos del proyecto	05.3	Implementado	Listados filtrables por centro de costo
Facturación de la orden de compra y de adicionales	05	Parcial	Certificaciones con registro y cobro; no exigen causa ni mitigación cuando se desvían (REQ 13.9)
Seguimiento y control de áreas de estructura	06, 06.2, 06.3	Implementado	Centros de costo de tipo estructura, con el mismo presupuesto
Rubros e ítems de gasto	06.1, 12	Reemplazado	Cuatro bolsas fijas: servicios, materiales, logística y subcontratos (REQ 12.2)
Permisos por módulo	11	Implementado	Roles globales y roles por centro de costo (REQ 2.1)
Consulta de proyectos históricos	11	Fuera de alcance	Los proyectos cerrados no se migran al ERP nuevo (sección 1.6); los activos se migran con su historia (RF-01)

Fuente: Elaboración propia, 2026, en base a ISI Mustang Bolivia (2020) y al código del ERP nuevo.

El relevamiento muestra que el ERP nuevo cubre la operación diaria del portal: horas, requisiciones, órdenes de compra, viáticos, rendiciones, presupuesto por centro de costo y permisos. Los procesos reemplazados responden a decisiones explícitas de la empresa y no a omisiones: la publicación mensual del flujo de caja se sustituyó por una bitácora, los rubros contables por cuatro bolsas fijas, y los topes de viáticos y la fórmula de avance se eliminaron o se difirieron. Los tres procesos parciales comparten un rasgo: el ERP nuevo registra la operación, pero no el dato que cierra el ciclo, ya sea la factura del proveedor, la causa del desvío o el estado

del plan en una fecha pasada.

3.2.2. PROCESOS A REIMPLEMENTAR EN EL ERP

Para decidir qué se reimplementa se aplicaron dos criterios, en este orden. El primero es que el proceso produzca datos que el módulo predictivo necesita como variable o como resultado a predecir; el segundo, que la empresa lo haya confirmado como requerimiento vigente. Un proceso reemplazado por decisión de la empresa no se reimplementa, aunque existiera en el portal. El cuadro 3.4 presenta el resultado.

Cuadro 3.4. Procesos a reimplementar en el ERP nuevo

Proceso	Dato que aporta al módulo predictivo	Requerimiento
Registro de la factura del proveedor y ejecutado de compras por factura	Costo real de materiales, logística y subcontratos, comparable con el costo contable del histórico; base del sobre costo	RF-02, RF-03; REQ 10.6, 10.7
Causa y mitigación en certificaciones desviadas	Registro de causas, ausente en el histórico, para ampliar el análisis en futuros reentrenamientos	RF-04; REQ 13.9
Consulta del presupuesto a una fecha	Seguimiento vigente en cada fecha de corte, sin usar información posterior; equivale a las fotos mensuales del portal	RF-05; REQ 12.7

Fuente: Elaboración propia, 2026.

Quedan sin reimplementar el avance por actividad con escalas de valor ganado, porque la empresa decidió no definir todavía una fórmula de avance, y la consulta de proyectos históricos, porque el histórico permanece en el portal anterior. En consecuencia, el módulo predictivo no usa variables de avance físico: el avance se aproxima con el tiempo transcurrido, el costo ejecutado y lo certificado frente a lo planificado, como se describe en la sección 3.4.

3.2.3. DEFECTOS Y BRECHAS QUE AFECTAN LA CALIDAD DE LOS DATOS

Los defectos que interesan a este trabajo son los que alteran una variable del módulo predictivo

o su resultado a predecir. Se presentan en dos grupos, según el sistema en que se originan (cuadro 3.5), y se clasifican con las dimensiones de calidad de la sección 2.3.

En el portal anterior los defectos provienen de cómo se operaron sus procesos durante más de diez años: el presupuesto se replanificaba hasta igualar lo ejecutado, los proyectos se cerraban administrativamente en fechas masivas y la moneda de los asientos contables no se registró de forma uniforme en todos los períodos. Estos defectos ya no pueden corregirse en la fuente; se tratan en la capa Silver del pipeline, y su magnitud se mide en la sección 3.1. En el ERP nuevo, en cambio, los defectos pueden corregirse antes de que produzcan datos: las brechas de la sección anterior se resuelven con los requerimientos RF-02 a RF-05, y de los defectos que la empresa había relevado en el cálculo de costos, la conversión de monedas en las rendiciones ya está corregida en el código, mientras que la imputación de ausencias al costo y el subsidio sin moneda no aplican a la operación actual.

Cuadro 3.5. Defectos que afectan los datos del módulo predictivo

Defecto	Dimensión	Dato afectado	Tratamiento
<i>Portal anterior</i>			
El presupuesto vigente se replanifica hasta igualar lo ejecutado	Exactitud	Sobrecosto y retraso	Planificado tomado de la línea base guardada en gbasebk, nunca del presupuesto vigente (Silver)
Fecha de cierre administrativa, asignada en lotes	Exactitud	Retraso en la fecha de cierre	Fin real a partir de la última certificación (Silver)
Moneda de los asientos contables no uniforme entre periodos	Validez	Costo ejecutado	Conversión con el tipo de cambio de cada asiento y clasificación completa del ejecutado por rubro, cuenta o descripción (Silver); los asientos con tipo de cambio igual a uno quedan pendientes de tratamiento
Horas sin centro de costo asignable	Compleitud	Ritmo de carga de horas	Asignación mediante horasactroctos (sección 3.1)
Causas de desviación no registradas	Compleitud	Causa del desvío	Sin tratamiento posible; queda como limitación
<i>ERP nuevo</i>			
Ejecutado de compras tomado de la orden emitida, sin factura	Exactitud	Sobrecosto	RF-02 y RF-03
Certificación desviada sin causa ni mitigación	Compleitud	Causa del desvío	RF-04
Sin consulta del presupuesto a una fecha pasada	Oportunidad	Variables a fecha de corte	RF-05

Fuente: Elaboración propia, 2026.

3.3. REQUERIMIENTOS DEL SISTEMA

Los requerimientos del sistema se derivan de cuatro fuentes. La primera son los objetivos específicos del trabajo (sección 1.3.2), que se citan como OE-1 a OE-7. La segunda es el relevamiento de requerimientos del ERP que la empresa consolidó en agosto de 2026 como base para su especificación de requerimientos de usuario (ISI Mustang, 2026); distingue lo que el sistema hace hoy de lo que el negocio confirmó como comportamiento vigente, y se cita como REQ seguido del apartado. La tercera es el relevamiento del código actual del ERP, que documenta defectos y brechas, y la cuarta son los hallazgos del análisis de datos históricos (sección 3.1) y

del análisis de procesos del portal anterior (sección 3.2). No se incluyen requerimientos obtenidos por entrevistas: de acuerdo con la metodología (sección 1.5), las entrevistas a usuarios clave se aplican en la etapa de evaluación y validación, y lo que surja de ellas se incorpora en ese momento.

Cada requerimiento lleva un identificador estable (RF-*nn* para los funcionales y RNF-*nn* para los no funcionales), una prioridad y su origen. La prioridad es *esencial* cuando el requerimiento es condición para cumplir un objetivo específico, y *deseable* cuando mejora el sistema sin ser condición de ninguno. El identificador no depende del incremento en que se construye el requerimiento, de modo que un ajuste en la planificación de los incrementos no obliga a reenumerar.

3.3.1. REQUERIMIENTOS FUNCIONALES

Los requerimientos funcionales se agrupan en cinco bloques que corresponden a los cinco incrementos del desarrollo (sección 2.6): procesos y calidad de datos del ERP, autenticación centralizada, pipeline de datos y modelos, servicio de predicción y tablero, e integraciones con Power BI y MCP.

El primer bloque (cuadro 3.6) reúne lo que falta en el ERP para que sus datos sirvan al módulo predictivo. El ERP ya calcula el ejecutado del presupuesto a partir de las provisiones, las órdenes de compra y las rendiciones de viáticos, pero toma las compras en el momento en que se emite la orden, con el importe estimado de la requisición, porque todavía no registra la factura del proveedor. El relevamiento de requerimientos define, en cambio, que la orden emitida es costo comprometido y que el ejecutado corresponde a la factura recibida (REQ 10.7 y 12.3). Mientras esa diferencia no se cierre, el sobre costo que se calculara sobre los proyectos activos no sería comparable con el costo real del histórico. Del mismo modo, el análisis de datos mostró que en el portal anterior no quedaron registradas las causas de las desviaciones; exigir las en las certificaciones que superan el umbral (REQ 13.9) permite que el ERP nuevo acumule esa información desde su puesta en producción. Por último, los proyectos activos al momento del corte se migran con su historia (RF-01), para que el módulo los evalúe desde el primer mes con las mismas variables que el histórico.

Cuadro 3.6. Requerimientos funcionales: procesos y calidad de datos del ERP

ID	Requerimiento	Prioridad	Origen
RF-01	Migrar al ERP nuevo los proyectos activos al momento del corte, con su historia desde el inicio: línea base, ejecutado, horas y certificaciones	Esencial	OE-4, sección 3.4
RF-02	Registrar en la orden de compra los datos de la factura del proveedor: número, fecha, importe real, moneda, estado de pago y fecha de pago	Esencial	REQ 10.6
RF-03	Calcular el ejecutado de materiales, logística y subcontratos a partir de la factura recibida, y conservar la orden emitida como costo comprometido	Esencial	REQ 10.7, 12.3
RF-04	Exigir causa y mitigación en las certificaciones cuya desviación supere un umbral parametrizable (inicial: 10 %)	Esencial	REQ 13.9, sección 3.1
RF-05	Reconstruir el estado del presupuesto de un centro de costo al cierre de cualquier mes a partir de la bitácora	Esencial	REQ 12.7, OE-5

Fuente: Elaboración propia, 2026.

El segundo bloque (cuadro 3.7) reemplaza la autenticación propia del ERP por Keycloak, según lo fundamentado en la sección 2.8. El control de acceso del ERP se compone de roles globales y de roles por centro de costo (REQ 2.1); la migración cambia quién autentica, no quién puede ver o hacer qué, por lo que esos dos ejes se conservan.

Cuadro 3.7. Requerimientos funcionales: autenticación centralizada

ID	Requerimiento	Prioridad	Origen
RF-06	Autenticar a los usuarios del ERP mediante Keycloak con el flujo de código de autorización y PKCE, en reemplazo de la emisión de tokens propia del backend	Esencial	OE-4
RF-07	Asociar cada identidad de Keycloak con su usuario del ERP y conservar los roles globales y los roles por centro de costo	Esencial	OE-4, REQ 2.1
RF-08	Emitir desde Keycloak las credenciales con las que se autentican los servicios entre sí y los clientes de Power BI y del servidor MCP	Esencial	OE-4, OE-6
RF-09	Cerrar la sesión del usuario en todos los componentes y aplicar la expiración de sesión definida por Keycloak	Esencial	REQ 2.4
RF-10	Mantener la recuperación de contraseña por correo, ahora gestionada por Keycloak	Deseable	REQ 2.4

Fuente: Elaboración propia, 2026.

El tercer bloque (cuadro 3.8) corresponde al pipeline y a los modelos. La unidad de análisis responde a lo encontrado en la sección 3.1: los tres casos se predicen sobre un panel en el que cada fila es un proyecto al cierre de un mes, con las etiquetas definidas en esa sección. El requerimiento RF-13 evita que el modelo aprenda de información que no existía en el momento de la predicción.

Cuadro 3.8. Requerimientos funcionales: pipeline de datos y modelos

ID	Requerimiento	Prioridad	Origen
RF-11	Copiar a la capa Bronze, sin modificarlos, los datos del volcado del portal anterior (MySQL), del ERP nuevo (PostgreSQL) y de las planillas de control	Esencial	OE-5
RF-12	Unificar en la capa Silver los dos esquemas y las planillas en un modelo común de variables: centros de costo, presupuesto base, ejecutado por categoría, certificaciones, republicaciones del presupuesto y horas, en dólares	Esencial	OE-5, sección 3.1
RF-13	Construir en la capa Gold un panel proyecto-mes cuyas variables usen solo información registrada hasta el cierre de cada mes	Esencial	OE-5
RF-14	Entrenar un modelo que estime, para cada proyecto y mes, el riesgo de que alguna certificación de los tres meses siguientes se atrase más de 15 días	Esencial	OE-5
RF-15	Entrenar un modelo que estime, para cada proyecto y mes, el sobre costo final respecto del presupuesto base	Esencial	OE-5
RF-16	Entrenar un modelo que estime, para cada proyecto y mes, el retraso final en días respecto del fin planificado	Esencial	OE-5
RF-17	Evaluar cada modelo con validación cruzada agrupada por proyecto, por banda de avance y frente a la regla del consumo (sección 3.1), y registrar métricas, datos y parámetros de cada versión	Esencial	OE-5, OE-7
RF-18	Reentrenar los modelos incorporando los proyectos cerrados en el ERP nuevo	Deseable	Sección 1.6

Fuente: Elaboración propia, 2026.

El cuarto bloque (cuadro 3.9) lleva las predicciones al usuario. Las predicciones se usan en la revisión mensual para decidir qué proyectos revisar primero, por lo que el tablero las presenta como una lista ordenada por riesgo. Se muestran en el tablero del ERP y no en Power BI; el acceso a ellas sigue las mismas reglas que el presupuesto del proyecto, que ven sus miembros y Gerencia (REQ 12.6).

Cuadro 3.9. Requerimientos funcionales: servicio de predicción y tablero

ID	Requerimiento	Prioridad	Origen
RF-19	Exponer un servicio que devuelva, para cada proyecto activo, la puntuación de riesgo de los tres casos, su posición en el orden de riesgo, su nivel de semáforo y la versión del modelo	Esencial	OE-6
RF-20	Acompañar cada predicción con las variables en que se basa y su valor	Esencial	OE-6
RF-21	Consultar las predicciones desde el ERP respetando la pertenencia del usuario al centro de costo, con visibilidad total para Gerencia	Esencial	OE-6, REQ 12.6
RF-22	Mostrar en el tablero los proyectos a los que el usuario tiene acceso, ordenados por riesgo y con el semáforo de cada caso de uso	Esencial	OE-6
RF-23	Mostrar el detalle de un proyecto con sus predicciones, su explicación y su evolución en el tiempo	Esencial	OE-6
RF-24	Notificar al responsable del centro de costo, mediante el módulo de notificaciones del ERP, cuando una predicción pasa a nivel rojo	Deseable	REQ 4.3

Fuente: Elaboración propia, 2026.

El quinto bloque (cuadro 3.10) define las integraciones. Los reportes y las herramientas que se listan son la propuesta inicial; su diseño detallado se desarrolla en la sección 3.4.

Cuadro 3.10. Requerimientos funcionales: integraciones con Power BI y MCP

ID	Requerimiento	Prioridad	Origen
RF-25	Exponer desde el ERP una API de datos descriptivos, sin predicciones, para cartera de proyectos, presupuesto contra ejecutado, certificaciones y cobranza, y horas por centro de costo	Esencial	OE-6
RF-26	Publicar en Power BI cuatro reportes descriptivos alimentados por esa API: cartera de proyectos, presupuesto contra ejecutado, certificaciones y cobranza, y horas por centro de costo	Esencial	OE-6
RF-27	Exponer en el servidor MCP herramientas para consultar un centro de costo, comparar vendido y ejecutado por bolsa, listar las certificaciones de un proyecto, obtener la predicción de riesgo de un proyecto y listar los proyectos en nivel rojo	Esencial	OE-6
RF-28	Limitar las herramientas del servidor MCP a consultas, sin operaciones que modifiquen datos del ERP	Esencial	OE-6, sección 1.6

Fuente: Elaboración propia, 2026.

3.3.2. *REQUERIMIENTOS NO FUNCIONALES*

Los requerimientos no funcionales (cuadros 3.11 y 3.12) restringen cómo se construye y opera el sistema. Dos de ellos derivan de condiciones propias del trabajo. La confidencialidad responde a que el documento se deposita en la biblioteca de la universidad y a que el ERP maneja nómina y datos personales: el costo de servicios de un proyecto se usa agregado, sin exponer el sueldo de cada persona (REQ 9.2). El despliegue responde a que el objetivo es dejar el sistema completo en operación en el servidor de la empresa, no una demostración local. El tiempo de respuesta de RNF-06 es un valor de diseño inicial, que se contrasta en la evaluación (sección 4.6).

Cuadro 3.11. Requerimientos no funcionales: seguridad y datos

ID	Atributo	Requerimiento	Origen
RNF-01	Seguridad	NestJS, FastAPI y el servidor MCP validan la firma, la expiración, el emisor y la audiencia de cada token emitido por Keycloak antes de atender una petición	OE-4, sección 2.8
RNF-02	Seguridad	Cada cliente recibe solo los permisos que necesita: Power BI y el servidor MCP acceden en modo de solo lectura, y el servidor MCP no reenvía a otros servicios el token que recibe	OE-6, sección 2.9
RNF-03	Confidencialidad	Ninguna capa del pipeline, reporte de Power BI ni herramienta MCP expone remuneraciones individuales ni datos personales de empleados; el costo de servicios se usa agregado por centro de costo	REQ 9.2, sección 1.6
RNF-04	Integridad de las fuentes	El pipeline accede a las fuentes en modo de solo lectura: no escribe en el ERP ni migra el histórico al esquema nuevo	Sección 1.6
RNF-05	Consistencia	Los montos se comparan en moneda base, convertidos con el tipo de cambio de la fecha de cada operación	REQ 4.1, 12.8

Fuente: Elaboración propia, 2026.

Cuadro 3.12. Requerimientos no funcionales: operación del sistema

ID	Atributo	Requerimiento	Origen
RNF-06	Rendimiento	El servicio de predicción responde la consulta de un proyecto en menos de dos segundos	Valor de diseño
RNF-07	Despliegue	Todos los componentes (Keycloak, PostgreSQL, NestJS, Angular, FastAPI y servidor MCP) se despliegan en contenedores Docker en el servidor propio de la empresa	OE-6, sección 1.6
RNF-08	Trazabilidad	Cada predicción queda registrada con el proyecto, el mes de cálculo, la versión del modelo y el resultado	OE-7
RNF-09	Reproducibilidad	El pipeline puede ejecutarse de nuevo desde la capa Bronze y, con los mismos datos y parámetros, produce los mismos modelos	OE-5, OE-7
RNF-10	Mantenibilidad	Los umbrales del semáforo y de desviación son parámetros configurables, y un modelo nuevo se publica sin cambiar el contrato del servicio de predicción	REQ 12.11, sección 3.1

Fuente: Elaboración propia, 2026.

3.4. DISEÑO DE LA ARQUITECTURA

El diseño parte de los requerimientos de la sección 3.3 y de lo encontrado en el análisis de datos (sección 3.1) y de procesos (sección 3.2). Se presenta de lo general a lo particular: primero la arquitectura por capas y sus vistas de contexto y de contenedores, y después cada componente.

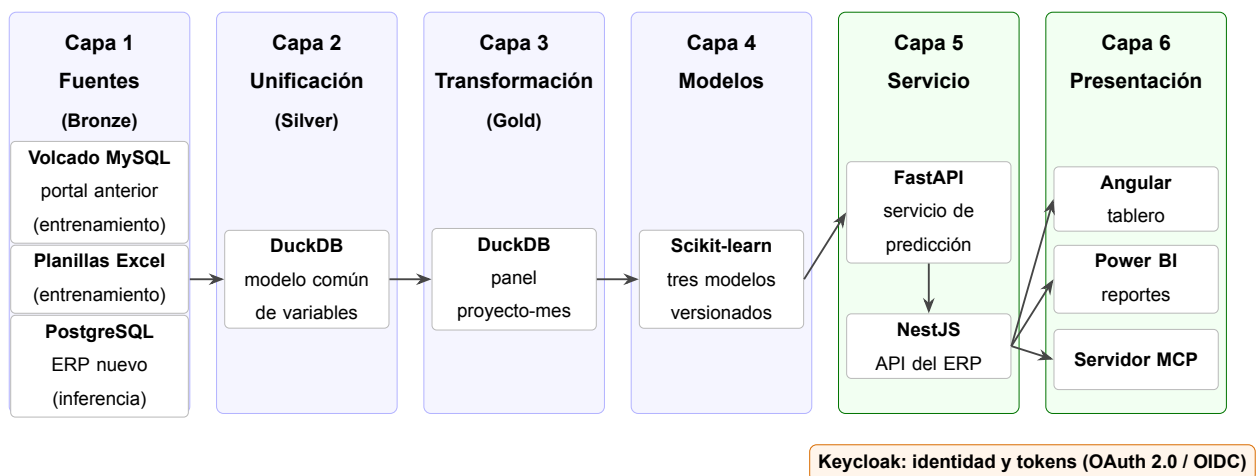
3.4.1. ARQUITECTURA POR CAPAS

La arquitectura se organiza en seis capas (figura 3.2). Las cuatro primeras forman el módulo predictivo y siguen la arquitectura *medallion* de la sección 2.3: fuentes (Bronze), unificación (Silver), transformación (Gold) y modelos. Las dos últimas integran el módulo con el ERP: la capa de

servicio, donde FastAPI expone las predicciones y NestJS los datos del ERP, y la capa de presentación, que comprende el tablero en Angular, los reportes de Power BI y el servidor MCP. Keycloak atraviesa las capas de servicio y de presentación, porque todos sus componentes validan los tokens que emite.

Respecto de la arquitectura propuesta en el perfil, además de las herramientas del pipeline y de los modelos, que se revisan en las secciones 3.4.2 y 3.4.3, el diseño introduce tres cambios de estructura. La capa de fuentes distingue el uso de cada una: el portal anterior y las planillas alimentan el entrenamiento, y el ERP nuevo, la inferencia. La capa de presentación incorpora Power BI y el servidor MCP. Y la autenticación pasa a ser un componente propio, compartido por todo el sistema.

Figura 3.2. Arquitectura por capas del sistema

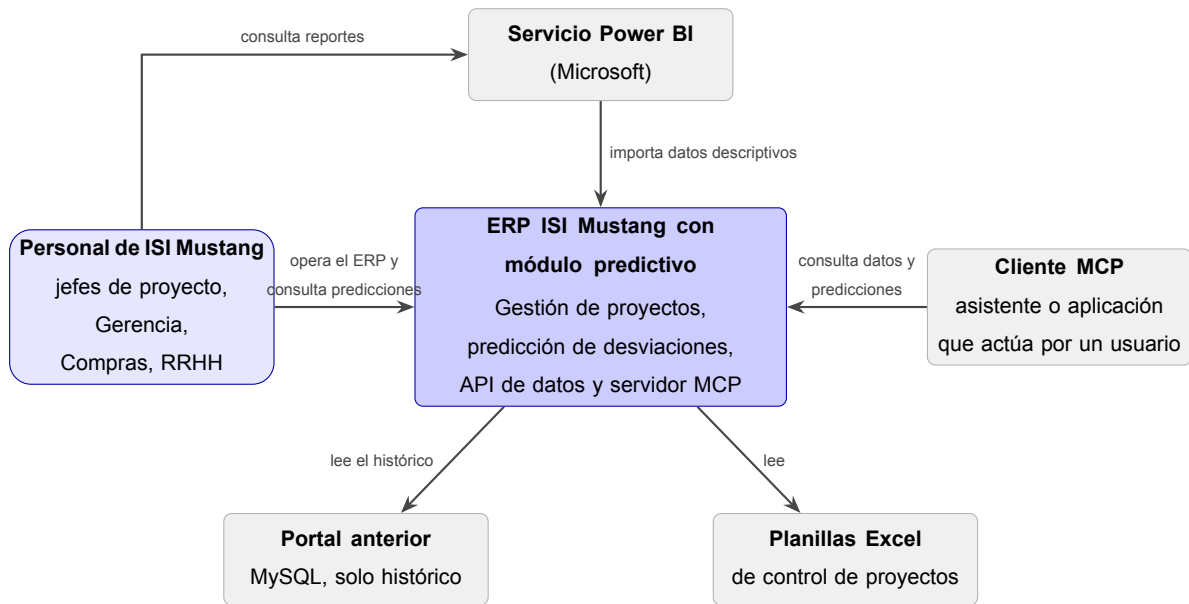


Fuente: Elaboración propia, 2026.

La vista de contexto del modelo C4 (figura 3.3) sitúa el sistema entre sus usuarios y los sistemas externos. El personal de ISI Mustang usa el ERP y su tablero desde el navegador y consulta los reportes en el servicio Power BI. Un cliente MCP, como un asistente de inteligencia artificial, consulta el sistema en nombre de un usuario. El portal anterior y las planillas de control solo se leen para construir el conjunto de entrenamiento: tras la puesta en producción del ERP nuevo no reciben datos nuevos. Los proyectos activos al momento del corte se migran al ERP nuevo con su historia (línea base, ejecutado, horas y certificaciones desde su inicio), de modo que la inferencia los evalúa igual que a los proyectos que nazcan en el ERP nuevo; los proyectos cerrados permanecen solo

en el portal anterior.

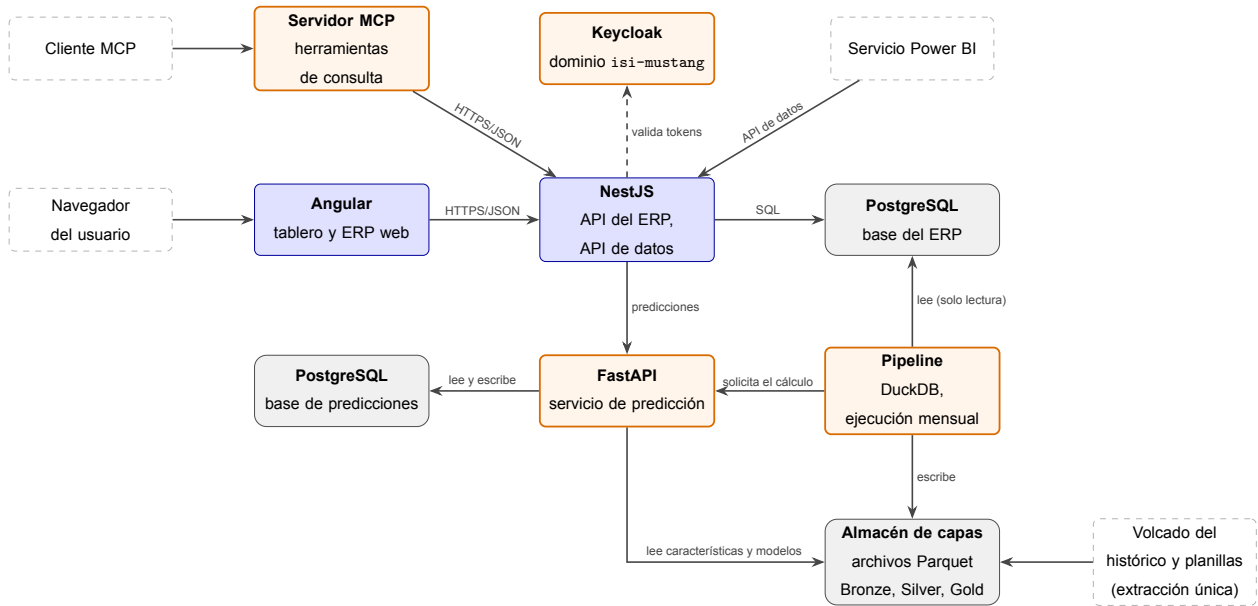
Figura 3.3. Diagrama de contexto (C4, nivel 1)



Fuente: Elaboración propia, 2026.

La vista de contenedores (figura 3.4) descompone el sistema en las unidades que se despliegan por separado. Todas corren como contenedores Docker en el servidor propio de la empresa (RNF-07), detrás de un proxy inverso que termina TLS, requisito de Keycloak y de los flujos OAuth fuera de un entorno local (Hardt, 2012; Keycloak Project, 2026b). Los contenedores nuevos son Keycloak, el servicio de predicción, el trabajo del pipeline, el almacén de capas y el servidor MCP; Angular, NestJS y PostgreSQL ya existen y se modifican. El portal anterior no se despliega: su histórico se extrae una sola vez a la capa Bronze.

Figura 3.4. Diagrama de contenedores (C4, nivel 2). En naranja, los contenedores nuevos; FastAPI y el servidor MCP también validan tokens de Keycloak



Fuente: Elaboración propia, 2026.

3.4.2. DISEÑO DEL PIPELINE DE DATOS

El pipeline es un proceso por lotes con dos modos de operación, como anticipa la sección 2.3, y sigue el patrón ELT: los datos se cargan sin transformar y se transforman dentro de DuckDB, un motor SQL embebido en el mismo proceso Python, sin servidor analítico que desplegar. La carga histórica se ejecuta una vez: procesa el volcado del portal anterior y las planillas, construye el conjunto de entrenamiento y produce los modelos. La carga de inferencia se ejecuta al cierre de cada mes, que es la unidad de los casos de uso (sección 3.1): extrae los centros de costo activos del ERP nuevo, calcula sus variables a esa fecha y solicita al servicio de predicción que las evalúe. En ningún modo el pipeline escribe en el ERP (RNF-04): se conecta a PostgreSQL con un usuario de base de datos de solo lectura.

3.4.2.1 Bronze

Cada fuente se copia a archivos Parquet sin transformar, uno por tabla y por fecha de

extracción. Como el portal anterior está congelado, su volcado es una copia única: el pipeline lo lee directamente y no necesita conectarse a un servidor MySQL. De él se extraen únicamente las tablas que usa el análisis (sección 3.1) y se excluyen las de nómina, contratos y datos personales (RNF-03); lo mismo se aplica a las tablas del ERP nuevo, donde el costo de servicios se toma ya agregado por centro de costo desde las provisiones. La bitácora del presupuesto del ERP nuevo (`cost_center_events`) se copia completa, porque es la única fuente de su historia.

3.4.2.2 *Silver*

DuckDB lee los archivos Bronze y los proyecta sobre un modelo común de variables, en dólares (RNF-05), con los dominios conformados: los estados de cada sistema se traducen a los mismos valores, los prefijos de centro de costo del portal anterior se traducen a los tipos del ERP nuevo y solo pasan a las capas siguientes los de tipo proyecto y servicio. El ejecutado del portal se clasifica con el catálogo de rubros y con la tabla de correspondencia de la sección 3.1, y las trece categorías de costo del portal se reducen a las cuatro del ERP nuevo. Los registros que no superan los controles no se descartan: pasan a una tabla de cuarentena, con el motivo. El cuadro 3.13 resume las correspondencias. Las variables que no tienen equivalente en ambos esquemas no se usan, porque un modelo entrenado con ellas no podría evaluar los proyectos activos. En el ERP nuevo, las líneas de lo vendido se editan sobre el mismo registro; si se tomara su valor vigente, se repetiría el problema de replanificación del portal (sección 3.1), por lo que la línea base se reconstruye con los eventos de creación de la bitácora.

Cuadro 3.13. Correspondencia de entidades en la capa Silver

Entidad común	Portal anterior (MySQL)	ERP nuevo (PostgreSQL)
Centro de costo	act_proyecto	cost_centers
Presupuesto base	Primera copia de gbasebk, sin contingencia	Líneas de lo vendido de cost_center_budget_lines, en su estado inicial reconstruido desde cost_center_events (RF-05)
Ejecutado	gpdire clasificado por rubro, cuenta o descripción	Ejecutado del presupuesto: provisiones, órdenes de compra con factura (RF-02) y rendiciones
Certificación planificada y real	gpre, rubros WI001 y WA001	certifications (planned_date, certification_date)
Republicaciones	Fotos mensuales de gprepublicadomensual	cost_center_events de las líneas de presupuesto
Horas	horas con horasctroctos	hour_entries aprobadas

Fuente: Elaboración propia, 2026, en base a los esquemas de ambos sistemas.

3.4.2.3 Gold

DuckDB construye, a partir de Silver, el panel proyecto-mes de la sección 3.1: una fila por proyecto y mes de ejecución, con variables calculadas solo con registros de fecha anterior o igual al cierre de ese mes (RF-13) y, en el entrenamiento, con las tres etiquetas. Las variables se agrupan en tres familias: el plan, fijo desde el inicio (costo base, duración planificada, composición por categoría y número de certificaciones); la ejecución acumulada hasta el mes (consumo sobre el costo base, categorías que superaron su presupuesto y certificaciones vencidas), y el ritmo y la estabilidad del proyecto (ritmo de carga de horas y republicaciones del presupuesto). El panel entrega sus resultados directamente como matrices numéricas a los modelos. Por eso no se usa una biblioteca de tablas adicional, como Polars o pandas: el mismo motor cubre Silver y Gold, y un segundo motor para la misma etapa sumaría una dependencia sin agregar ninguna capacidad.

Los archivos Gold de entrenamiento y los de cada ejecución mensual se conservan con la fecha de ejecución, de modo que cada predicción puede rastrearse hasta las variables que la produjeron (RNF-08) y el entrenamiento puede repetirse con los mismos datos (RNF-09).

3.4.3. DISEÑO DE MODELOS ML

El diseño de los modelos lo fija el tamaño del conjunto (sección 3.1): 79 desenlaces de sobrecosto, con 20 positivos, y 63 de retraso, con 14. Con esa cantidad de casos, un modelo con muchas variables o muchos parámetros encuentra patrones en el ruido, y las métricas de validación dejan de ser confiables. Por eso, además de las reglas de referencia, se prueban tres alternativas de complejidad creciente y se adopta la más simple que las supere; se descarta de antemano el *gradient boosting*, que necesitaría un orden de magnitud más de casos. Todas se implementan con Scikit-learn (cuadro 3.14):

1. Reglas de referencia: predecir siempre la mediana del histórico y la regla del consumo de la sección 3.1, que extrapola el consumo acumulado según el avance.
2. Una sola variable de la sección 3.1 usada como puntaje de riesgo.
3. Regresión lineal regularizada (*ridge*), con un máximo de ocho variables, una por cada diez casos, elegidas dentro de cada partición de la validación y nunca antes.
4. Árbol de decisión de profundidad máxima tres, que se lee como reglas de negocio.

El sobrecosto se modela sobre el logaritmo del cociente, para que los pocos proyectos extremos no dominen el ajuste, y el retraso, sobre los días; en ambos, la alerta se aplica después sobre la predicción. El atraso en certificaciones es una clasificación por fila. En los tres casos la salida que se usa es un puntaje con el que se ordenan los proyectos activos cada mes.

Cuadro 3.14. Modelos del módulo predictivo

Caso de uso	Variable objetivo	Métricas
Desviación en certificaciones	Alguna certificación de los próximos tres meses con más de 15 días de atraso	AUC por banda de avance; precisión y sensibilidad en el umbral operativo
Sobrecosto	Logaritmo del costo ejecutado sobre el presupuesto base; alerta sobre 1,10	MAE y AUC por banda de avance; precisión y sensibilidad en el umbral operativo
Retraso en la fecha de cierre	Días entre el fin real y el planificado; alerta sobre 30 días	MAE y AUC por banda de avance; precisión y sensibilidad en el umbral operativo

Fuente: Elaboración propia, 2026.

El protocolo de evaluación sigue la sección 2.2, con tres precisiones que impone la forma de los datos. La primera es que las filas de un mismo proyecto están correlacionadas; si quedaran repartidas entre entrenamiento y validación, el modelo se evaluaría sobre proyectos que ya vio. Por eso la validación principal es una validación cruzada agrupada por proyecto, de cinco particiones repetida diez veces, que usa todos los proyectos y da una estimación con su dispersión. La segunda es que las métricas se informan por banda de avance y nunca agregadas, porque los meses finales son fáciles de predecir y un promedio global ocultaría el desempeño en la ventana útil, antes del 50 % de avance; en cada banda el modelo se compara con las reglas de referencia. La tercera es que, además del error absoluto medio (MAE), que mide cuánto se acierta el valor, se informa el área bajo la curva ROC (AUC), que mide si el puntaje ordena primero a los proyectos que terminan mal, y la precisión y la sensibilidad al alertar una fracción fija de los proyectos de cada mes, fracción que se elige con esos resultados como umbral operativo. Como verificación secundaria se entrena con los proyectos iniciados hasta 2016 y se prueba con los posteriores. Un modelo que solo supera a las reglas después del 75 % de avance no se acepta, cualquiera sea su métrica global.

Cada entrenamiento produce una versión del modelo compuesta por el archivo del modelo y un registro con la fecha, el identificador de los archivos Gold usados, los parámetros, la semilla aleatoria y las métricas de validación (RF-17). El servicio de predicción carga la versión marcada como vigente; publicar una versión nueva no cambia el contrato de la API (RNF-10).

La explicación de cada predicción (RF-20) se deriva de la simplicidad de los modelos: en un modelo lineal, el aporte de cada variable es su coeficiente por su valor estandarizado; en el árbol, la explicación es la secuencia de reglas que lleva a la hoja, y en un puntaje de una sola variable, esa variable. El servicio devuelve las variables en que se basa la predicción, con su valor y el sentido en que la empujan.

3.4.4. DISEÑO DE LA API PREDICTIVA

El servicio de predicción es una aplicación FastAPI con su propia base de datos PostgreSQL, separada de la del ERP, donde guarda cada predicción emitida (RNF-08). Las predicciones no se calculan en cada consulta: al cierre de cada mes el pipeline solicita una ejecución de cálculo, el servicio evalúa todos los centros de costo activos con la versión vigente de cada modelo, los ordena por puntaje y guarda los resultados, y las consultas leen el último resultado. Así el tiempo de respuesta no depende del cálculo (RNF-06) y el tablero puede mostrar la evolución de cada proyecto (RF-23). El cuadro 3.15 lista los endpoints.

Cuadro 3.15. Endpoints del servicio de predicción

Endpoint	Respuesta	Cliente
POST /v1/scoring-runs	Inicia el cálculo sobre la última carga Gold; devuelve el identificador de la ejecución	Pipeline
GET /v1/predictions	Centros de costo activos ordenados por riesgo, filtrables por caso de uso y por nivel de semáforo	NestJS
GET /v1/cost-centers/{id}/predictions	Última predicción de los tres casos para un centro de costo	NestJS
GET /v1/cost-centers/{id}/predictions/history	Serie mensual de predicciones del centro de costo	NestJS
GET /v1/models	Versiones vigentes de los modelos y sus métricas	NestJS

Fuente: Elaboración propia, 2026.

Cada predicción se devuelve en JSON con el centro de costo, el mes, la versión del modelo y, por caso de uso, el puntaje, la posición en el orden de riesgo, el nivel de semáforo y las variables en

que se basa. Los esquemas de entrada y salida se declaran con Pydantic, de modo que FastAPI valida las peticiones y publica la especificación OpenAPI del servicio.

El servicio no conoce a los usuarios del ERP ni sus permisos. Solo acepta tokens de acceso de Keycloak cuya audiencia sea el propio servicio, emitidos por credenciales de cliente al backend del ERP o al pipeline (sección 2.8), y cada endpoint exige el rol correspondiente: cálculo para el pipeline y lectura para NestJS. El control de qué predicciones ve cada persona queda en NestJS, que aplica las reglas de acceso del ERP antes de llamar al servicio (RF-21). De ese modo las reglas de acceso viven en un solo lugar.

3.4.5. DISEÑO DEL DASHBOARD

Las predicciones se presentan dentro del ERP existente en Angular, en dos vistas nuevas (cuadro 3.16), y no en una aplicación aparte. NestJS agrega un módulo de predicciones que verifica el acceso del usuario al centro de costo, con las mismas reglas que el presupuesto (miembros del centro y Gerencia), llama al servicio de predicción y devuelve el resultado al frontend.

Cuadro 3.16. Vistas del tablero de predicciones

Vista	Contenido	Endpoint NestJS
Proyectos a revisar	Lista de los proyectos accesibles para el usuario ordenada por riesgo, con el semáforo de cada caso de uso; filtro por caso, por nivel y por tipo de centro de costo (RF-22)	GET /predictions
Pestaña <i>Riesgo</i> del centro de costo	Puntaje y semáforo de los tres casos, variables en que se basa cada uno y gráfico de su evolución mensual (RF-23)	GET /cost-centers/:id/predictions y su variante /history

Fuente: Elaboración propia, 2026.

La lista está pensada para la revisión mensual: responde qué proyectos mirar primero, no cuánto se desviará cada uno. La pestaña se agrega al detalle del centro de costo, junto a las de presupuesto, tareas y bitácora que ya existen, de modo que la predicción aparece al lado de los datos que la explican. El semáforo usa los mismos colores que el semáforo de desviaciones del presupuesto que ya muestra el ERP, y el nivel rojo corresponde a los proyectos que caen dentro

del umbral operativo. Cada vista indica el mes y la versión del modelo, para que el usuario sepa con qué información se calculó. Cuando un nivel pasa a rojo, el módulo de notificaciones del ERP avisa al responsable del centro de costo (RF-24).

3.4.6. DISEÑO DE LA AUTENTICACIÓN CENTRALIZADA

Se elige Keycloak frente a servicios de identidad en la nube, como Auth0 o Microsoft Entra ID, porque es de código abierto, sin costo por usuario, se despliega en el servidor de la empresa junto al resto del sistema (RNF-07) y cubre con estándares abiertos (OAuth 2.0 y OpenID Connect) todos los flujos que el sistema necesita (sección 2.8). Keycloak se despliega con un único dominio, *isi-mustang*, que contiene a los usuarios de la empresa. El cuadro 3.17 lista sus clientes; los dos primeros ya están configurados y los demás se agregan con los incrementos que los requieren. Las audiencias se asignan con mapeadores de audiencia (*audience mappers*) en cada cliente, de modo que cada token sirve solo para el servicio al que va dirigido (RNF-01).

Cuadro 3.17. Clientes registrados en Keycloak

Cliente	Tipo	Flujo	Uso
erp-web	Público	Código de autorización con PKCE	Inicio de sesión de las personas en Angular; token con audiencia <code>erp-api</code> , que valida NestJS
erp-backend	Confidencial	Credenciales de cliente	NestJS: administra las cuentas en Keycloak (alta, cambio de correo, desactivación y cierre de sesiones) e invoca al servicio de predicción con audiencia <code>prediction-api</code>
prediction-api	Servidor de recursos	—	Audiencia del servicio FastAPI
pipeline	Confidencial	Credenciales de cliente	Solicita las ejecuciones de cálculo al servicio de predicción
powerbi	Confidencial	Credenciales de cliente	Lectura de la API de datos descriptivos, con un único rol de lectura
mcp-server	Confidencial	Intercambio de tokens	Recibe tokens con audiencia del servidor MCP y los intercambia por un token del mismo usuario con audiencia <code>erp-api</code>
mcp-client	Público	Código de autorización con PKCE	Cliente MCP que actúa en nombre de un usuario

Fuente: Elaboración propia, 2026, en base a la sección 2.8.

Keycloak resuelve la autenticación; la autorización sigue en el ERP. Los roles globales y los roles por centro de costo (sección 3.3, RF-07) permanecen en la base del ERP, porque dependen de datos que solo el ERP conoce, como la pertenencia de cada persona a cada centro de costo. El token identifica a la persona y, en cada petición, NestJS resuelve su usuario y sus roles a partir del identificador de Keycloak (`sub`); por eso archivar a un usuario en el ERP le quita el acceso de inmediato. Keycloak guarda solo los roles técnicos de los clientes (lectura, cálculo), que no dependen del negocio. El ERP sigue siendo el lugar donde se administran las personas: al dar de alta un usuario, cambiar su correo o desactivarlo, NestJS replica el cambio en Keycloak mediante su API de administración y, al desactivarlo, cierra sus sesiones abiertas.

La migración desde la autenticación anterior, basada en Passport y en tokens firmados por

el propio backend, se hace en cuatro pasos, precedidos por un respaldo completo de la base. Primero, la tabla de usuarios del ERP incorpora el identificador de Keycloak y cada usuario existente se enlaza con su cuenta antes de que el backend nuevo entre en servicio, de modo que ningún usuario queda sin identidad. Segundo, las contraseñas no se migran: cada persona recibe una clave temporal o la redefine con la recuperación por correo que ofrece Keycloak (RF-10). Tercero, NestJS reemplaza la validación con secreto compartido por la validación con las claves públicas del dominio y comprueba firma, emisor, expiración y audiencia. Cuarto, se retiran los endpoints propios de inicio de sesión, renovación y recuperación de contraseña, con las tablas de credenciales y de tokens que los sostienen. El cierre de sesión y la expiración pasan a ser los de Keycloak (RF-09).

3.4.7. DISEÑO DE LAS INTEGRACIONES CON MCP Y POWER BI

3.4.7.1 Power BI

NestJS expone un módulo de datos descriptivos, separado de la API que usa el frontend, con cuatro endpoints de solo lectura, uno por reporte (cuadro 3.18). Devuelven datos agregados por centro de costo y mes, sin predicciones ni importes de nómina individuales (RNF-03). Entre los dos caminos de autenticación que plantea la sección 2.10 se elige la credencial de servicio: el cliente `powerbi` de Keycloak obtiene un token por credenciales de cliente y Power Query lo envía en el encabezado `Authorization`. Se descarta el conector personalizado con OAuth y PKCE porque autenticaría a cada persona sin necesidad: los reportes son de la empresa y los consultan jefes de proyecto y administración con los permisos del servicio Power BI. Los reportes usan el modo de importación con refresco programado; si la API del servidor no es alcanzable desde Internet, el refresco pasa por una puerta de enlace de datos local instalada en la red de la empresa.

Cuadro 3.18. Endpoints de la API de datos descriptivos

Endpoint	Datos
GET /reporting/portfolio	Centros de costo con tipo, estado, cliente, responsable, fechas y vendido total
GET /reporting/budget	Vendido, seguimiento, comprometido y ejecutado por centro de costo, bolsa y mes
GET /reporting/certifications	Certificaciones por centro de costo y mes: planificado, certificado y cobrado
GET /reporting/hours	Horas aprobadas por centro de costo, tipo de hora y mes, sin detalle por persona

Fuente: Elaboración propia, 2026.

3.4.7.2 Servidor MCP

Es un servicio independiente, escrito en TypeScript con el SDK oficial del protocolo, el mismo lenguaje del backend del ERP, con transporte Streamable HTTP; expone cinco herramientas de solo lectura (cuadro 3.19), con esquemas de entrada estrictos (sección 2.9). No accede a bases de datos: cada herramienta llama a la API de NestJS, de modo que las reglas de acceso del ERP se aplican igual que en el tablero (RF-28). Se despliega como servicio aparte del backend para aislar al ERP de un canal de acceso nuevo: una falla o una carga elevada originada en los clientes MCP no afecta al ERP, y el servicio puede habilitarse, restringirse o retirarse sin modificar ni volver a desplegar el backend, con el mismo criterio de separación en servicios de la sección 2.6. El costo de esa separación es el intercambio de tokens que se describe a continuación. El cliente MCP obtiene de Keycloak un token con la audiencia del servidor MCP; el servidor lo valida y, como no debe reenviarlo (sección 2.9), lo intercambia en Keycloak por un token del mismo usuario con audiencia `erp-api`, mediante el intercambio de tokens estándar que Keycloak implementa según el RFC 8693 (Keycloak Project, 2026a). Así NestJS recibe la identidad de la persona que consulta y le muestra solo lo que esa persona puede ver.

Cuadro 3.19. Herramientas del servidor MCP

Herramienta	Descripción
get_cost_center	Datos generales de un centro de costo: tipo, estado, cliente, responsable y fechas
compare_budget	Vendido, seguimiento y ejecutado por bolsa de un centro de costo, con su desviación
list_certifications	Certificaciones de un centro de costo con fechas y montos planificados y reales
get_risk_prediction	Última predicción de los tres casos de uso de un centro de costo, con su explicación
list_red_projects	Centros de costo accesibles para el usuario con algún caso en nivel rojo

Fuente: Elaboración propia, 2026.

CAPÍTULO IV

CONSTRUCCIÓN E IMPLEMENTACIÓN DEL SISTEMA

4.1. INCREMENTO 1: PROCESOS DEL PORTAL ANTERIOR MIGRADOS AL ERP

4.2. INCREMENTO 2: AUTENTICACIÓN CENTRALIZADA CON KEYCLOAK

4.3. INCREMENTO 3: PIPELINE DE DATOS Y ENTRENAMIENTO DE MODELOS

4.3.1. INTEGRACIÓN DUCKDB CON MYSQL Y POSTGRESQL

4.3.2. TRANSFORMACIÓN DE DATOS CON POLARS Y FEATURE ENGINEERING

4.3.3. MODELO DE DESVIACIÓN EN CERTIFICACIONES

4.3.4. MODELO DE SOBRECOSTO DE PROYECTOS

4.3.5. MODELO DE RETRASO EN LA FECHA DE CIERRE

4.4. INCREMENTO 4: MICROSERVICIO FASTAPI E INTEGRACIÓN CON EL ERP

4.4.1. DESARROLLO DEL MICROSERVICIO FASTAPI

4.4.2. ENDPOINTS DE PREDICCIÓN EN NESTJS

4.4.3. DASHBOARD DE INTELIGENCIA PREDICTIVA EN ANGULAR

4.5. INCREMENTO 5: INTEGRACIÓN CON POWER BI Y MCP

4.5.1. API DE DATOS PARA POWER BI

4.5.2. SERVIDOR MCP

4.6. EVALUACIÓN Y VALIDACIÓN

4.6.1. MÉTRICAS DE RENDIMIENTO DE MODELOS

4.6.2. PRUEBAS CON USUARIOS CLAVE

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

RECOMENDACIONES

REFERENCIAS BIBLIOGRÁFICAS

- Al Jafari, A., & Nair, S. S. K. (2018). ERP Implementation in the Oil and Gas Sector: A Case Study in Sultanate of Oman. *2018 7th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*, 848-854. <https://doi.org/10.1109/ICRITO.2018.8748779>
- Ali, M., Edghiem, F., & Alkhalifah, E. S. (2023). Cultural Challenges of ERP Implementation in Middle-Eastern Oil & Gas Sector: An Action Research Approach. *Systemic Practice and Action Research*, 36(1), 111-140. <https://doi.org/10.1007/s11213-022-09600-4>
- Anbari, F. T. (2003). Earned Value Project Management Method and Extensions. *Project Management Journal*, 34(4), 12-23. <https://doi.org/10.1177/875697280303400403>
- Anthropic. (2024, 25 de noviembre). *Introducing the Model Context Protocol*. Consultado el 17 de septiembre de 2026, desde <https://www.anthropic.com/news/model-context-protocol>
- Apache Software Foundation. (s.f.). *Apache Superset*. Consultado el 17 de septiembre de 2026, desde <https://superset.apache.org/>
- Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics. *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR '21)*. https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf
- Assaf, S. A., & Al-Hejji, S. (2006). Causes of Delay in Large Construction Projects. *International Journal of Project Management*, 24(4), 349-357. <https://doi.org/10.1016/j.ijproman.2005.11.010>
- Basili, V. R., & Turner, A. J. (1975). Iterative Enhancement: A Practical Technique for Software Development. *IEEE Transactions on Software Engineering*, SE-1(4), 390-396. <https://doi.org/10.1109/TSE.1975.6312870>
- Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4.^a ed.). Addison-Wesley Professional.
- Batini, C., Cappiello, C., Francalanci, C., & Maurino, A. (2009). Methodologies for Data Quality Assessment and Improvement. *ACM Computing Surveys*, 41(3), 16:1-16:52. <https://doi.org/10.1145/1541880.1541883>
- Batini, C., & Scannapieco, M. (2016). *Data and Information Quality. Dimensions, Principles and Techniques*. Springer. <https://doi.org/10.1007/978-3-319-24106-7>
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile Software Development*. Consultado el 17 de septiembre de 2026, desde <https://agilemanifesto.org/>
- Boehm, B. W. (1988). A Spiral Model of Software Development and Enhancement. *Computer*, 21(5), 61-72. <https://doi.org/10.1109/2.59>
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>
- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Chapman and Hall/CRC. <https://doi.org/10.1201/9781315139470>
- Campbell, B., Bradley, J., & Sakimura, N. (2020). *Resource Indicators for OAuth 2.0* (RFC N.º 8707). IETF. <https://doi.org/10.17487/RFC8707>

- Chai, T., & Draxler, R. R. (2014). Root Mean Square Error (RMSE) or Mean Absolute Error (MAE)? – Arguments against Avoiding RMSE in the Literature. *Geoscientific Model Development*, 7(3), 1247-1250. <https://doi.org/10.5194/gmd-7-1247-2014>
- Chaudhuri, S., & Dayal, U. (1997). An Overview of Data Warehousing and OLAP Technology. *ACM SIGMOD Record*, 26(1), 65-74. <https://doi.org/10.1145/248603.248616>
- Chen, H., Chiang, R. H. L., & Storey, V. C. (2012). Business Intelligence and Analytics: From Big Data to Big Impact. *MIS Quarterly*, 36(4), 1165-1188. <https://doi.org/10.2307/41703503>
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, 785-794. <https://doi.org/10.1145/2939672.2939785>
- Christie, T., et al. (s.f.). *Starlette*. Consultado el 17 de septiembre de 2026, desde <https://starlette.dev/>
- Chung, B. Y., Skibniewski, M. J., Lucas, Jr., Henry C., & Kwak, Y. H. (2008). Analyzing Enterprise Resource Planning System Implementation Success Factors in the Engineering–Construction Industry. *Journal of Computing in Civil Engineering*, 22(6), 373-382. [https://doi.org/10.1061/\(ASCE\)0887-3801\(2008\)22:6\(373\)](https://doi.org/10.1061/(ASCE)0887-3801(2008)22:6(373))
- Colvin, S., et al. (s.f.). *Pydantic Validation*. *Pydantic Docs*. Consultado el 17 de septiembre de 2026, desde <https://docs.pydantic.dev/latest/>
- DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2.^a ed.). Technics Publications.
- Databricks. (2025). *What Is the Medallion Lakehouse Architecture?* Consultado el 19 de junio de 2026, desde <https://docs.databricks.com/aws/en/lakehouse/medallion>
- Davenport, T. H. (1998). Putting the Enterprise into the Enterprise System. *Harvard Business Review*, 76(4), 121-131.
- Davenport, T. H., & Harris, J. G. (2007). *Competing on Analytics. The New Science of Winning*. Harvard Business School Press.
- Delen, D., & Demirkan, H. (2013). Data, Information and Analytics as Services. *Decision Support Systems*, 55(1), 359-363. <https://doi.org/10.1016/j.dss.2012.05.044>
- Django Software Foundation. (s.f.). *ASGI (Asynchronous Server Gateway Interface) Specification. ASGI 3.0 Documentation*. Consultado el 17 de septiembre de 2026, desde <https://asgi.readthedocs.io/en/latest/specs/main.html>
- Docker, Inc. (s.f.). *What Is Docker?*. *Docker Docs*. Consultado el 17 de septiembre de 2026, desde <https://docs.docker.com/get-started/docker-overview/>
- DuckDB Foundation. (s.f.-a). *Integration with Polars*. Consultado el 17 de septiembre de 2026, desde <https://duckdb.org/docs/current/guides/python/polars.html>
- DuckDB Foundation. (s.f.-b). *MySQL Extension*. Consultado el 17 de septiembre de 2026, desde https://duckdb.org/docs/current/core_extensions/mysql.html
- DuckDB Foundation. (s.f.-c). *PostgreSQL Extension*. Consultado el 17 de septiembre de 2026, desde https://duckdb.org/docs/current/core_extensions/postgres/overview.html

- DuckDB Foundation. (s.f.-d). *Why DuckDB*. Consultado el 17 de septiembre de 2026, desde https://duckdb.org/why_duckdb
- Eckerson, W. W. (2010). *Performance Dashboards. Measuring, Monitoring, and Managing Your Business* (2.^a ed.). John Wiley & Sons.
- Ehtesham, A., Singh, A., Gupta, G. K., & Kumar, S. (2025). *A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)* (2). arXiv: 2505.02279 [cs.AI]. Consultado el 17 de septiembre de 2026, desde <https://arxiv.org/abs/2505.02279>
- Elmousalami, H. H. (2020). Artificial Intelligence and Parametric Construction Cost Estimate Modeling: State-of-the-Art Review. *Journal of Construction Engineering and Management*, 146(03119008). [https://doi.org/10.1061/\(ASCE\)CO.1943-7862.0001678](https://doi.org/10.1061/(ASCE)CO.1943-7862.0001678)
- Fett, D., Küsters, R., & Schmitz, G. (2016). A Comprehensive Formal Security Analysis of OAuth 2.0. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*, 1204-1215. <https://doi.org/10.1145/2976749.2978385>
- Few, S. (2013). *Information Dashboard Design. Displaying Data for at-a-Glance Monitoring* (2.^a ed.). Analytics Press.
- FIDIC. (2017). *Conditions of Contract for EPC/Turnkey Projects. Silver Book* (2.^a ed.). Fédération Internationale des Ingénieurs-Conseils.
- Fleming, Q. W., & Koppelman, J. M. (2010). *Earned Value Project Management* (4.^a ed.). Project Management Institute.
- Flyvbjerg, B. (2014). What You Should Know about Megaprojects and Why: An Overview. *Project Management Journal*, 45(2), 6-19. <https://doi.org/10.1002/pmj.21409>
- Flyvbjerg, B., Holm, M. S., & Buhl, S. (2002). Underestimating Costs in Public Works Projects: Error or Lie? *Journal of the American Planning Association*, 68(3), 279-295. <https://doi.org/10.1080/01944360208976273>
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, 29(5), 1189-1232. <https://doi.org/10.1214/aos/1013203451>
- Galloway, P. (2009). Design-Build/EPC Contractor's Heightened Risk—Changes in a Changing World. *Journal of Legal Affairs and Dispute Resolution in Engineering and Construction*, 1(1), 7-15. [https://doi.org/10.1061/\(ASCE\)1943-4162\(2009\)1:1\(7\)](https://doi.org/10.1061/(ASCE)1943-4162(2009)1:1(7))
- Géron, A. (2022). *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow. Concepts, Tools, and Techniques to Build Intelligent Systems* (3.^a ed.). O'Reilly Media.
- Góes de Almeida, M., & Canedo, E. D. (2022). Authentication and Authorization in Microservices Architecture: A Systematic Literature Review. *Applied Sciences*, 12(6), 3023. <https://doi.org/10.3390/app12063023>
- Google LLC. (s.f.-a). *Server-Side and Hybrid-Rendering. Angular Documentation*. Consultado el 17 de septiembre de 2026, desde <https://angular.dev/guide/ssr>
- Google LLC. (s.f.-b). *What Is Angular?. Angular Documentation*. Consultado el 17 de septiembre de 2026, desde <https://angular.dev/overview>

- Gool, S., & Seymour, L. F. (2018). Managing Enterprise Resource Planning System Customisation Post-Implementation – The Case of an African Petroleum Organisation. *Proceedings of the 20th International Conference on Enterprise Information Systems (ICEIS 2018)*, 2, 111-119. <https://doi.org/10.5220/0006676401110119>
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*.
- Gudivada, V., Apon, A., & Ding, J. (2017). Data Quality Considerations for Big Data and Machine Learning: Going beyond Data Cleaning and Transformations. *International Journal on Advances in Software*, 10(1–2), 1-20.
- Guyon, I., & Elisseeff, A. (2003). An Introduction to Variable and Feature Selection. *Journal of Machine Learning Research*, 3, 1157-1182.
- Habibi, M., Kermanshachi, S., & Safapour, E. (2018). Engineering, Procurement, and Construction Cost and Schedule Performance Leading Indicators: State-of-the-Art Review. *Construction Research Congress 2018*, 378-388. <https://doi.org/10.1061/9780784481271.037>
- Hardt, D. (2012, octubre). *The OAuth 2.0 Authorization Framework* (D. Hardt, Ed.; RFC N.º 6749). Internet Engineering Task Force. <https://doi.org/10.17487/RFC6749>
- Hardt, D., Parecki, A., & Lodderstedt, T. (2026, septiembre). *The OAuth 2.1 Authorization Framework* (Internet-Draft N.º draft-ietf-oauth-v2-1-16). Internet Engineering Task Force, OAuth Working Group. Consultado el 17 de septiembre de 2026, desde <https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array Programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning. Data Mining, Inference, and Prediction* (2.ª ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
- Hou, X., Zhao, Y., Wang, S., & Wang, H. (2026). Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *ACM Transactions on Software Engineering and Methodology*, 35(10), 1-37. <https://doi.org/10.1145/3796519>
- İnan, T., Narbaev, T., & Hazir, Ö. (2022). A Machine Learning Study to Enhance Project Cost Forecasting. *IFAC-PapersOnLine*, 55(10), 3286-3291. <https://doi.org/10.1016/j.ifacol.2022.10.127>
- Inmon, W. H. (2005). *Building the Data Warehouse* (4.ª ed.). Wiley.
- ISI Mustang. (2026, 24 de agosto). *Base Funcional TO-BE Para La Especificación de Requerimientos de Usuario Del ERP ISI Mustang* (Documento interno). ISI Mustang. Santa Cruz de la Sierra.
- ISI Mustang Bolivia. (2020, abril). *Manual de Usuario Del Portal Isimustang Bolivia*. Consultado el 17 de septiembre de 2026, desde <https://manualportalbolivia.blogspot.com/>

- ISO/IEC/IEEE. (2017). *ISO/IEC/IEEE 12207:2017 – Systems and Software Engineering – Software Life Cycle Processes* (1.^a ed., Standard). International Organization for Standardization. Geneva. <https://doi.org/10.1109/IEEESTD.2017.8100771>
- Jacobs, F. R., & Weston, Jr., F. C. 'Ted'. (2007). Enterprise Resource Planning (ERP)—A Brief History. *Journal of Operations Management*, 25(2), 357-363. <https://doi.org/10.1016/j.jom.2006.11.005>
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning. with Applications in R* (2.^a ed.). Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- Jawad, Z. N., & Villányi, B. (2024). Machine Learning-Driven Optimization of Enterprise Resource Planning (ERP) Systems: A Comprehensive Review. *Beni-Suef University Journal of Basic and Applied Sciences*, 13(4). <https://doi.org/10.1186/s43088-023-00460-y>
- Jones, M. B., Bradley, J., & Sakimura, N. (2015, mayo). *JSON Web Token (JWT)* (RFC N.º 7519). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7519>
- Jones, M. B., & Hardt, D. (2012, octubre). *The OAuth 2.0 Authorization Framework: Bearer Token Usage* (RFC N.º 6750). Internet Engineering Task Force. <https://doi.org/10.17487/RFC6750>
- Jones, M. B., Hunt, P., & Parecki, A. (2025). *OAuth 2.0 Protected Resource Metadata* (RFC N.º 9728). IETF. <https://doi.org/10.17487/RFC9728>
- JSON-RPC Working Group. (2013). *JSON-RPC 2.0 Specification*. Consultado el 17 de septiembre de 2026, desde <https://www.jsonrpc.org/specification>
- Keycloak. (2026). *Integrating with Model Context Protocol (MCP)*. Consultado el 17 de septiembre de 2026, desde <https://www.keycloak.org/securing-apps/mcp-authz-server>
- Keycloak Project. (2026a). *Configuring and Using Token Exchange* (26.7.4). Consultado el 27 de septiembre de 2026, desde <https://www.keycloak.org/securing-apps/token-exchange>
- Keycloak Project. (2026b). *Configuring Keycloak for Production* (26.7.4). Consultado el 27 de septiembre de 2026, desde <https://www.keycloak.org/server/configuration-production>
- Keycloak Project. (2026c). *Securing Applications and Services with OpenID Connect* (26.7.4). Consultado el 17 de septiembre de 2026, desde <https://www.keycloak.org/securing-apps/oidc-layers>
- Keycloak Project. (2026d). *Server Administration Guide* (26.7.4). Consultado el 17 de septiembre de 2026, desde https://www.keycloak.org/docs/latest/server_admin/index.html
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit. The Definitive Guide to Dimensional Modeling* (3.^a ed.). John Wiley & Sons.
- Klaus, H., Rosemann, M., & Gable, G. G. (2000). What Is ERP? *Information Systems Frontiers*, 2(2), 141-162. <https://doi.org/10.1023/A:1026543906354>
- Kohavi, R. (1995). A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI'95)*, 2, 1137-1143.
- Kuhn, M., & Johnson, K. (2019). *Feature Engineering and Selection. A Practical Approach for Predictive Models*. Chapman and Hall/CRC. <https://doi.org/10.1201/9781315108230>

- Larasati, S. D., Eitiveni, I., & Mahardhika, P. (2023). Analysis of ERP Critical Failure Factors: A Case Study in an Indonesian Mining Company. *Jurnal Sistem Informasi*, 19(2), 34-47. <https://doi.org/10.21609/jsi.v19i2.1291>
- Larman, C., & Basili, V. R. (2003). Iterative and Incremental Development: A Brief History. *Computer*, 36(6), 47-56. <https://doi.org/10.1109/MC.2003.1204375>
- Lepenioti, K., Bousdekis, A., Apostolou, D., & Mentzas, G. (2020). Prescriptive Analytics: Literature Review and Research Challenges. *International Journal of Information Management*, 50, 57-70. <https://doi.org/10.1016/j.ijinfomgt.2019.04.003>
- Lewis, J., & Fowler, M. (2014, marzo). *Microservices. A Definition of This New Architectural Term*. Consultado el 17 de septiembre de 2026, desde <https://martinfowler.com/articles/microservices.html>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- Lipke, W., Zwikaël, O., Henderson, K., & Anbari, F. (2009). Prediction of Project Outcome: The Application of Statistical Methods to Earned Value Management and Earned Schedule Performance Indexes. *International Journal of Project Management*, 27(4), 400-407. <https://doi.org/10.1016/j.ijproman.2008.02.009>
- Lodderstedt, T., Bradley, J., Labunets, A., & Fett, D. (2025, enero). *Best Current Practice for OAuth 2.0 Security* (RFC N.º 9700). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9700>
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. En S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference (SciPy 2010)* (pp. 56-61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- Morrow, E. W. (2011). *Industrial Megaprojects. Concepts, Strategies, and Practices for Success*. John Wiley & Sons.
- Metabase. (s.f.). *Adding and Managing Databases*. Consultado el 17 de septiembre de 2026, desde <https://www.metabase.com/docs/latest/databases/connecting>
- Microsoft. (2023, 20 de octubre). *DAX Overview*. Consultado el 17 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/dax/dax-overview>
- Microsoft. (2025a, 18 de septiembre). *Data Refresh in Power BI*. Consultado el 17 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/power-bi/connect-data/refresh-data>
- Microsoft. (2025b, 25 de septiembre). *DirectQuery in Power BI: When to Use, Limitations, Alternatives*. Consultado el 17 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-directquery-about>
- Microsoft. (2025c, 7 de febrero). *On-Premises Data Gateway*. Consultado el 17 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/power-bi/connect-data/service-gateway-onprem>
- Microsoft. (2026a, 8 de abril). *Authentication in Desktop Apps*. Consultado el 17 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/power-query/connector-authentication>

- Microsoft. (2026b, 13 de julio). *Handling Authentication for Power Query Connectors*. Consultado el 17 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/power-query/handling-authentication>
- Microsoft. (2026c, 23 de marzo). *Power BI Service Basics: Key Concepts and Terms*. Consultado el 17 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/power-bi/fundamentals/service-basic-concepts>
- Microsoft. (2026d, 8 de abril). *Power Query Web Connector*. Consultado el 17 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/power-query/connectors/web/web>
- Microsoft. (2026e, 23 de marzo). *What Is Power BI?* Consultado el 17 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/power-bi/fundamentals/power-bi-overview>
- Microsoft. (2026f, 10 de agosto). *What Is Power Query?* Consultado el 17 de septiembre de 2026, desde <https://learn.microsoft.com/en-us/power-query/power-query-what-is-power-query>
- Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
- Model Context Protocol. (2025). *Model Context Protocol Specification (2025-06-18)*. Consultado el 17 de septiembre de 2026, desde <https://modelcontextprotocol.io/specification/2025-06-18>
- Model Context Protocol. (2026). *Model Context Protocol Specification (2026-07-28)*. Consultado el 17 de septiembre de 2026, desde <https://modelcontextprotocol.io/specification/2026-07-28>
- Mohammed, S., Budach, L., Feuerpfeil, M., Ihde, N., Nathansen, A., Noack, N., Patzlaff, H., Naumann, F., & Harmouch, H. (2025). The Effects of Data Quality on Machine Learning Performance on Tabular Data. *Information Systems*, 132, 102549. <https://doi.org/10.1016/j.is.2025.102549>
- Mohd Roshdi, F. R., Ismail, K., Lop, N. S., & Ab Wahab, L. (2022). Cost Overruns in Engineering Procurement Construction (EPC) Fabrication Oil and Gas Megaprojects in Malaysia: The Importance of Resource Allocation (5M). *International Journal of Academic Research in Business and Social Sciences*, 12(9). <https://doi.org/10.6007/ijarbss/v12-i9/14752>
- Myśliwiec, K., et al. (s.f.-a). *Documentation. NestJS - A Progressive Node.Js Framework*. Consultado el 17 de septiembre de 2026, desde <https://docs.nestjs.com/>
- Myśliwiec, K., et al. (s.f.-b). *Modules. NestJS Documentation*. Consultado el 17 de septiembre de 2026, desde <https://docs.nestjs.com/modules>
- Negash, S. (2004). Business Intelligence. *Communications of the Association for Information Systems*, 13, 177-195. <https://doi.org/10.17705/1CAIS.01315>
- Newman, S. (2021). *Building Microservices. Designing Fine-Grained Systems (2.^a ed.)*. O'Reilly Media.
- Nguyen, M.-T., Phan, V.-T., Tran, H.-L., & Nguyen Vu, M.-A. (2026). An Explainable XGBoost Model for Predicting Cost Overruns in Construction Projects. *Asian Journal of Civil Engineering*, 27(7), 3611-3628. <https://doi.org/10.1007/s42107-026-01686-8>
- Olaniran, O. J., Love, P. E. D., Edwards, D., Olatunji, O. A., & Matthews, J. (2015). Cost Overruns in Hydrocarbon Megaprojects: A Critical Review and Implications for Research. *Project Management Journal*, 46(6), 126-138. <https://doi.org/10.1002/pmj.21556>

- Olawale, Y. A., & Sun, M. (2010). Cost and Time Control of Construction Projects: Inhibiting Factors and Mitigating Measures in Practice. *Construction Management and Economics*, 28(5), 509-526. <https://doi.org/10.1080/01446191003674519>
- Olsen, K. A., & Sætre, P. (2007). ERP for SMEs – Is Proprietary Software an Alternative? *Business Process Management Journal*, 13(3), 379-389. <https://doi.org/10.1108/14637150710752290>
- Ottaviani, F. M., Ballesteros-Pérez, P., & Narbaev, T. (2025). Automated Machine Learning Pipeline for Robust Project Cost and Duration Forecasting. *Automation in Construction*, 178. <https://doi.org/10.1016/j.autcon.2025.106426>
- pandas development team. (s.f.). *Package Overview. Pandas 3.0.5 Documentation*. Consultado el 17 de septiembre de 2026, desde https://pandas.pydata.org/docs/getting_started/overview.html
- Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2024). Gorilla: Large Language Model Connected with Massive APIs. *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 126544-126565. <https://doi.org/10.52202/079017-4020>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-Learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Polars developers. (s.f.-a). *Lazy API. Polars User Guide*. Consultado el 17 de septiembre de 2026, desde <https://docs.pola.rs/user-guide/concepts/lazy-api/>
- Polars developers. (s.f.-b). *Polars User Guide*. Consultado el 17 de septiembre de 2026, desde <https://docs.pola.rs/>
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering. A Practitioner's Approach* (9.^a ed.). McGraw-Hill Education.
- Prisma Data, Inc. (s.f.). *What Is Prisma 7?. Prisma Documentation*. Consultado el 17 de septiembre de 2026, desde <https://www.prisma.io/docs/orm/v7>
- Project Management Institute. (2019). *The Standard for Earned Value Management*.
- Project Management Institute. (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide) and the Standard for Project Management* (7.^a ed.).
- Python Software Foundation. (s.f.). *General Python FAQ. Python 3.14.7 Documentation*. Consultado el 17 de septiembre de 2026, desde <https://docs.python.org/3/faq/general.html>
- Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., & Sun, M. (2024). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. *The Twelfth International Conference on Learning Representations (ICLR 2024)*.
- Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An Embeddable Analytical Database. *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*, 1981-1984. <https://doi.org/10.1145/3299869.3320212>
- Radosevich, B., & Halloran, J. (2025). *MCP Safety Audit: LLMs with the Model Context Protocol Allow Major Security Exploits* (2). arXiv: 2504.03767 [cs.CR]. Consultado el 17 de septiembre de 2026, desde <https://arxiv.org/abs/2504.03767>

- Ramírez, S. (s.f.-a). *FastAPI*. Consultado el 17 de septiembre de 2026, desde <https://fastapi.tiangolo.com/>
- Ramírez, S. (s.f.-b). *Run a Server Manually. FastAPI Documentation*. Consultado el 17 de septiembre de 2026, desde <https://fastapi.tiangolo.com/deployment/manually/>
- Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering: Plan and Build Robust Data Systems*. O'Reilly Media.
- Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture. An Engineering Approach*. O'Reilly Media.
- Royce, W. W. (1970). Managing the Development of Large Software Systems. *Proceedings of IEEE WESCON*, 1-9.
- Sakimura, N., Bradley, J., & Agarwal, N. (2015, septiembre). *Proof Key for Code Exchange by OAuth Public Clients* (N. Sakimura, Ed.; RFC N.º 7636). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7636>
- Sakimura, N., Bradley, J., Jones, M. B., de Medeiros, B., & Mortimore, C. (2023, diciembre). *OpenID Connect Core 1.0 Incorporating Errata Set 2* (Final Specification). OpenID Foundation. Consultado el 17 de septiembre de 2026, desde https://openid.net/specs/openid-connect-core-1_0.html
- Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., & Aroyo, L. M. (2021). "Everyone Wants to Do the Model Work, Not the Data Work": Data Cascades in High-Stakes AI. *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI '21)*, 1-15. <https://doi.org/10.1145/3411764.3445518>
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 68539-68551. <https://doi.org/10.52202/075280-2997>
- Shamim, M. M. I., Abdul Hamid, A. B. bin, Nyamasvisva, T. E., & Rafi, N. S. B. (2025). Advancement of Artificial Intelligence in Cost Estimation for Project Management Success: A Systematic Review of Machine Learning, Deep Learning, Regression, and Hybrid Models. *Modelling*, 6(2), 35. <https://doi.org/10.3390/modelling6020035>
- Shang, S., & Seddon, P. B. (2002). Assessing and Managing the Benefits of Enterprise Systems: The Business Manager's Perspective. *Information Systems Journal*, 12(4), 271-299. <https://doi.org/10.1046/j.1365-2575.2002.00132.x>
- Shen, Z., Wang, X., Zhang, Z., & Kim, H. (2025). Domain-Aware Machine Learning for Project Delay Prediction: A Rule-Constrained XGBoost Framework. *2025 10th International Conference on Computer and Information Processing Technology (ISCIPT)*, 9-13. <https://doi.org/10.1109/ISCIPT67144.2025.11265281>
- Shi, J. J., & Halpin, D. W. (2003). Enterprise Resource Planning for Construction Business Management. *Journal of Construction Engineering and Management*, 129(2), 214-221. [https://doi.org/10.1061/\(ASCE\)0733-9364\(2003\)129:2\(214\)](https://doi.org/10.1061/(ASCE)0733-9364(2003)129:2(214))
- Shmueli, G. (2010). To Explain or to Predict? *Statistical Science*, 25(3), 289-310. <https://doi.org/10.1214/10-STS330>

- Shwartz-Ziv, R., & Armon, A. (2022). Tabular Data: Deep Learning Is Not All You Need. *Information Fusion*, 81, 84-90. <https://doi.org/10.1016/j.inffus.2021.11.011>
- Skibniewski, M. J., & Ghosh, S. (2009). Determination of Key Performance Indicators with Enterprise Resource Planning Systems in Engineering Construction Firms. *Journal of Construction Engineering and Management*, 135(10), 965-978. [https://doi.org/10.1061/\(ASCE\)0733-9364\(2009\)135:10\(965\)](https://doi.org/10.1061/(ASCE)0733-9364(2009)135:10(965))
- Sokolova, M., & Lapalme, G. (2009). A Systematic Analysis of Performance Measures for Classification Tasks. *Information Processing & Management*, 45(4), 427-437. <https://doi.org/10.1016/j.ipm.2009.03.002>
- Sommerville, I. (2016). *Software Engineering* (10.^a ed.). Pearson.
- Soria Parra, D. (2025, 9 de diciembre). *MCP Joins the Agentic AI Foundation*. Consultado el 17 de septiembre de 2026, desde <https://blog.modelcontextprotocol.io/posts/2025-12-09-mcp-joins-agentic-ai-foundation/>
- Stonebraker, M., & Çetintemel, U. (2005). "One Size Fits All": An Idea Whose Time Has Come and Gone. *Proceedings of the 21st International Conference on Data Engineering (ICDE 2005)*, 2-11. <https://doi.org/10.1109/ICDE.2005.1>
- Tableau (Salesforce). (s.f.). *Web Data Connector 2.0*. Consultado el 17 de septiembre de 2026, desde https://help.tableau.com/current/pro/desktop/en-us/examples_web_data_connector.htm
- The PostgreSQL Global Development Group. (s.f.). *What Is PostgreSQL?*. *PostgreSQL 18 Documentation*. Consultado el 17 de septiembre de 2026, desde <https://www.postgresql.org/docs/current/intro-what-is.html>
- Thorgersen, S., & Silva, P. I. (2023). *Keycloak – Identity and Access Management for Modern Applications. Harness the Power of Keycloak, OpenID Connect, and OAuth 2.0 to Secure Applications* (2.^a ed.). Packt Publishing.
- Umble, E. J., Haft, R. R., & Umble, M. M. (2003). Enterprise Resource Planning: Implementation Procedures and Critical Success Factors. *European Journal of Operational Research*, 146(2), 241-257. [https://doi.org/10.1016/S0377-2217\(02\)00547-7](https://doi.org/10.1016/S0377-2217(02)00547-7)
- Vandevoorde, S., & Vanhoucke, M. (2006). A Comparison of Different Project Duration Forecasting Methods Using Earned Value Metrics. *International Journal of Project Management*, 24(4), 289-302. <https://doi.org/10.1016/j.ijproman.2005.10.004>
- Vassiliadis, P. (2009). A Survey of Extract–Transform–Load Technology. *International Journal of Data Warehousing and Mining*, 5(3), 1-27. <https://doi.org/10.4018/jdwm.2009070101>
- Waller, M. A., & Fawcett, S. E. (2013). Data Science, Predictive Analytics, and Big Data: A Revolution That Will Transform Supply Chain Design and Management. *Journal of Business Logistics*, 34(2), 77-84. <https://doi.org/10.1111/jbl.12010>
- Wang, R. Y., & Strong, D. M. (1996). Beyond Accuracy: What Data Quality Means to Data Consumers. *Journal of Management Information Systems*, 12(4), 5-33. <https://doi.org/10.1080/07421222.1996.11518099>
- Watson, H. J., & Wixom, B. H. (2007). The Current State of Business Intelligence. *Computer*, 40(9), 96-99. <https://doi.org/10.1109/MC.2007.331>

- Wauters, M., & Vanhoucke, M. (2016). A Comparative Study of Artificial Intelligence Methods for Project Duration Forecasting. *Expert Systems with Applications*, 46, 249-261. <https://doi.org/10.1016/j.eswa.2015.10.008>
- xgboost developers. (s.f.). *XGBoost Documentation. Xgboost 3.4.2 Documentation*. Consultado el 17 de septiembre de 2026, desde <https://xgboost.readthedocs.io/en/stable/>
- Yeo, K. T., & Ning, J. H. (2002). Integrating Supply Chain and Critical Chain Concepts in Engineer-Procure-Construct (EPC) Projects. *International Journal of Project Management*, 20(4), 253-262. [https://doi.org/10.1016/S0263-7863\(01\)00021-7](https://doi.org/10.1016/S0263-7863(01)00021-7)